PORTABLE INDOOR FEEDBACK

REPIF

MAY 11, 2026

AIMAN AYASH
AyaAi172

- Workshop user stories

A1- PVC Holder

Informal Description

As a technician, I want to build a PVC holder for the touch display and the Raspberry Pi so that both components can be mounted safely inside the station.

1.1.1 Requirements

Must:

- Make the PVC display holder
- Make the PVC base plate
- Drill all holes with the correct drill sizes
- Deburr all holes and edges
- Make the ribbon-cable opening
- Mount the display with 5 mm spacers
- Mount the Raspberry Pi with 5 mm spacers
- Screw the two PVC plates together
- Mark the plates with my IAM code

Should:

- Make all cuts and holes clean

Could:

- Draw measurement lines to avoid mistakes

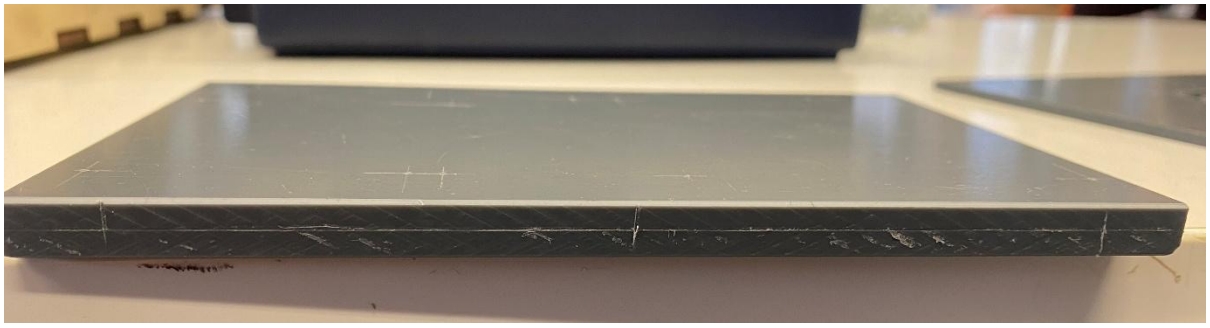
1.1.2 Introduction

The PVC holder is made of two parts: a **front plate** for the touch display and a **base plate** for the Raspberry Pi.
Both parts are screwed together to make the inner frame of the station.
This frame keeps all components stable when the station is placed inside the wooden housing.

1.1.3 Fabrication Steps

- *Drawing the Hole Points (Center Marking)*

Before drilling anything, I marked all hole positions using a ruler and a pencil. I checked my measurements twice to avoid mistakes.



1.1.4 Drilling the Holes

After the marks were done, I drilled all holes:

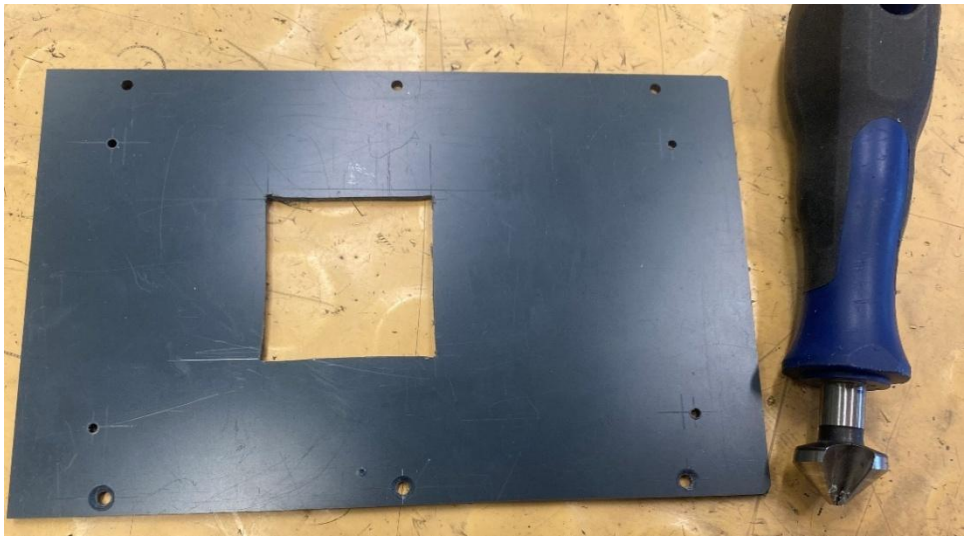
- 2.5 mm holes for Raspberry Pi screws
- 2.5 mm holes for display screws
- 2.3 mm holes for the M3 threaded holes
- 4 mm hole for the wood screw
- Holes for connecting both PVC plates

I made sure to drill straight and hold the plate firmly.

1.1.5 Deburring

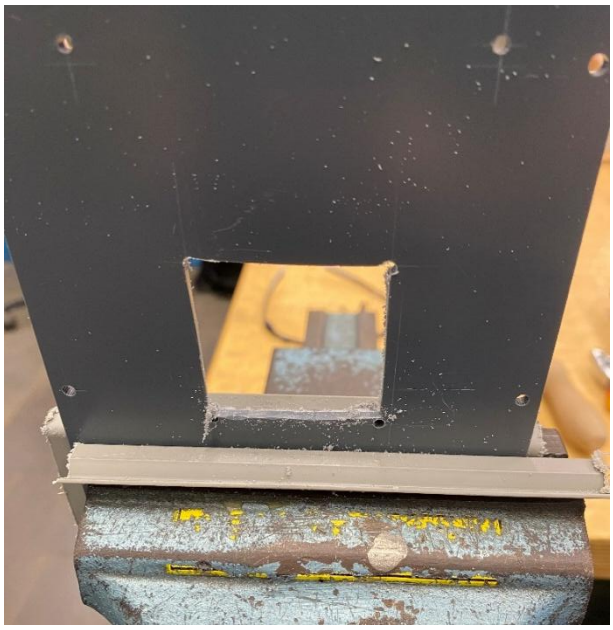
After drilling, sharp edges appeared around the holes.
I used a deburring tool and a small file to clean them.

This makes the holes smooth and prevents cracks later.



1.1.6 Making the Ribbon Cable Opening

I cut the square opening for the display ribbon cable using a saw and a file.
I made the hole slowly to keep the shape as clean as possible.



1.1.7 Mounting the Raspberry Pi

I placed the Raspberry Pi on the base plate and mounted it with:

- 5 mm spacers
- M2.5 screws

- Washers and nuts

The Raspberry Pi sits firmly without wobbling.

1.1.8 Mounting the Display

I attached the touch display to the front PVC plate using the spacer sleeves.
Then I connected the ribbon cable through the opening.

1.1.9 Mistakes Encountered

- *Drilling Mistakes*

At first, I drilled some holes in the wrong place.
Because of that, a few components did not line up correctly.
I fixed the problem by re-measuring the points and drilling the holes again in the correct position.

- *Missing Documentation Photos*

Another mistake I made was not taking enough photos during the process.
This made the documentation a bit harder to complete, but I used the photos I had and explained the steps as clearly as possible.

1.1.10 Completion

The PVC holder is now fully finished.
Both parts were manufactured, all holes were drilled and deburred, the opening was made, and the Raspberry Pi and display were mounted correctly.
The frame is stable and ready to be installed inside the wooden housing.


AYaAi172
AYaAi

A2- Printed circuit board

Introduction

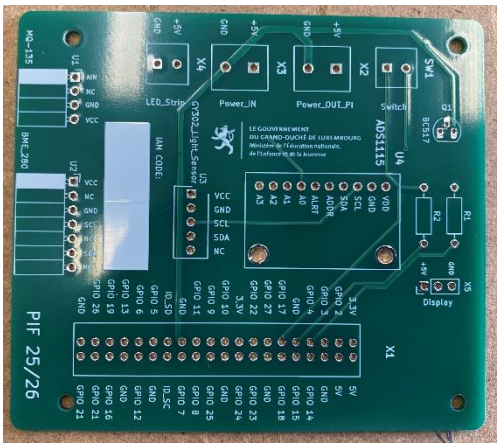
In this user story, a printed circuit board (PCB) is assembled for the Portable Indoor Feedback Station.
The PCB is used as an interface between the Raspberry Pi, the sensors, and other components. All required components were soldered onto the PCB and the finished board was mounted onto the PVC base plate.

1.1.11 Work Steps

User Story	**A2- Printed circuit board**	**to**	**Calendar week 51**
Informal description	As the operator of Portable Indoor Feedback		
	… I would like to manufacture a printed circuit board that serves as an interface between the Raspberry, sensors, and actuators		
	… so that I can neatly integrate my sensors for recording values into my station.		
Requirements according to the work order	**Must** • The necessary components are soldered onto the circuit board. • The circuit board is screwed onto the PVC base plate. • <u>Documentation:</u> ○ Work steps ○ List of materials		
Tasks according to work order			
• The components are selected according to the diagram (except for sensors). • The components are professionally mounted on the circuit board and soldered. • All safety regulations for carrying out the work must be observed. • The circuit board is screwed onto the PVC base plate using the **5 mm** spacers. • Marking the circuit board with the IAM code			

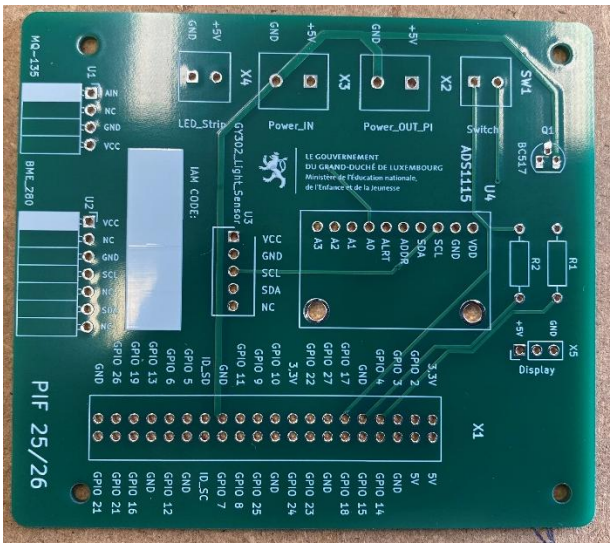
1.1.12 List of Materials

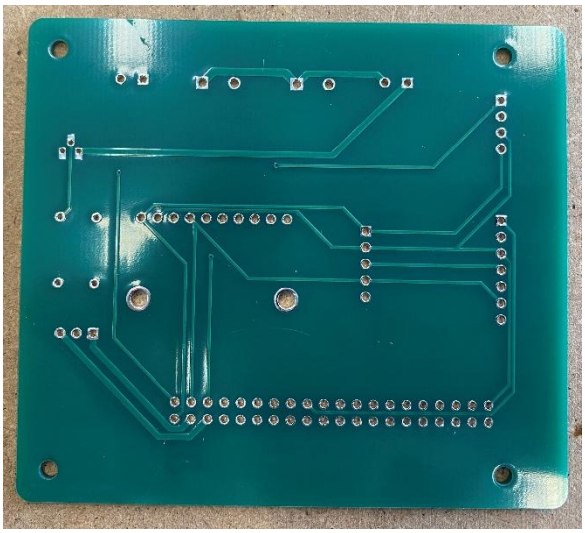
- Printed circuit board (PCB)
- Female header
- Pin headers
- Screw terminals
- Resistors: R1 (10 kΩ) , R2 (56 kΩ)
- ADS1115
- BC517 transistor
- Other connectors according to the PCB layout
- Solder
- Screws
- 5 mm spacer sleeves
- PVC base plate

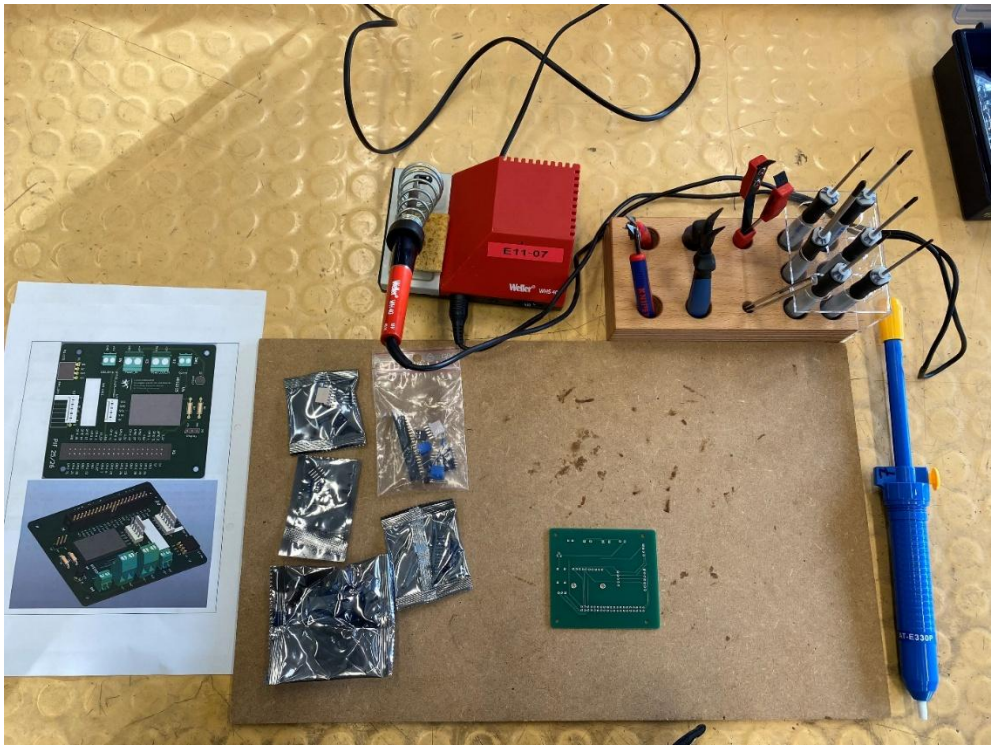


1.1.13 Preparing the Circuit Board

First, the printed circuit board was checked for damage.
During preparation, we noticed that a **female header** was missing.
We informed the teacher and asked for the missing component before continuing the work.
After all components were available, the work could continue







1.1.14 Placing the Components

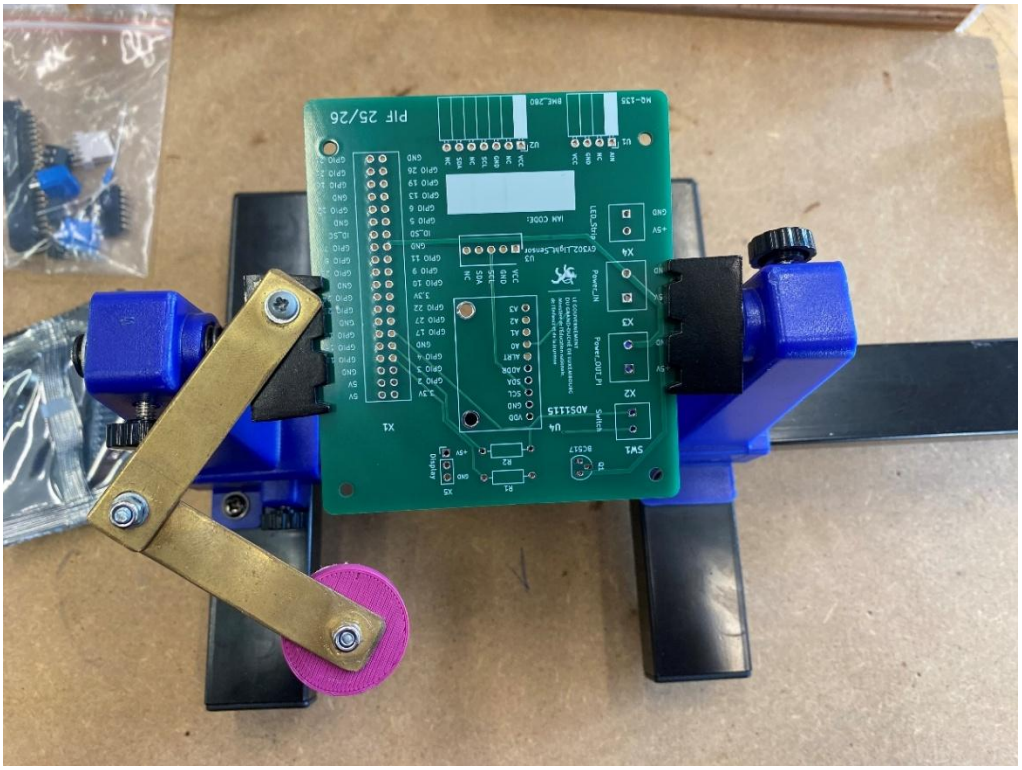
As learned in class, soldering was done starting with the **smallest components first**.

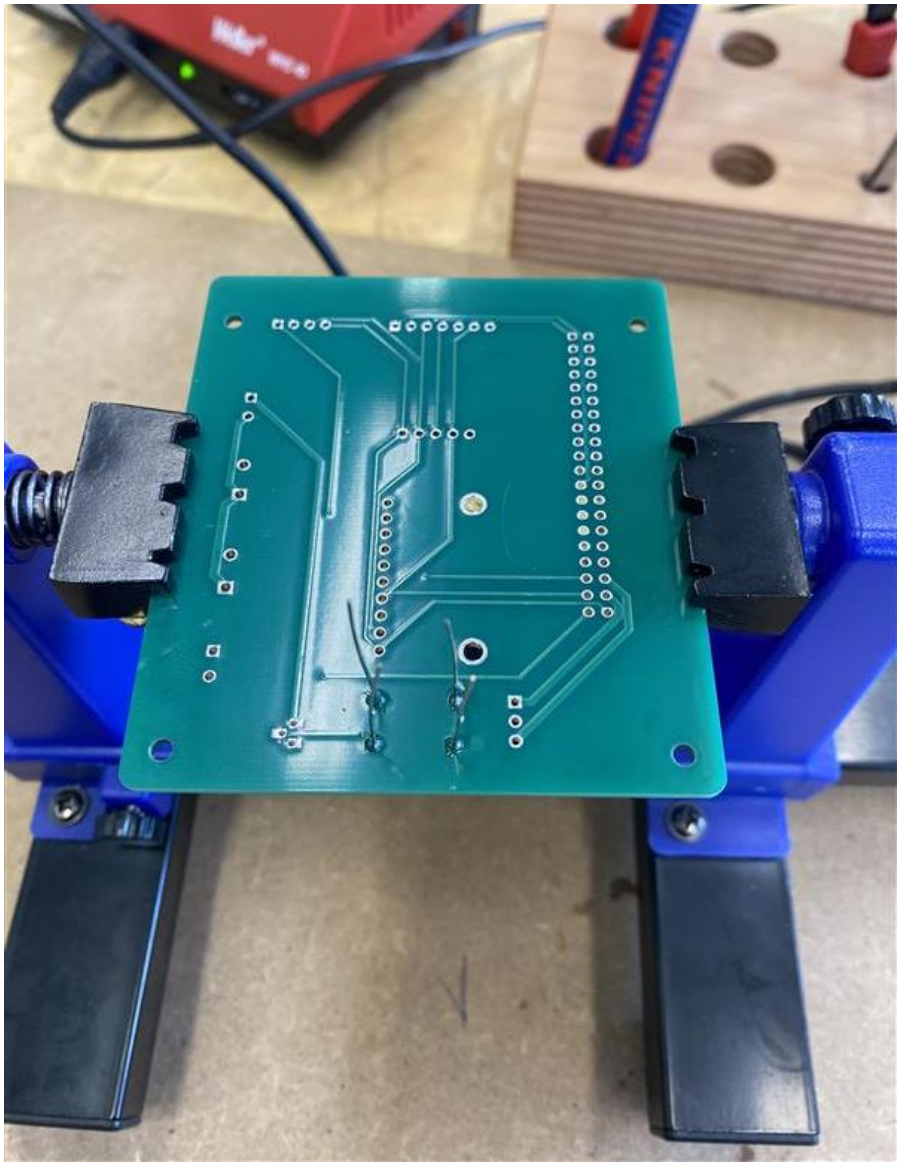
We started with the **resistors**, after checking their values:

- **R1 = 10 kΩ**
- **R2 = 56 kΩ**

To make soldering easier, a **helping hands tool** was used to hold the circuit board in place.
This tool keeps the PCB stable and has supports that press the components slightly, so they do not move, fall out, or shake while soldering.

After the resistors, the remaining components such as pin headers, screw terminals, connectors, and the female header were placed on the PCB.

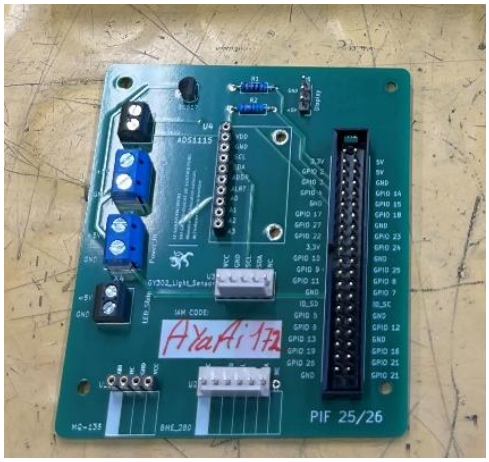


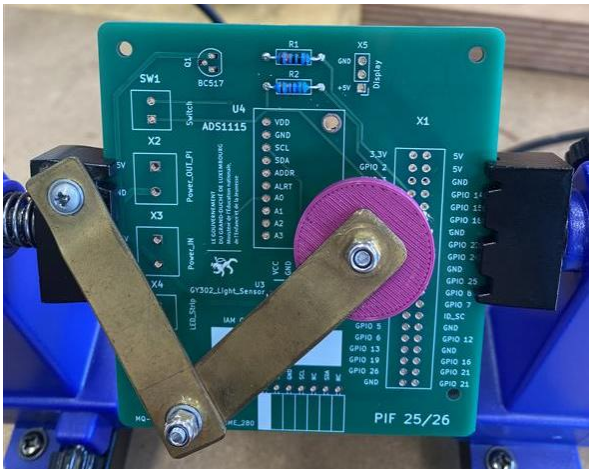


1.1.15 Soldering the Components

After placing the components, they were soldered onto the circuit board.
The soldering was done carefully and step by step.
All safety rules were followed during soldering.
Each solder joint was checked to make sure it was clean and properly connected.

After soldewring we checked for cold solder joints and solder bridges.
Any small issues were corrected before continuing.





1.1.16 Mounting the Circuit Board

The finished circuit board was screwed onto the PVC base plate using **5 mm spacer sleeves** and screws.
This creates enough space between the PCB and the PVC plate and prevents short circuits.


PIF 25/26

1.1.17 Completion

The printed circuit board was successfully assembled and soldered.
All required components were mounted, and the PCB was fixed onto the PVC base plate using spacer sleeves.
The circuit board is now ready to be connected to the Raspberry Pi and sensors.

A3 - Power supply

<table>
<tr><td>User Story</td><td>A3 - Power supply</td><td>to</td><td>KW 03</td></tr>
<tr><td colspan="2">Informal description</td><td colspan="2">As the operator of the Portable Indoor Feedback Station,</td></tr>
<tr><td colspan="4">… I would like to prepare a power supply,</td></tr>
<tr><td colspan="4">… so that the power cable is prepared correctly.</td></tr>
<tr><td colspan="2">Requirements according to the work order</td><td colspan="2">Must
The conductor strands are separated from each other.
The conductor strands are insulated.
Documentation is provided:
Work steps
Materials</td></tr>
<tr><td colspan="4">Tasks according to work order</td></tr>
<tr><td colspan="4">Cut the cable into two halves.
Strip both cable ends.
Separate the inner conductors.
Twist the shield (ground) braid into one strand.
Place heat-shrink tubing on the wires and shield braid.
Tin the wire ends and the shield braid using solder.
Shrink the heat-shrink tubing using a heat gun.</td></tr>
</table>

1.1.1 Power Supply

1.1.1.1 Objective

The goal of this user story is to prepare a power supply cable for the Portable Indoor Feedback Station.
The cable is prepared by separating, insulating, and securing the conductors.

1.1.1.2 Design Decision and Deviation from Work Order

The original Raspberry Pi power supply was **not used**.
On explicit instruction of the supervising teacher, an **alternative multi-core shielded cable** was prepared instead.
This avoided unnecessary modification of the original power supply and allowed correct demonstration of cable preparation.

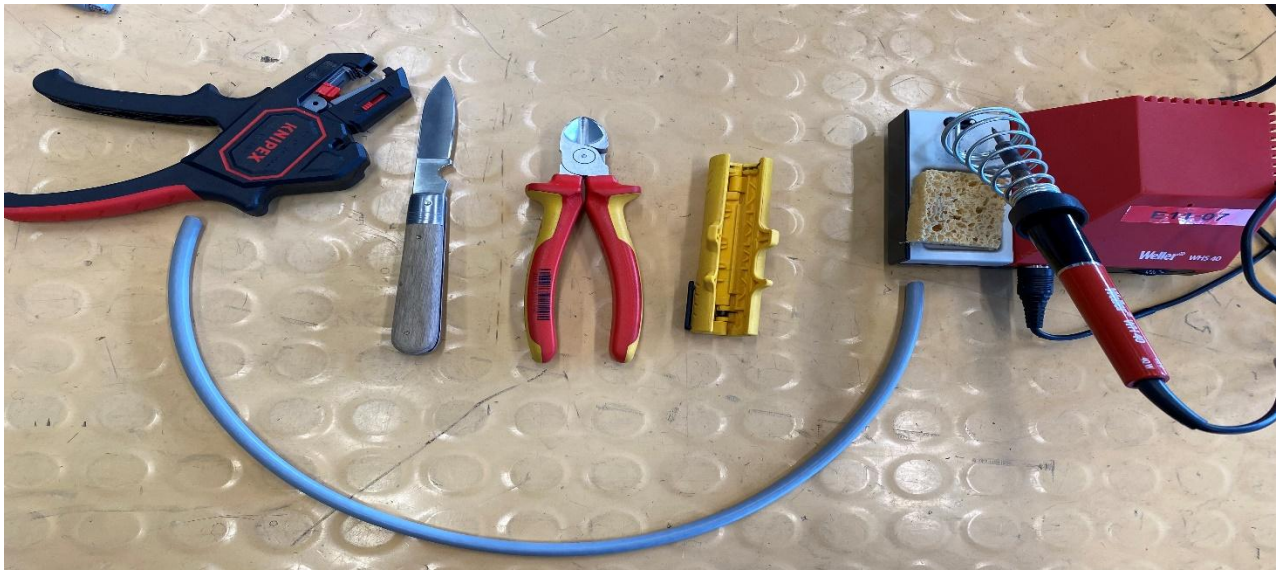
1.1.1.3 Materials Used

Materials

- Multi-core shielded cable
- Heat-shrink tubing

Tools

- Wire stripper
- Side cutters
- Cable stripping tool
- Heat gun



1.1.1.4 Safety Scope

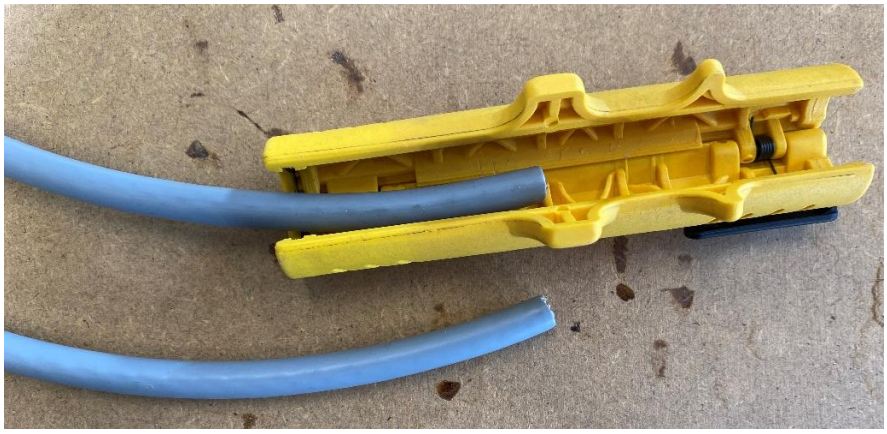
- Low-voltage DC only
- No mains voltage used
- Cable not connected to a power source during this step

1.1.1.5 Work Steps

- Cut the cable into two halves

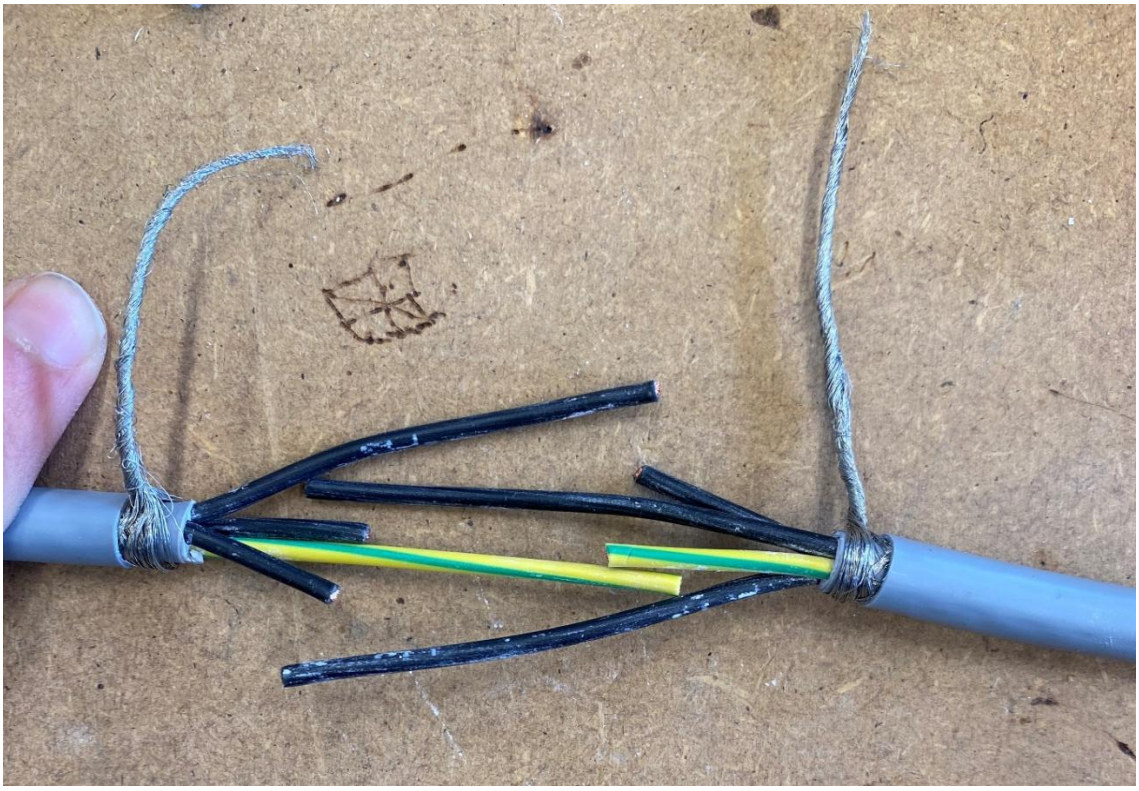
  The grey multi-core cable was cut into two separate parts.

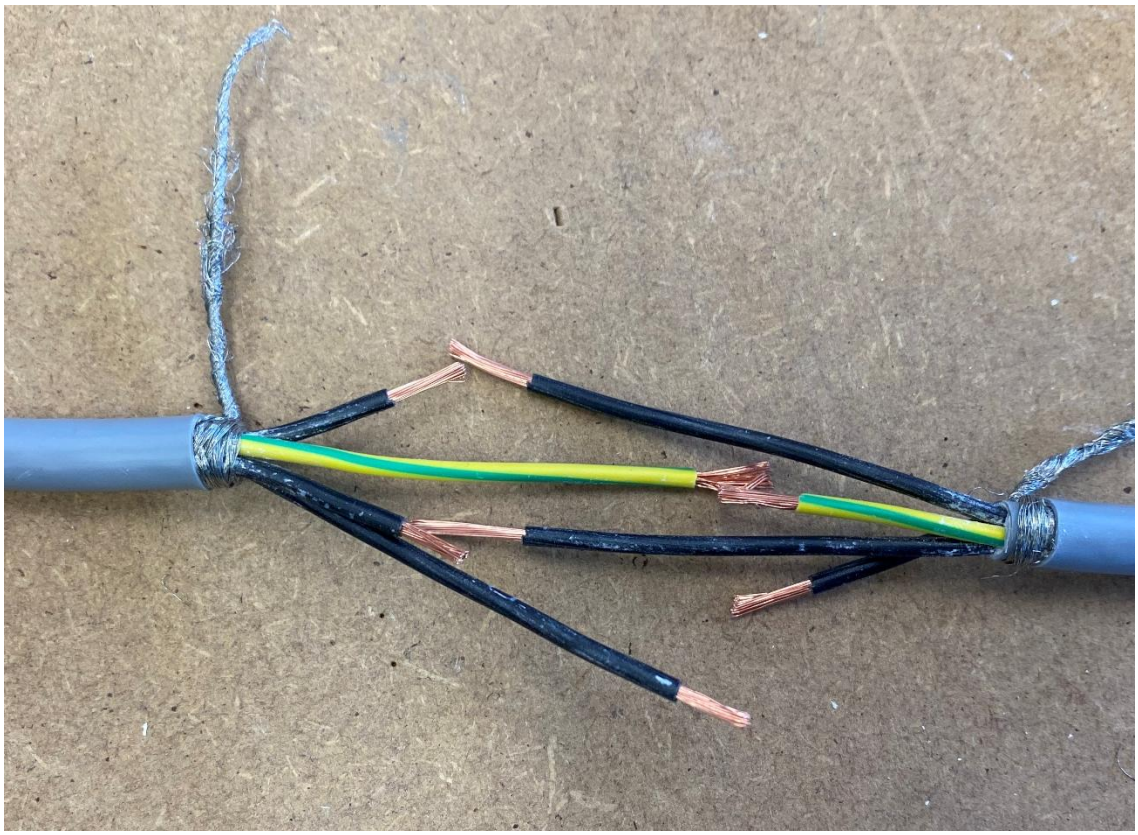
- Remove the outer insulation
  The grey outer sheath was stripped on both cable ends using a cable stripping tool.



- Expose and separate the shield braid
  After removing the sheath, the metal shield braid became visible.
  The braid was pulled out and separated from the inner wires.

- Separate the inner wires and strip the wire ends
  The inner wires (black wires and one yellow/green wire) were separated.
  The insulation on the black wires was stripped to expose the copper ends.





- Place heat-shrink tubing and tin the wire ends
  Heat-shrink tubing was placed on the wires and the shield braid before soldering. Solder was then melted onto the ends of all wires and the shield braid to ensure reliable electrical contact.

- Shrink the heat-shrink tubing
  After finishing the preparation, the heat-shrink tubing was tightened using a heat gun.

- Conclusion:

The power supply cable was successfully prepared according to User Story A3.
All conductors were separated, tinned, and insulated correctly.
An alternative multi-core shielded cable was used on the instructions of the supervising teacher.
The task was completed safely and as documented.

A4 - Sensors 1

User Story	**A4 - Sensors 1**	**to**	**KW 05**
Informal description	As the operator of Portable Infoor Feedback		
	... I would like to connect and test the MQ135 gas sensor		
	... so that I can obtain data on air quality.		

Requirements according to the work order	**Must**
	• Solder the pin headers to the MQ135 sensor module.
	• Solder the pin headers to the ADS1115 16-bit converter module.
	• Circuit setup on Digilab, directly on the Raspberry or the circuit board.
	• Determine and configure the I2C address of the ADS1115.
	• Function test using the sample program (MQ135/test.py).
	• Documentation
	  ○ Work steps
	  ○ Materials list
	  ○ Video of how it works		

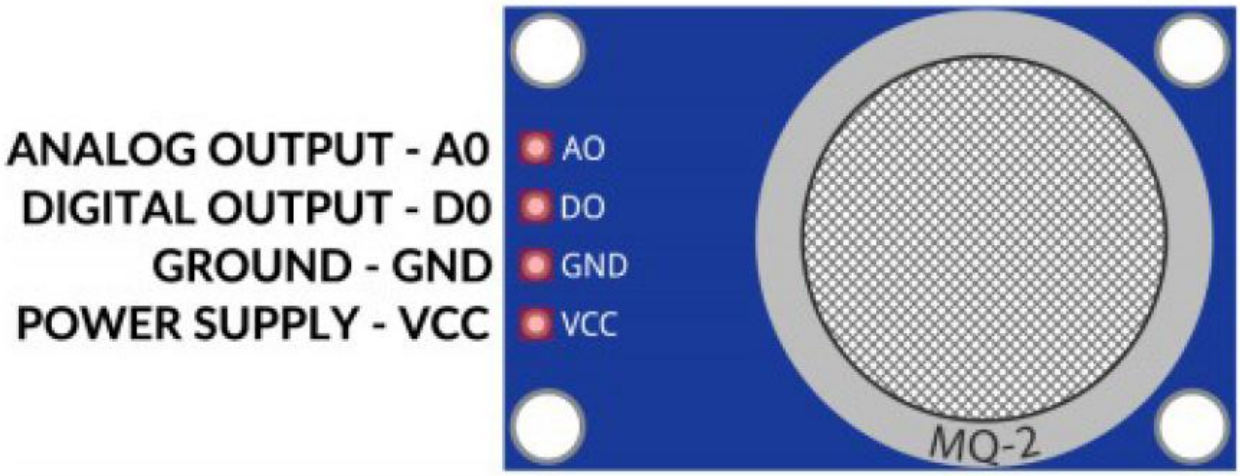
Tasks according to work order			

• Checking the necessary components (modules)
• Soldering the pin headers on both modules
• Assemble the circuit according to the diagram on the Digilab breadboard or in the designated slots on the circuit board.
• Determining the I2C address and inserting it into the configuration file.
• Loading the sample program and reading out the values.
• Create documentation:
  ○ List of materials
  ○ Photo or wiring diagram of the circuit on the Digilab
  ○ Link to a video showing the circuit functioning correctly (reading the values)
  ○ Explain any difficulties/problems that may have arisen and indicate how they were solved.			

Resources
The gas sensor module has four pins.The pinout is shown in the following diagram:

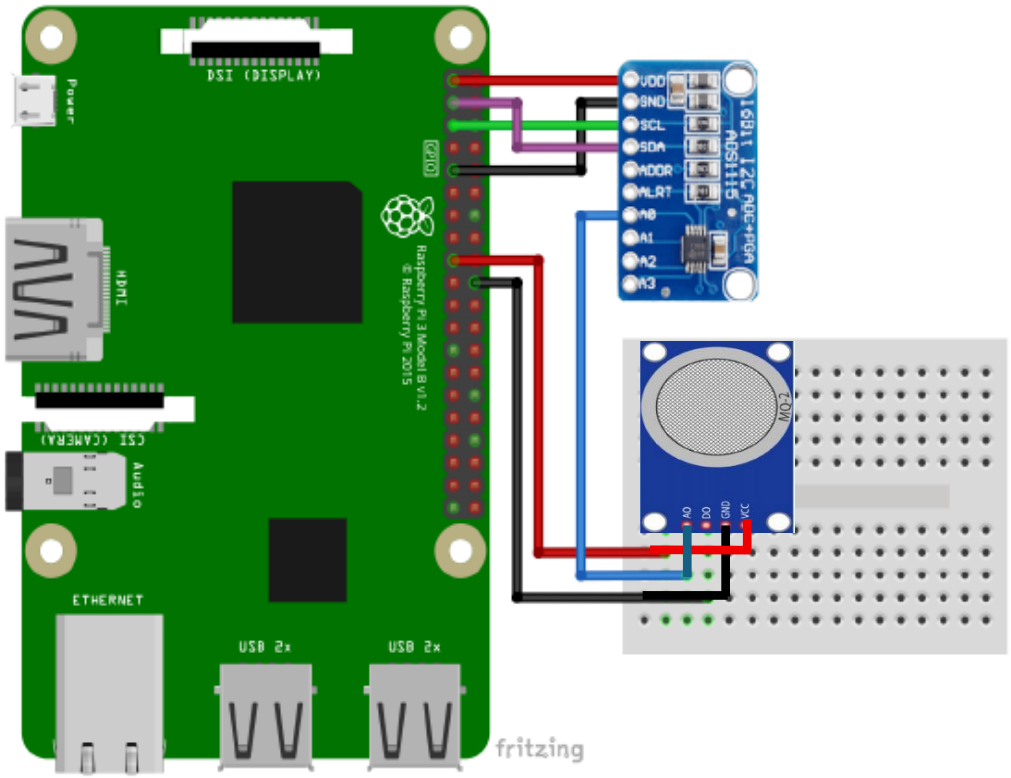
Die Pinbelegung

Das Gassensormodul hat vier Stifte. Die Pinbelegung ist in der folgenden Abbildung dargestellt:



HINWEIS: Der Raspberry Pi verfügt nicht über einen Digital-Analog-Wandler und kann nicht zum Lesen analoger Spannungen verwendet werden.

Note: The Raspberry Pi does not have a digital -to-analog converter and cannot be used to read analog voltages.



Modul Pin	**>**	**Raspberry Pin**		
VDD	>	3.3V	[pin 1]	Roter Draht
GND	>	GND	[pin 9]	Schwarzer Draht
SCL	>	GPIO 3	[pin 5]	Grüner Draht
SDA	>	GPIO 2	[pin 3]	Violetter Draht
Modul Pin	**>**	**MQ-135 gas**		
A0	>	Left pin		Blauer Draht
RaspPi Pin	**>**	**MQ-135 gas**		
3.3V	[pin 17]	>	Third pin	Red wire
GND	[pin 20]	>	Right pin	Schwarzer Draht

Task Description

The objective of this task was to connect and test the **MQ-135 gas sensor** using the **ADS1115 16-bit analog-to-digital converter**, mounted on the station's circuit board and interfaced with a **Raspberry Pi**.
The goal was to obtain air-quality–related measurement values and verify correct hardware integration, I^2C communication, and software functionality.

Because the Raspberry Pi does not support analog signal input, the ADS1115 ADC is required to convert the analog output of the MQ-135 sensor into a digital signal that can be processed by the system.

Workshop – Sensor Assembly

1.1 Components Used

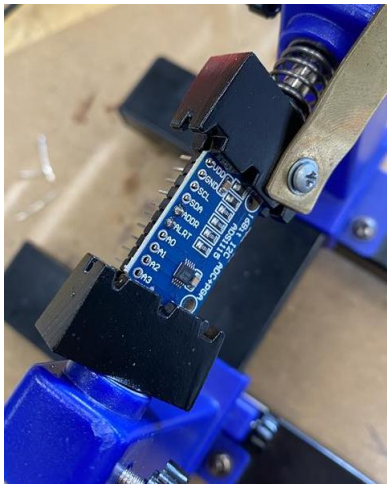
- MQ-135 gas sensor module
- DS1115 16-bit ADC module (I^2C)
- Pin headers
- Custom PIF circuit board
- Soldering iron and solder
- Third-hand soldering too

1.2 Soldering the Modules

Pin headers were soldered onto the ADS1115 ADC module to allow stable mounting on the circuit board.
Care was taken to ensure:

- Correct pin alignment
- Clean solder joints
- No short circuits between adjacent pins

The soldering process was performed using a third-hand tool to keep the module stable during soldering.

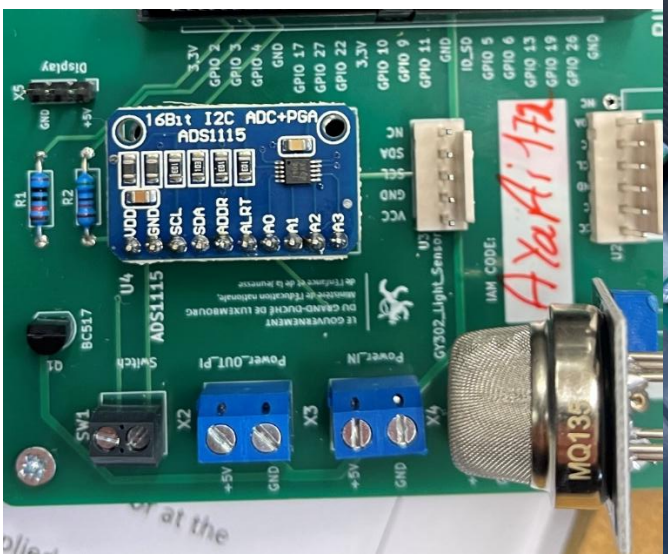


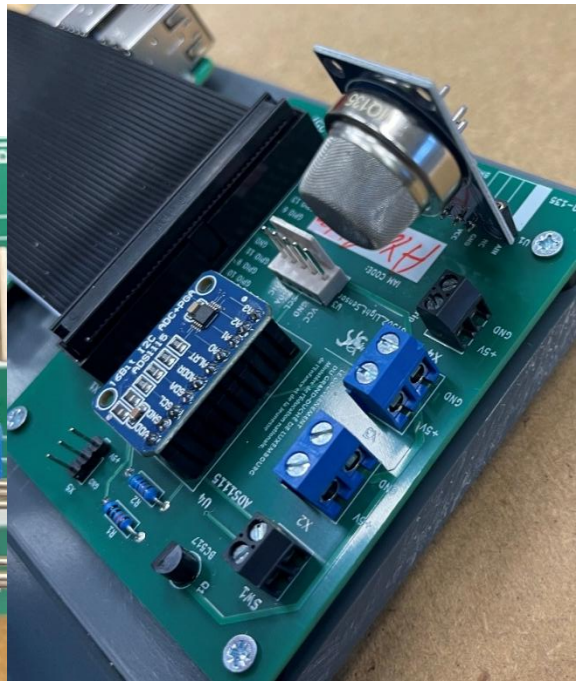
1.3 Mechanical Integration

After soldering, the ADS1115 module was mounted onto its designated socket on the station PCB. The MQ-135 gas sensor module was also inserted into its corresponding connector on the PCB.

The circuit board handles:

- Power distribution
- Analog signal routing from the MQ-135 to the ADC
- I^2C communication between the ADS1115 and the Raspberry Pi via the GPIO connector





Lab – Electrical Configuration and Software Setup

1.4 I²C Tools Installation

To detect connected I²C devices, the required tools were installed on the Raspberry Pi:

```
ayaai172@ayaai172:~ $ sudo apt-get install i2c-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
i2c-tools is already the newest version (4.4-2).
i2c-tools set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 193 not upgraded.
```

The package was already present and up to date on the system.

1.5 Detecting I²C Addresses

All connected sensors were scanned using:

```
ayaai172@ayaai172:~ $ sudo i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- 23 -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- 48 -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- 76 --
```

The following I²C addresses were detected:

- **0x23** → BH1750 light sensor
- **0x48** → ADS1115 (used for MQ-135)
- **0x76** → BME280 environmental sensor

This confirmed correct electrical wiring and functional I²C communication.

1.6 Configuration File Setup

The detected I²C addresses were inserted into the station configuration file (`config.toml`):

```
ayaai172@ayaai172:~/RPIF1/Ressourcen/Ressourcen/Firmware und Scripts $ ls
bh1750  config.toml  mq135           test-bme280.py
bme280  measure.py   test-bh1750.py  test-mq135.py
ayaai172@ayaai172:~/RPIF1/Ressourcen/Ressourcen/Firmware und Scripts $
```

```
ayaai172@ayaai172:~/RPIF1/Ressourcen/Ressourcen/Firmware und Scripts $ sudo nano config.toml
  GNU nano 8.4                                                    config.toml
#station_serial =                  # Serial number of the station

interval = 2                       # Time between measurements

toggle_intensity = 100             # The light level at which the LED should be turned on

[i2c]                              # I2C-Addresses of the sensors
#bme280 =
#bh1750 =
#mq135 =

[destination]                      # Information about the server
#ip =                             # IP-Address or Hostname of the Server
#path =                           # Path to the script that accepts the measurements
```

```
[i2c]
bme280 = 0x76
bh1750 = 0x23
mq135  = 0x48
```

This step is necessary for the firmware to correctly initialize and read the sensors.

1.7 Firmware Verification

The main measurement script (measure.py) was reviewed to ensure:

- Proper initialization of the MQ-135 sensor via the ADS1115
- Correct exception handling when sensors are unavailable
- Continuous data acquisition during runtime

The script reads the analog gas sensor values through the ADC and processes them as gas concentration values (CO_2-equivalent).

```
ayaail72@ayaail72:~/RPIF1/Ressourcen/Ressourcen/Firmware und Scripts $ cat measure.py
```

```
        try:
            bme280 = BME280(bus=bus, address=cfg['i2c']['bme280'])
        except Exception:
            print("Connection to BME280 failed, switching to 'no-sensor-mode'.", file=sys.stderr)
            options.sensors = False
        try:
            bh1750 = BH1750(bus=bus, address=cfg['i2c']['bh1750'])
        except Exception:
            print("Connection to BH1750 failed, switching to 'no-sensor-mode'.", file=sys.stderr)
            options.sensors = False
        try:
            mq135 = MQ135(bus=bus, address=cfg['i2c']['mq135'])
        except Exception:
            print("Connection to ADC failed, switching to 'no-sensor-mode'.", file=sys.stderr)
            options.sensors = False

    # Collect data until the script is stopped
    while True:
        try:
            # Read the next datum and store it in the corresponding array
            if (options.sensors):
                light = bh1750.read_light()
                temperature = bme280.read_temperature()
                pressure = bme280.read_pressure()
                humidity = bme280.read_humidity()
                gas = mq135.read_CO2()
            else:
                light = 0
                temperature = 20
                pressure = 1000
                humidity = 50
                gas = 500

            timestamp = datetime.now()

            # If the current light level is below the configured threshold, turn on the light
            if light < cfg['toggle_intensity']:
                GPIO.output(4, GPIO.HIGH)
            else:
                GPIO.output(4, GPIO.LOW)

            # Output data to standard output
            if options.verbose:
                if options.pretty:
                    print(f"{timestamp}: {light:05.2f} Lux {temperature:05.2f} C {pressure:05.2f} hPa {hu
midity:05.2f} % {gas:05.3f} ppm")
                else:
                    print(f"{timestamp}, {light:05.2f}, {temperature:05.2f}, {pressure:05.2f}, {humidity:
05.2f}, {gas:05.3f}", flush=True)
```

1.8 Sensor Test Program

To test the MQ-135 sensor independently, the provided test script was configured with the correct I^2C address:

```
ayaail72@ayaail72:/opt/station $ sudo nano test-mq135.py
  GNU nano 8.4                        test-mq135.py

ADDRESS = 0x48 # TODO: Change this

from mq135 import MQ135
import smbus
def main():
    bus = smbus.SMBus(1)  # Use 1 for Raspberry Pi, 0 for older models
    sensor = MQ135(bus, ADDRESS)
```

The test was executed using:

```
ayaail72@ayaail72:/opt/station $ python test-mq135.py
CO2 concentration: 3401 ppm
```

The script returned continuously changing ppm values, confirming:

- Correct ADC communication
- Functional MQ-135 sensor output
- Proper software configuration

Testing and Results

The system successfully produced live gas sensor readings expressed in **ppm (CO_2-equivalent values)**.
A video recording was created showing:

- The assembled circuit
- The running test script
- Live measurement output in the terminal

This confirms that:

- The hardware is correctly assembled
- The ADS1115 properly converts the analog signal
- I^2C communication works as intended

- The sensor provides usable measurement data

  A video demonstrating the assembled circuit and live sensor readings during operation is provided as supplementary material

Problems and Solutions

Problem:

Initially, the MQ-135 sensor could not be read because the I^2C address was not configured in the `config.toml` file.

Solution:

The correct address was identified using `i2cdetect -y 1` and added to the configuration file. After restarting the test script, the sensor values were read correctly.

No further issues occurred during testing.

Conclusion

The MQ-135 gas sensor was successfully connected, configured, and tested using the ADS1115 ADC and Raspberry Pi.
Both the workshop (hardware assembly) and lab (software and electrical configuration) objectives were achieved. The system is now capable of reading air-quality-related gas values and integrating them into the station's measurement workflow.

All required documentation, images, terminal outputs, and a functional demonstration video are provided.

Materials Used

- MQ-135 gas sensor module
- ADS1115 16-bit ADC module
- Raspberry Pi
- Custom PIF circuit board
- Pin headers
- Soldering iron and solder

A5 - Sensors 2

<table>
<tr><td>User Story</td><td>A5 - Sensors 2</td><td>to</td><td>KW 09</td></tr>
<tr><td rowspan="3">Informal description</td><td colspan="3">As the operator of Portable Indoor Feedback</td></tr>
<tr><td colspan="3">… I would like to connect and test the light sensor and the environmental sensor</td></tr>
<tr><td colspan="3">… so that I can obtain data on temperature, air pressure, and brightness.</td></tr>
<tr><td>Requirements according to the work order</td><td colspan="3">Must
Solder the socket strip with cable to the light sensor module.
Solder the pin headers to the environmental sensor module.
Circuit setup on Digilab or the circuit board.
Determine and configure the I2C addresses.
Function test using the test programs (BME280/test.py, LH1750/test.py).
<u>Documentation:</u>
Work steps
List of materials
Photo or wiring diagram of the circuit at Digilab
Video of the circuit functioning correctly (reading out the values)
Explain any difficulties/problems that may have arisen and indicate how they were solved.</td></tr>
<tr><td colspan="4">Tasks according to the work assignment</td></tr>
<tr><td colspan="4">Check the necessary components (modules).
Solder the socket strip with cable to the light sensor module (BH1750).
Solder the socket strip with the cable to the barometric sensor module (BME280).
Assemble the circuit according to the diagram on the Digilab breadboard or at the designated slots on the circuit board.</td></tr>
</table>

- Tests on the breadboard can be carried out using the supplied plug-in cables (see photo in the appendix).

- Function test using the test programs (BME280/test.py, LH1750/test.py).

Resources


ADDR
SDA
SCL
GND
VCC
GY-302

Workshop part:

Objective

The objective of this workshop task was to prepare and integrate the light sensor (GY-302 / BH1750) and the environmental sensor (BMP280) for use in the Portable Indoor Feedback Station. This included soldering the required pin headers and connector cables, assembling the wiring harness, and integrating the modules onto the PCB.

Materials Used

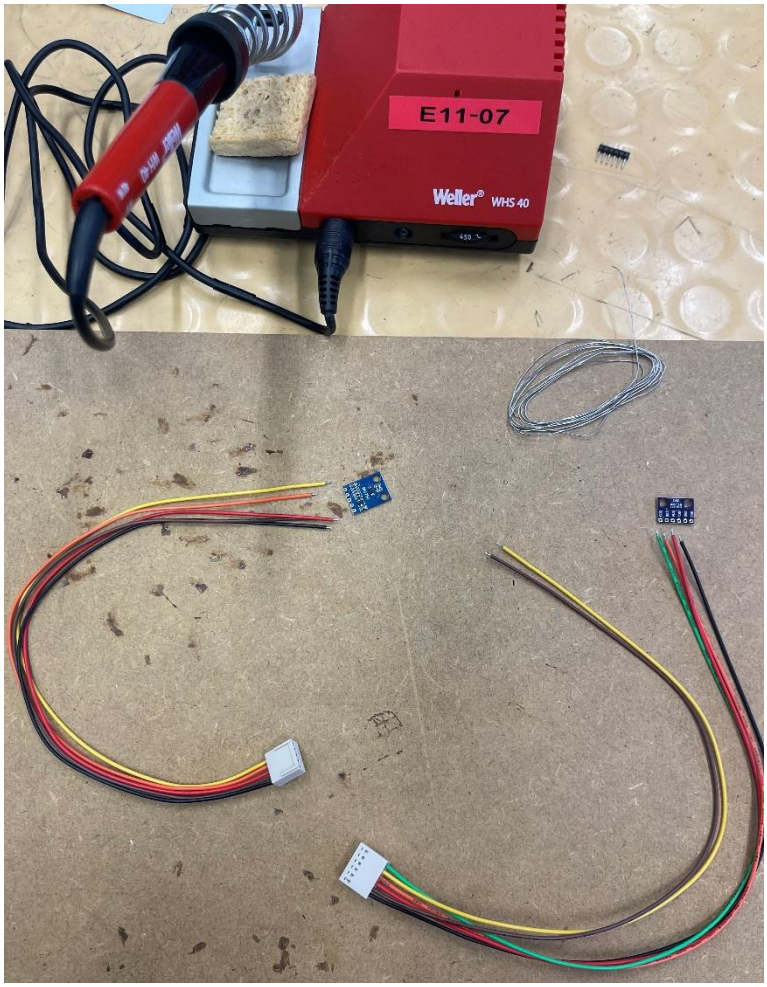
- GY-302 Light Sensor Module (BH1750)
- BMP280 Environmental Sensor Module
- 4-pin socket strip with cable
- 4-pin header pins
- Solder wire (lead-free)
- Heat shrink tubing
- Cable ties
- Weller WHS 40 soldering station
- Multimeter
- Raspberry Pi PCB with I2C connectors

Preparation

Before starting with the soldering, I first checked all required components. I verified that both sensor modules (BH1750 and BMP280) were complete and that the pin labels (VCC, GND, SDA, SCL) were clearly visible on the boards.

I prepared the soldering workstation by switching on the soldering station and cleaning the soldering tip with the sponge. I also placed the sensors and cables on the table in a way that I could easily reach them during soldering.

Before soldering, I made sure that I knew which pins had to be connected and checked the pin assignment once again to avoid connecting VCC and GND incorrectly.



Soldering Process

- ### BH1750 Light Sensor

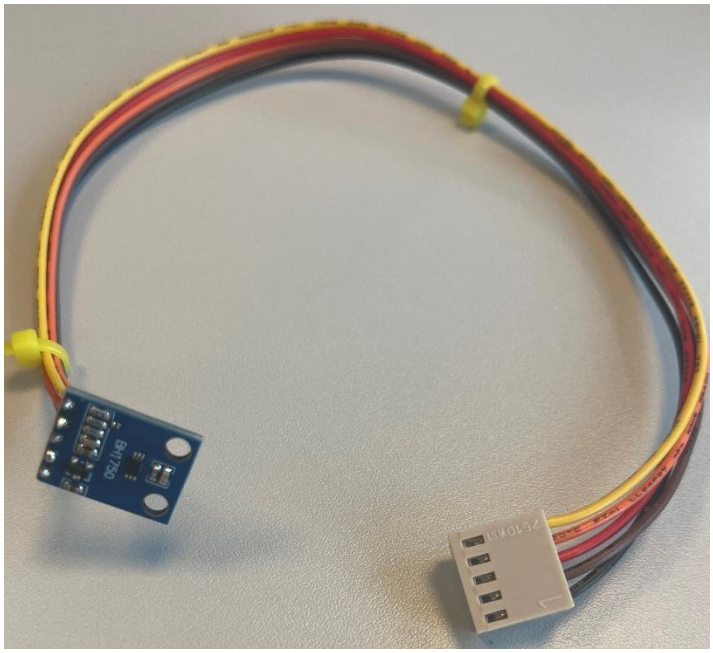
For the BH1750 light sensor, I soldered a 4-pin cable to the module. Before soldering, I checked the pin labels (VCC, GND, SDA, SCL) to make sure the wires were connected correctly.

Each pin was soldered carefully to avoid short circuits. After finishing, I checked the solder joints visually to see if everything was clean and stable.

The sensor was later connected to the PCB with:

- VCC → 5V
- GND → GND
- SDA → SDA
- SCL → SCL

I made sure that the correct voltage (5V) was used for this sensor.



- BMP280 Environmental Sensor

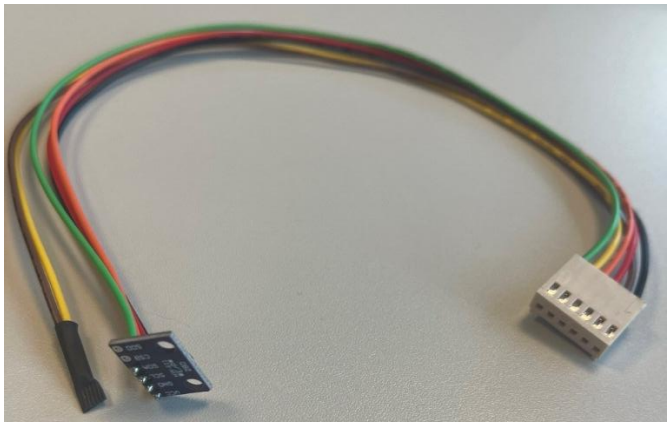
For the BMP280 sensor, I used a prepared cable with a connector. The cable originally contained more wires than required for the sensor.

The BMP280 only needs four connections:

- VCC
- GND
- SDA
- SCL

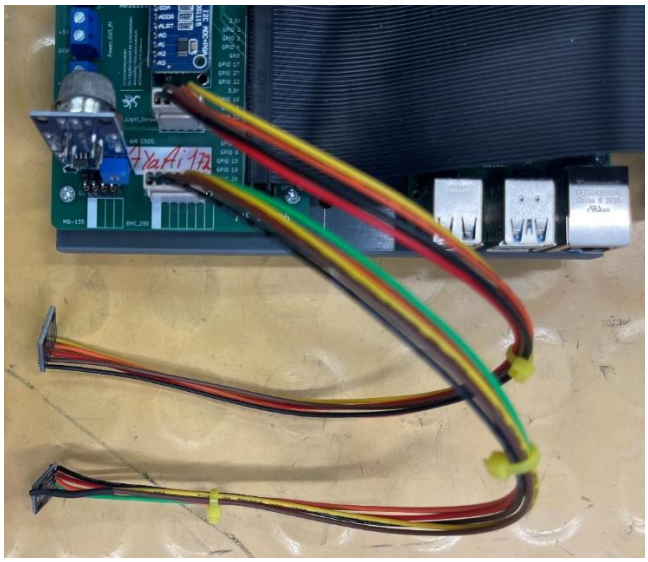
The cable, however, had additional wires that were not needed. These extra wires were not connected to the sensor. To avoid short circuits or accidental contact, the unused wires were insulated using heat shrink tubing.

After insulating the extra wires, I soldered the four required wires to the corresponding pins of the BMP280 module. I carefully checked the pin labels on the PCB (VCC, GND, SDA, SCL) to ensure correct wiring.



Cable Assembly

After soldering the header pins to both sensors, I directly connected them to the PCB using the soldered wires.



Installation on PCB

Both sensors were connected to the designated I^2C connectors on the PCB.

The SDA and SCL lines are shared between both sensors.
The BH1750 was connected to 5V, and the BMP280 was connected to 3.3V.

After connecting everything, I ensured that the cables were properly inserted and stable.

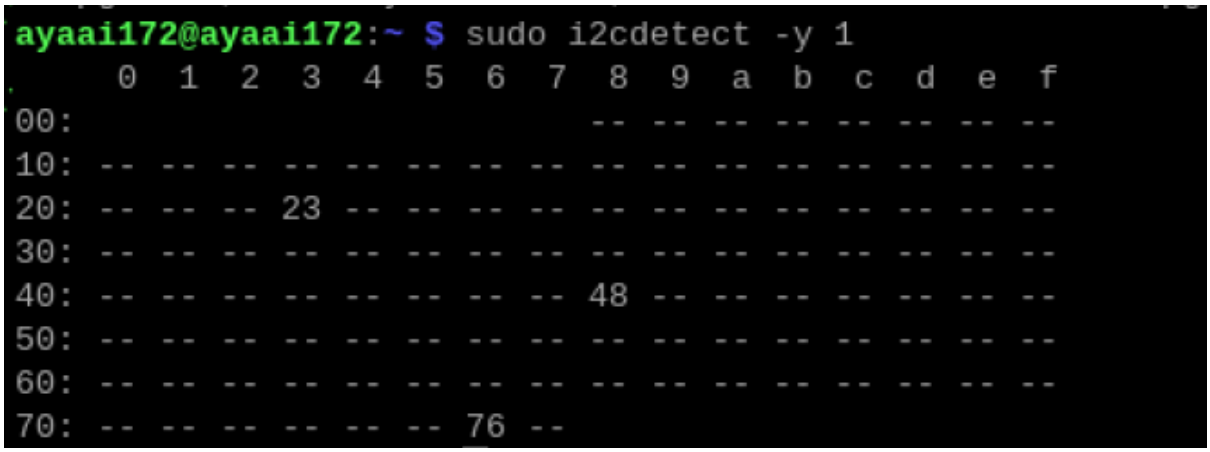
Lab Part

Objective

The objective of the lab part was to test the connected sensors, determine their I²C addresses, configure the firmware correctly, and verify that the sensors provide valid measurement data.

Detecting the I²C Addresses

First, I checked whether the sensors were detected on the I²C bus using:



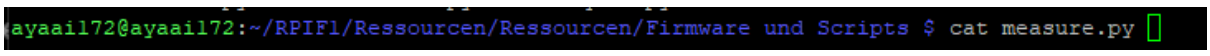
The scan showed the following addresses:

- 0x23 → BH1750 (light sensor)
- 0x48 → MQ135 via ADC
- 0x76 → BME/BMP280

This confirmed that all sensors were physically connected and recognized by the system.

Configuration of measure.py and config.toml

After detecting the addresses, I checked the measure.py file using:



I verified that the program reads the sensor addresses with the correct once.

```
        try:
            bme280 = BME280(bus=bus, address=cfg['i2c']['bme280'])
        except Exception:
            print("Connection to BME280 failed, switching to 'no-sensor-mode'.", file=sys.stderr)
            options.sensors = False
        try:
            bh1750 = BH1750(bus=bus, address=cfg['i2c']['bh1750'])
        except Exception:
            print("Connection to BH1750 failed, switching to 'no-sensor-mode'.", file=sys.stderr)
            options.sensors = False
        try:
            mq135 = MQ135(bus=bus, address=cfg['i2c']['mq135'])
        except Exception:
            print("Connection to ADC failed, switching to 'no-sensor-mode'.", file=sys.stderr)
            options.sensors = False

    # Collect data until the script is stopped
    while True:
        try:
            # Read the next datum and store it in the corresponding array
            if (options.sensors):
                light = bh1750.read_light()
                temperature = bme280.read_temperature()
                pressure = bme280.read_pressure()
                humidity = bme280.read_humidity()
                gas = mq135.read_CO2()
            else:
                light = 0
                temperature = 20
                pressure = 1000
                humidity = 50
                gas = 500

            timestamp = datetime.now()

            # If the current light level is below the configured threshold, turn on the light
            if light < cfg['toggle_intensity']:
                GPIO.output(4, GPIO.HIGH)
            else:
                GPIO.output(4, GPIO.LOW)

            # Output data to standard output
            if options.verbose:
                if options.pretty:
                    print(f"{timestamp}: {light:05.2f} Lux {temperature:05.2f} C {pressure:05.2f} hPa {hu
midity:05.2f} % {gas:05.3f} ppm")
                else:
                    print(f"{timestamp}, {light:05.2f}, {temperature:05.2f}, {pressure:05.2f}, {humidity:
05.2f}, {gas:05.3f}", flush=True)
```

In the config.toml file, the I²C addresses were commented out with #.
Therefore, I edited the file:

```
ayaail72@ayaail72:~/RPIF1/Ressourcen/Ressourcen/Firmware und Scripts $ sudo nano config.toml
  GNU nano 8.4                                                                    config.toml
#station_serial =                  # Serial number of the station

interval = 2                       # Time between measurements

toggle_intensity = 100             # The light level at which the LED should be turned on

[i2c]                              # I2C-Addresses of the sensors
#bme280 =
#bh1750 =
#mq135 =

[destination]                      # Information about the server
#ip =                              # IP-Address or Hostname of the Server
#path =                            # Path to the script that accepts the measurements
```

I removed the hashtags and entered the detected addresses:

```
[i2c]
bme280 = 0x76
bh1750 = 0x23
mq135  = 0x48
```

After this step, the firmware could read the correct addresses.

Testing the BH1750 (Light Sensor)

Next, I tested the BH1750 separately.

I opened the test file:

```
ayaail72@ayaail72:/opt/station $ sudo nano test-bh1750.py
  GNU nano 8.4                        test-bh1750.py

ADDRESS = 0x23 # TODO: Change this

from bh1750 import BH1750
import smbus
def main():
    bus = smbus.SMBus(1)  # Use 1 for Raspberry Pi, 0 for older models
    sensor = BH1750(bus, ADDRESS)
```

The address in the file had to be adjusted to: **ADDRESS = 0x23**

After saving the file, I executed:

```
ayaail72@ayaail72:/opt/station $ python test-bh1750.py
Light Level: 115.83333333333334 lux
```

The sensor returned a light value in lux, which confirmed that the BH1750 was working correctly.

Testing the BME280

For the environmental sensor, I opened:

```
ayaail72@ayaail72:/opt/station $ sudo nano test-bme280.py
  GNU nano 8.4                        test-bme280.py

ADDRESS = 0x76 # TODO: Change this
from bme280 import BME280
import smbus

def main():
    bus = smbus.SMBus(1)  # Use 1 for Raspberry Pi, 0 for older models
    sensor = BME280(bus, ADDRESS)
    try:
```

I changed the address to: **ADDRESS = 0x76**

However, when running: **python test-bme280.py**

```
ayaai172@ayaai172:/opt/station $ python test-bme280.py
Error reading BME280 sensor: Unable to find bme280 on 0x76, CHIP_ID returned 58
```

This indicated that the sensor was not a BME280 but a BMP280.
The chip ID 0x58 corresponds to a BMP280 sensor.

2.2.1 Solution: Using the Correct BMP280 Driver

To solve the problem, I used a different Python script specifically for the BMP280 sensor.

First, I prepared a virtual environment:

```
ayaai172@ayaai172:/opt/station $ sudo apt install -y python3-venv python3-full
python3-venv is already the newest version (3.13.5-1).
Installing:
  python3-full

Installing dependencies:
  fonts-mathjax    libjs-mathjax           python3-examples  python3.13-examples
  idle             libpython3.13-testsuite python3-gdbm      python3.13-full
  idle-python3.13  python3-doc             python3.13-doc    python3.13-gdbm

Suggested packages:
```

```
ayaai172@ayaai172:/opt/station $ sudo python3 -m venv venv
ayaai172@ayaai172:/opt/station $ source venv/bin/activate
```

Then I installed the required packages:

```
ayaai172@ayaai172:~/RPIF1/Ressourcen/Ressourcen/Firmware und Scripts $ python3 -m pip install bmp280 --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting bmp280
  Downloading https://www.piwheels.org/simple/bmp280/bmp280-1.0.0-py3-none-any.whl (7.2 kB)
Collecting i2cdevice>=1.0.0 (from bmp280)
  Downloading https://www.piwheels.org/simple/i2cdevice/i2cdevice-1.0.0-py3-none-any.whl (10 kB)
Requirement already satisfied: smbus2 in /usr/lib/python3/dist-packages (from i2cdevice>=1.0.0->bmp280) (0.4.3)
Installing collected packages: i2cdevice, bmp280
Successfully installed bmp280-1.0.0 i2cdevice-1.0.0
ayaai172@ayaai172:~/RPIF1/Ressourcen/Ressourcen/Firmware und Scripts $
```

After that, I used the following script:

```
 GNU nano 8.4                                                  test-bme280.py
from bmp280 import BMP280
from smbus2 import SMBus
import time

ADDRESS = 0x76

def main():
    bus = SMBus(1)
    sensor = BMP280(i2c_dev=bus, i2c_addr=ADDRESS)

    while True:
        temperature = sensor.get_temperature()
        pressure = sensor.get_pressure()

        print(f"Temperature: {temperature:.2f} °C")
        print(f"Pressure: {pressure:.2f} hPa")
        print("-" * 30)
        time.sleep(2)

if __name__ == "__main__":
    main()
```

The script was executed using:

```
ayaai172@ayaai172:~/RPIF1/Ressourcen/Ressourcen/Firmware und Scripts $ python test-bme280.py
Temperature: 22.34 °C
Pressure: 650.76 hPa
------------------------------
Temperature: 28.37 °C
Pressure: 993.92 hPa
------------------------------
Temperature: 28.39 °C
Pressure: 993.92 hPa
------------------------------
Temperature: 28.39 °C
Pressure: 993.92 hPa
------------------------------
Temperature: 28.40 °C
Pressure: 993.90 hPa
------------------------------
```

This time, the sensor returned valid temperature and pressure values.
This confirmed that the sensor was a BMP280 and not a BME280.

Conclusion of the Lab Part

During the lab phase, all sensors were successfully detected and tested.

The main difficulty was the confusion between BME280 and BMP280.
The error message (CHIP_ID returned 58) helped identify the correct sensor type.

After installing the correct driver and adjusting the configuration files, all sensors worked correctly and provided valid measurement data.

A6 - Assembly

<table>
<tr><td>User Story</td><td>A6 - Assembly</td><td>to</td><td>Calendar week 11</td></tr>
<tr><td rowspan="3">Informal description</td><td colspan="3">As the operator of Portable Indoor Feedback</td></tr>
<tr><td colspan="3">… I would like to assemble all the parts</td></tr>
<tr><td colspan="3">… so that I can test my station.</td></tr>
<tr><td>Requirements according to the work order</td><td colspan="3">Must
Assemble wooden housing consisting of 3 parts (frame, removable cover, and rear panel for insertion).
Install the PVC bracket with Raspberry Pi and display in the housing.
Connect all necessary devices.
Place sensors on circuit board.
Connect USB-C to Raspberry.
<u>Documentation:</u>
Work steps
List of materials
Photos of the circuit in the housing
Video of how it works
Explain any difficulties/problems that may have arisen and indicate how they were solved.
Could
All steps involved in attaching the button are planned and carried out independently.
Caution: Ensure that the push button wires are long enough so that the display and bracket can still be inserted and removed!</td></tr>
</table>

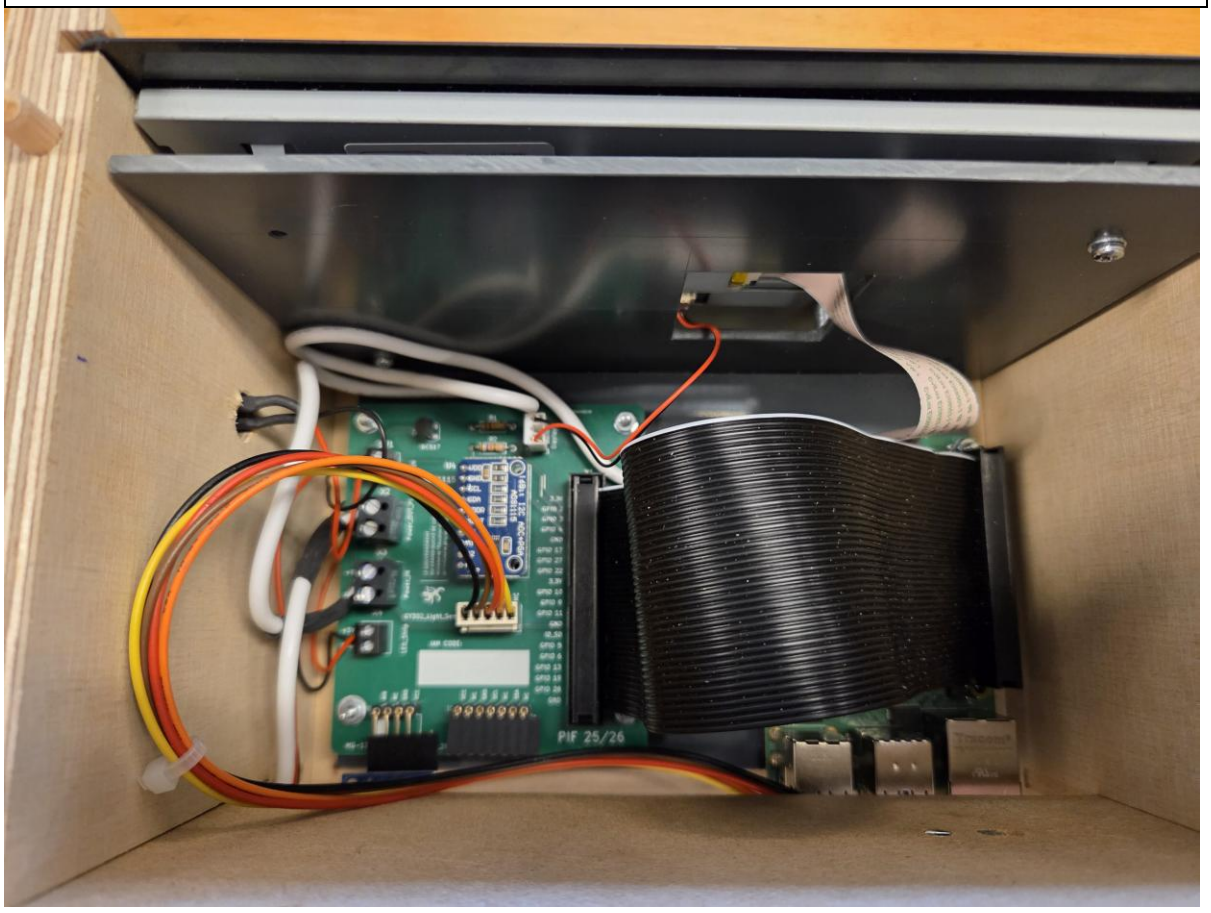
	The button variant shown is not the same as in the example photos!

Tasks according to the work order

- Check the necessary components.
- Connecting the power supply.
- Connect the display (power supply to the corresponding connection on the circuit board).
- Plug the sensors into the designated slots on the circuit board.
- Mount the light sensor and barometric sensor on the rear breadboard of the housing.

Resources

- Sample photos of the setup:



Purpose:

The goal of this task was to assemble the wooden housing of the station and install the components so the station can be tested.

Materials:

Wooden housing pieces

Push button / switch

Wires (red and black)

Screws and nuts

Plastic spacers

Screwdriver

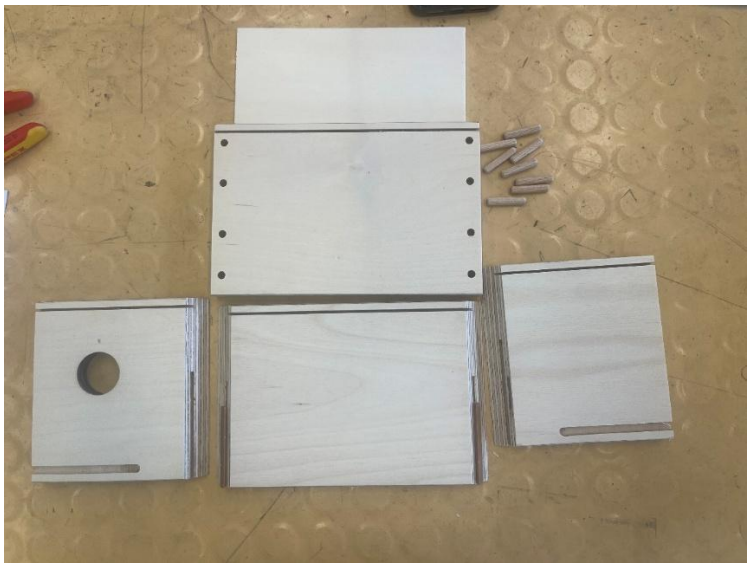
Soldering station

Solder

Work Steps

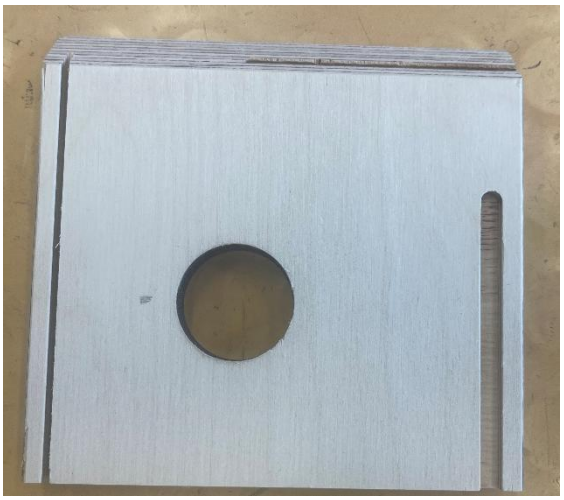
1.1 Preparation

First all materials and tools were prepared.



Drilling the hole for the push button

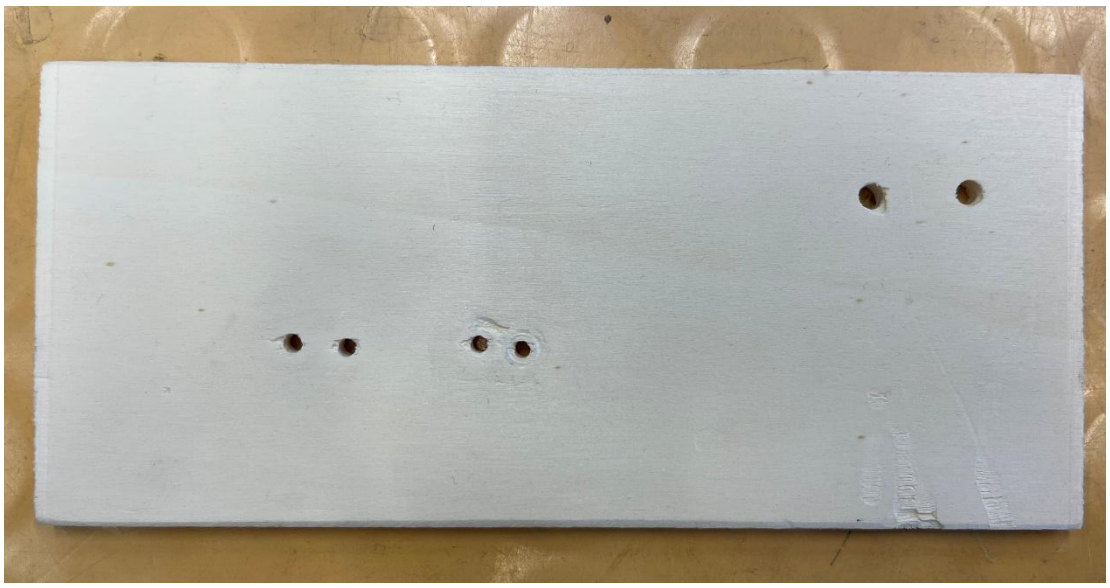
A hole was drilled in the wooden side panel for the push button.
After drilling, the hole was filed so that the surface became smooth.



Drilling the rear panel

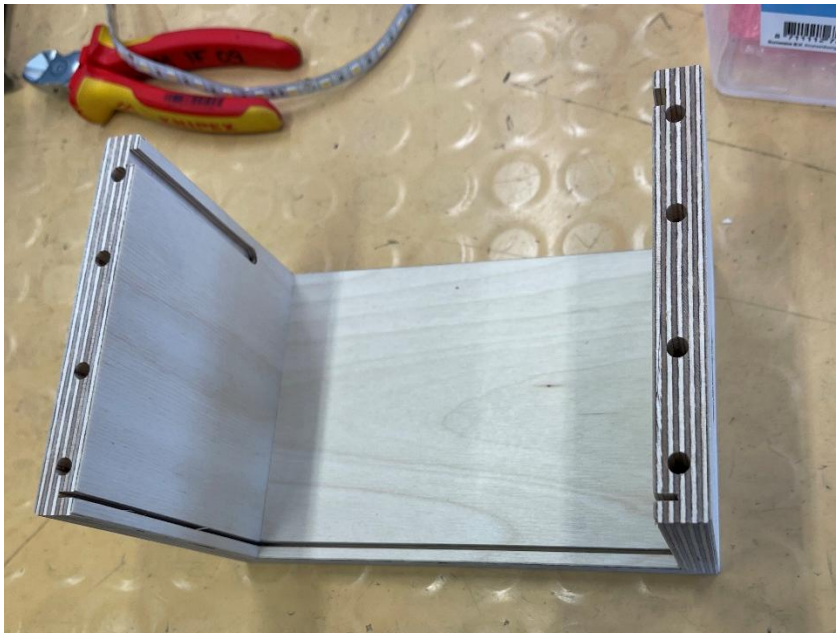
The rear wooden panel was drilled:

- two holes for the environmental sensor
- two holes for the light sensor
- one hole for the power supply cable



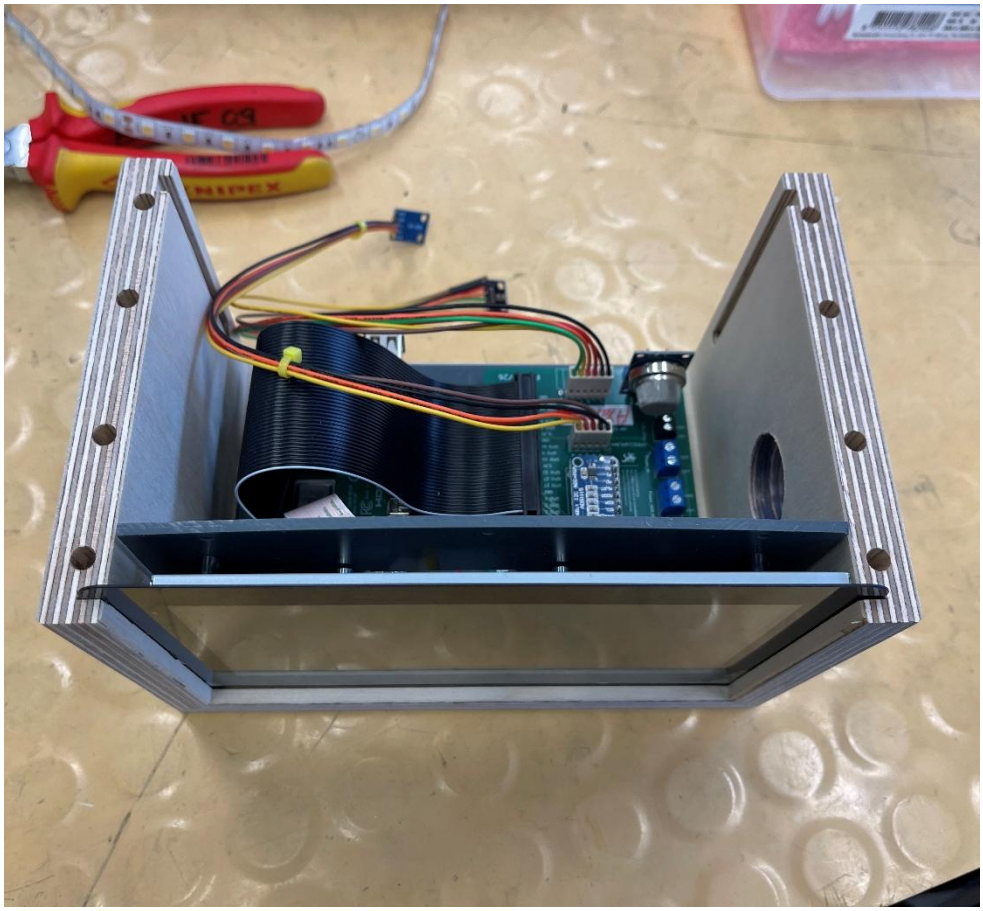
Building the housing

Two plastic spacers were attached to the base wooden plate.
Then the side wooden panels were connected to the base to create the housing frame.



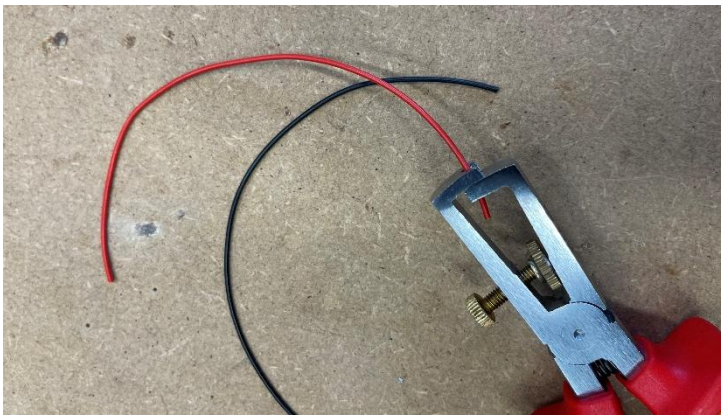
Installing the Raspberry Pi bracket

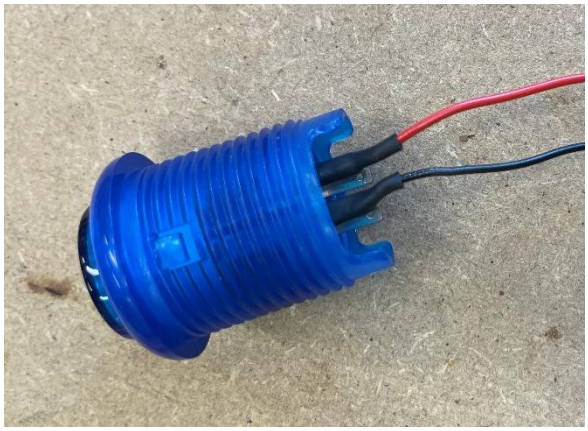
The PVC bracket containing the station components was placed inside the housing to check that everything fits correctly.



Preparing the push button

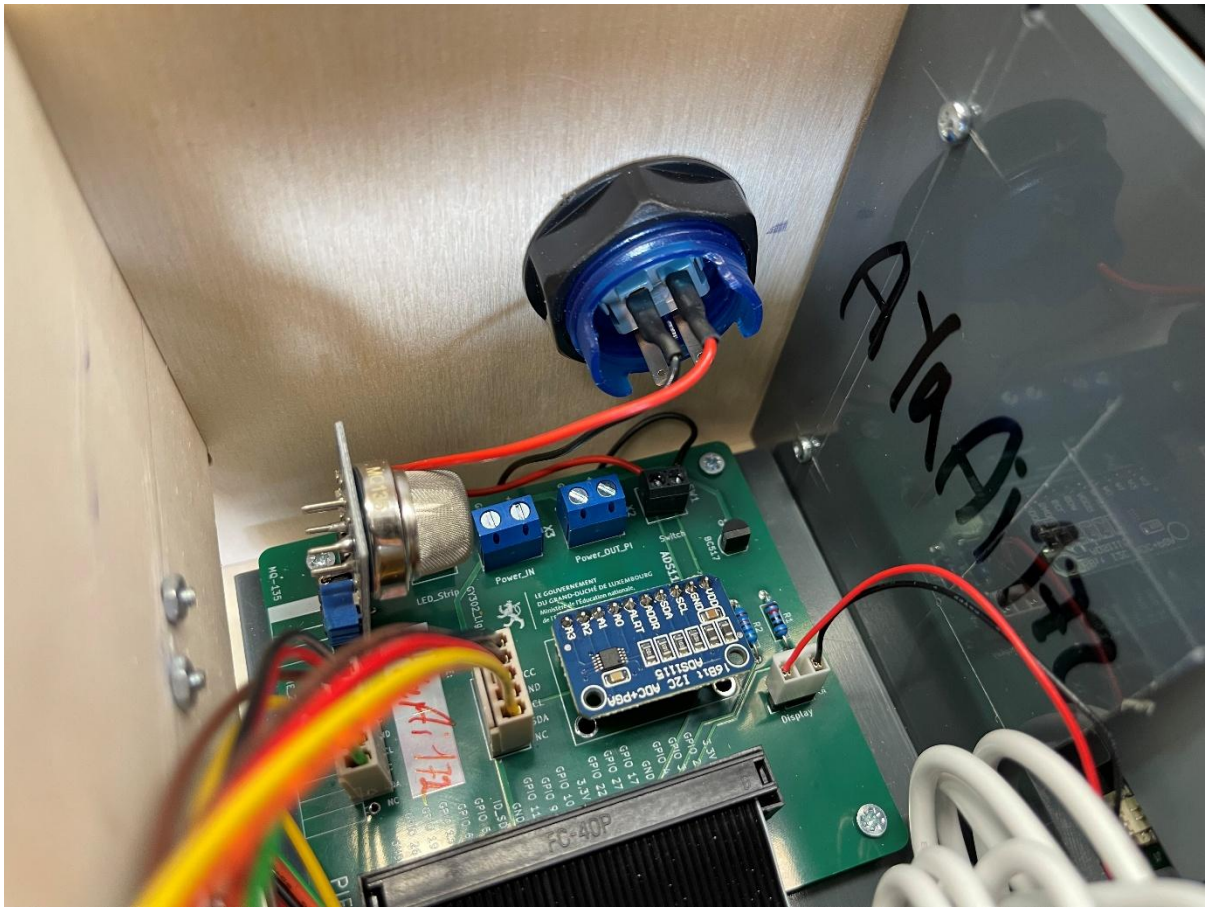
The wires were stripped and soldered to the push button using a soldering station.
After soldering, the connections were covered with **heat shrink tubing** to protect the wires and ensure insulation.





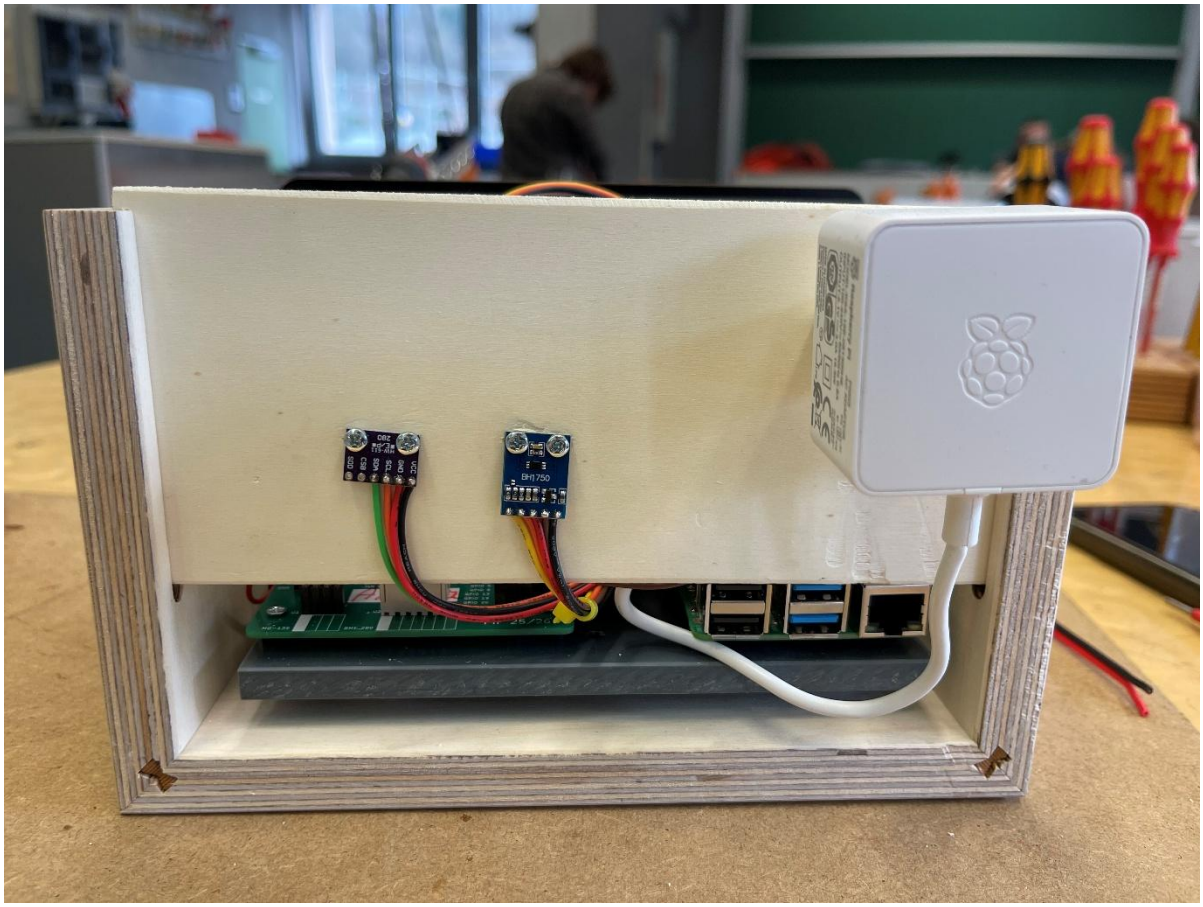
Installing the push button

The push button was inserted into the drilled hole in the housing and fixed with the nut. The wires were then connected to the switch port on the PCB.



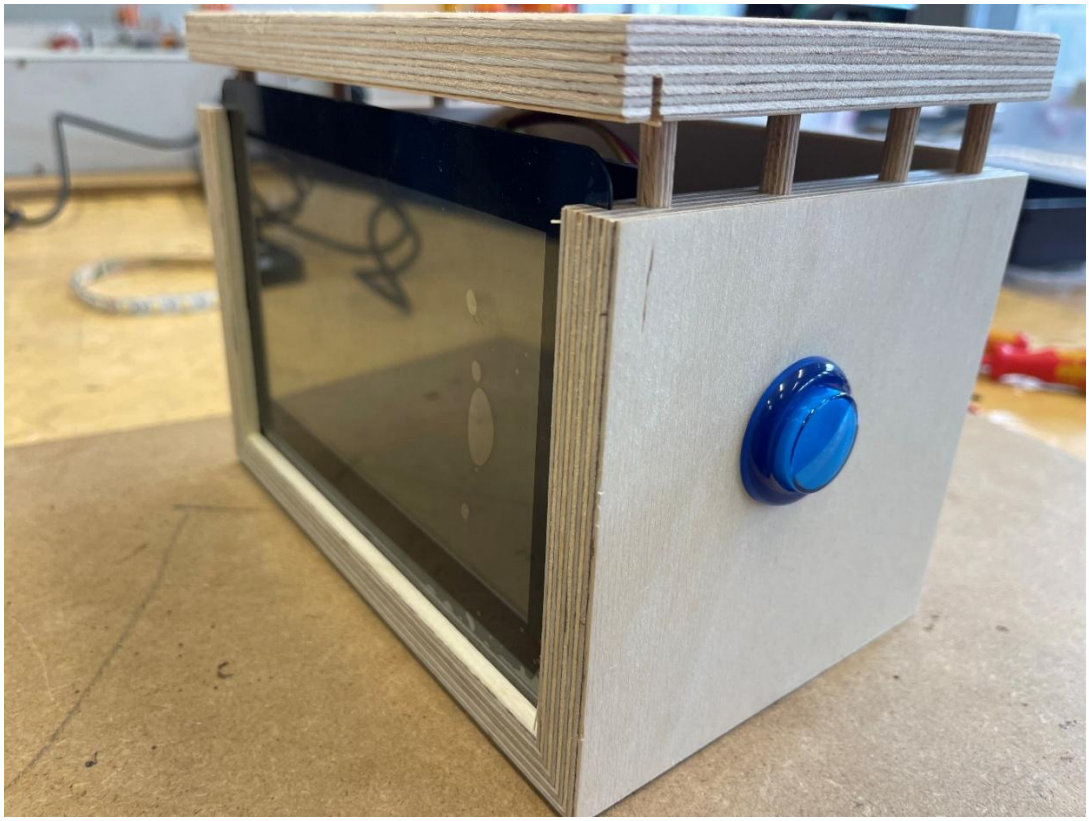
Installing the sensors

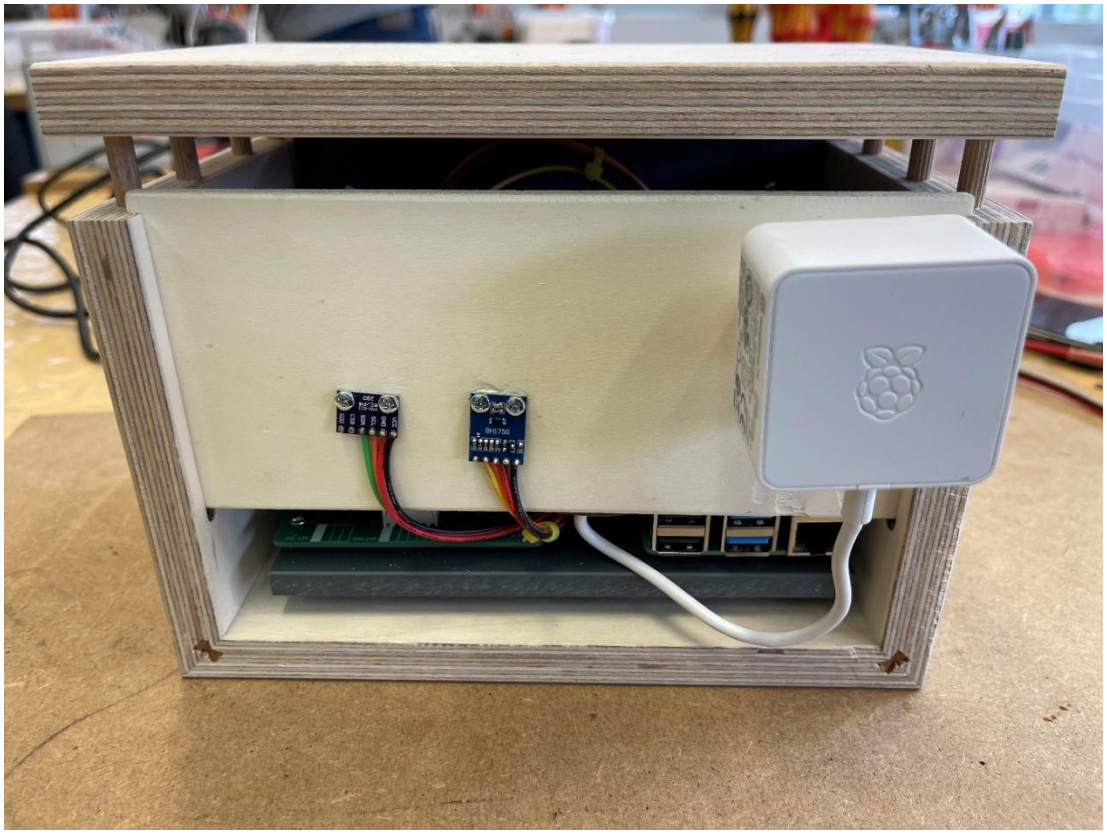
The environmental sensor and the light sensor were mounted on the rear wooden panel.



Final assembly

Finally all components were connected and installed in the housing.





A7 - LIGHTING

Introduction

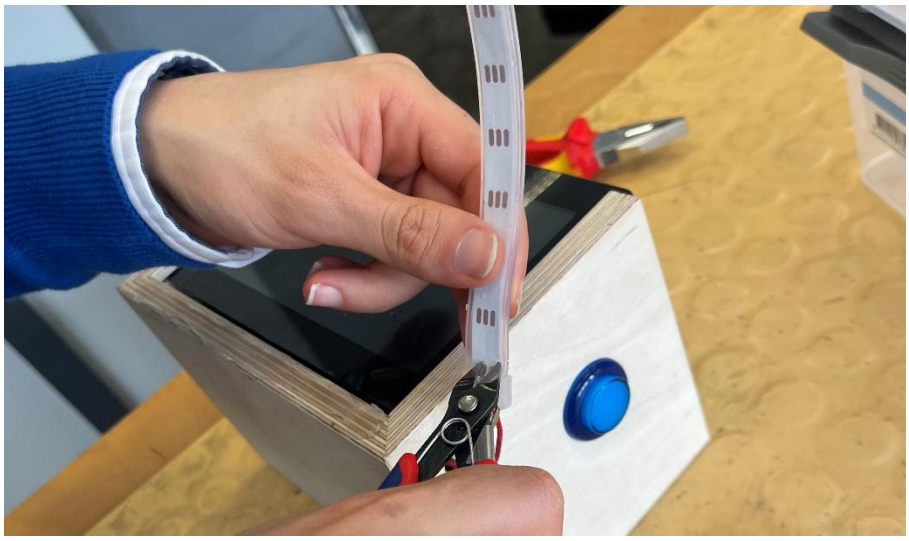
The goal of this task was to integrate an LED strip into the Portable Indoor Feedback Station. The LED strip should illuminate the station and be connected to the power supply via the PCB. Additionally, the system should be tested to ensure correct functionality.

List of Materials

Item	Quantity	Purpose
LED strip	1	Lighting element to be mounted on the lower front edge
Red/black wires	2	Power connection for + and -
Heat-shrink tubing	as needed	Electrical insulation of repaired wire joints
Hot glue gun	1	Fixing and securing the LED strip/cable
Drill	1	Making the cable passage in the wooden housing

Removing the Plastic Cover

The LED strip was covered with a silicone/plastic protection layer. This layer was partially removed to expose the contact points and allow proper soldering and better adhesion to the wooden surface.

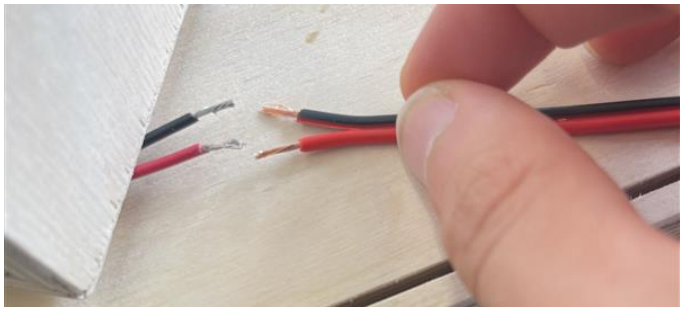


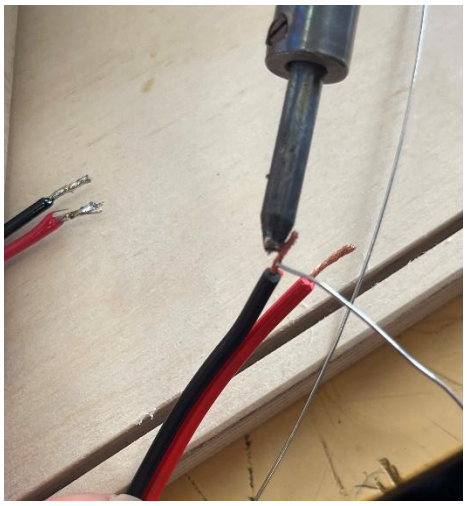
Drilling the Hole

A hole was drilled into the wooden housing to pass the wires from the LED strip into the interior of the station.

Preparing the Wires

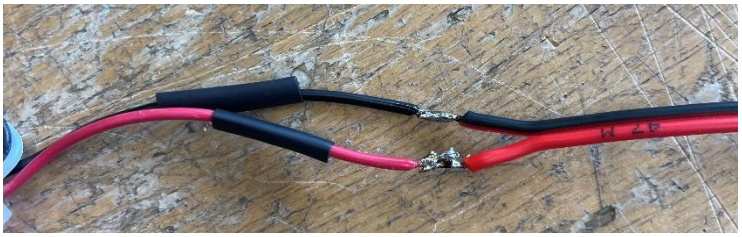
The wires were cut to the required length and stripped at the ends to prepare them for soldering.





Soldering the LED Strip

The wires were soldered to the LED strip at the designated contact points. The red wire was connected to the positive terminal (+), and the black wire to the ground (GND). Care was taken to avoid overheating the LED strip

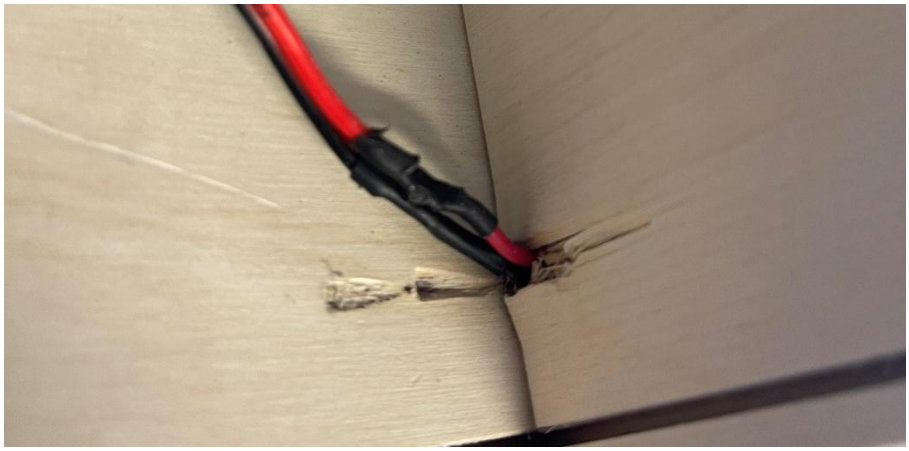


Insulating the Connections

Heat shrink tubing was used to insulate the soldered connections and prevent short circuits.

Routing the Wires

The wires were passed through the drilled hole into the inside of the housing.



Connecting to PCB

The wires were connected to the PCB using the designated terminals (+5V and GND).

Attaching the LED Strip

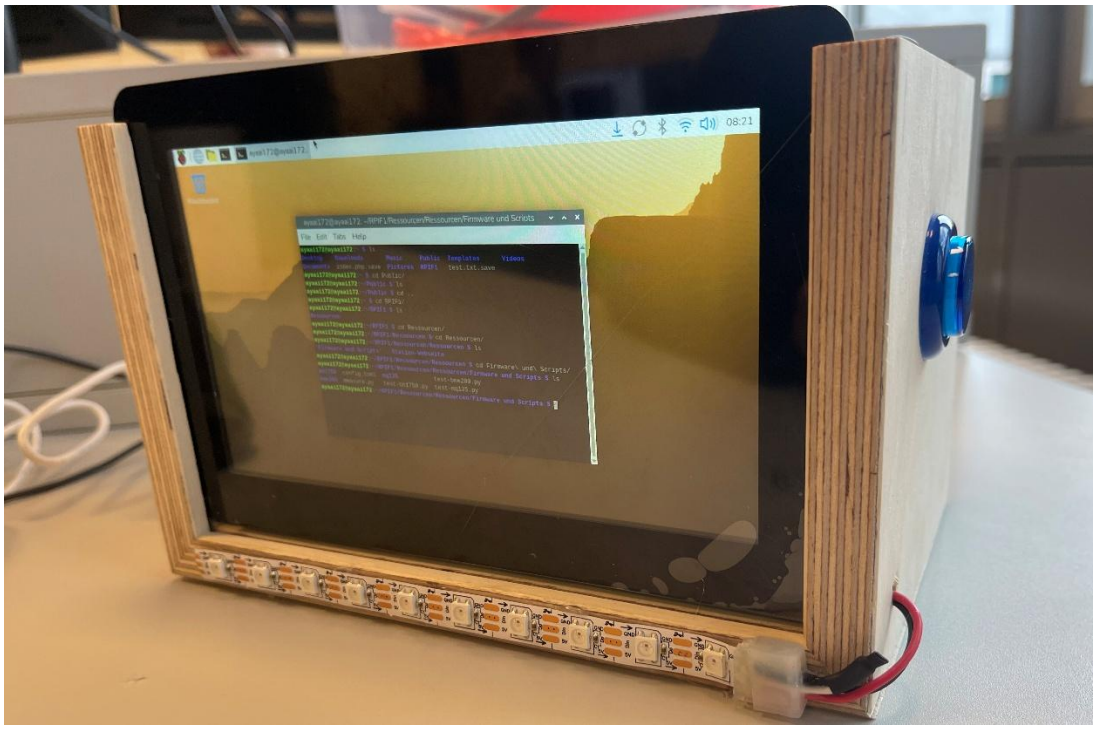
The LED strip was attached to the lower front edge of the housing using adhesive backing and/or glue.

Problems / Difficulties

During the assembly, a problem occurred with attaching the LED strip. The silicone coating prevented proper adhesion to the wooden surface.
This was solved by removing the protective layer from the LED strip.

Final Assembly and Result

After connecting all components, the station was reassembled and prepared for testing.



- Lab user stories

L1 - Raspberry Pi

1.2 Installation of Raspberry Pi OS and Basic Configuration

Brief Introduction

This section covers preparing the microSD card with Raspberry Pi OS, configuring essential system settings, and enabling remote access.
All configuration tasks were completed before inserting the SD card into the Raspberry Pi.

1.2.1 Preparing the microSD Card with Raspberry Pi Imager

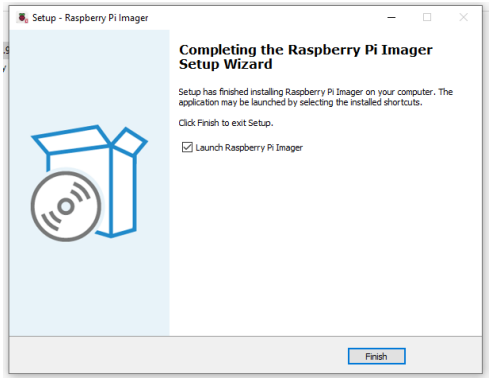
Raspberry Pi Imager was downloaded from the official website and installed.
The correct device (Raspberry Pi 4), OS (Raspberry Pi OS 64-bit), and storage were selected.

• Download and install Raspberry Pi Imager:

Raspberry Pi Imager was downloaded from the official Raspberry Pi website:

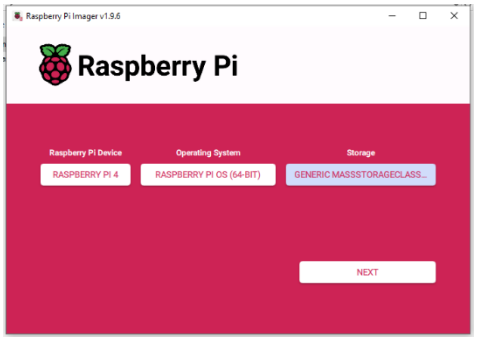
Link: https://www.raspberrypi.com/software/

The installer was executed, and the application was installed on the computer. After launching it, the language selection was confirmed.



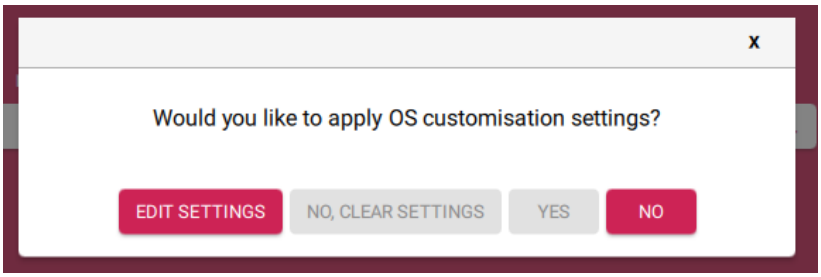
• Choose Device, OS and Storage:

Device: Raspberry Pi 4, OS: Raspberry Pi OS (64-bit), Storage: microSD card.



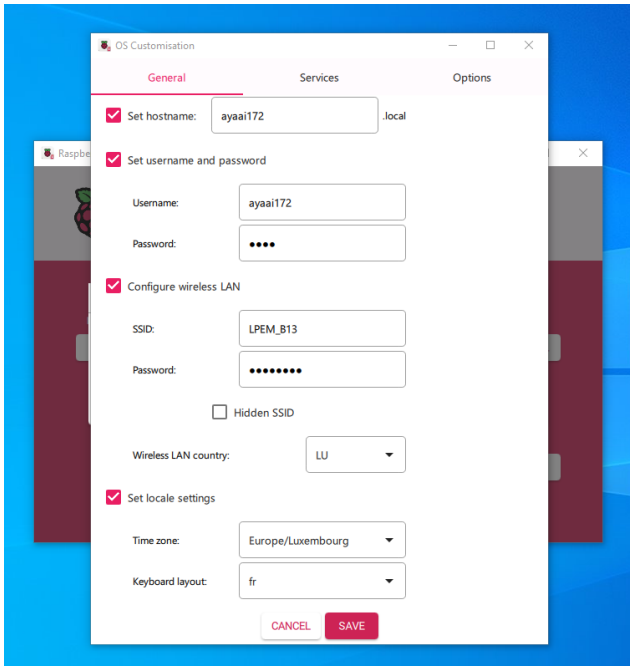
• **Opening the OS customization menu:**

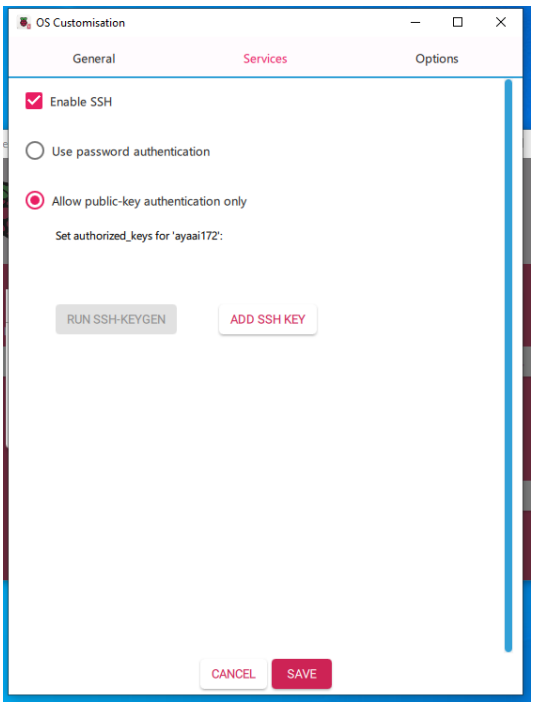
The Imager asked whether to apply custom settings.The settings window was opened using “Edit Settings”



• **Configuring general settings:**

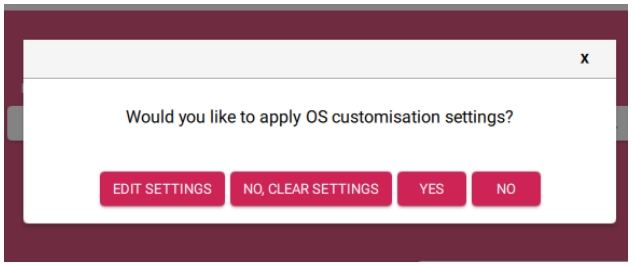
Hostname, Username + password,Locale (timezone, keyboard),WLAN (if used),Enable SSH: Click SAVE.

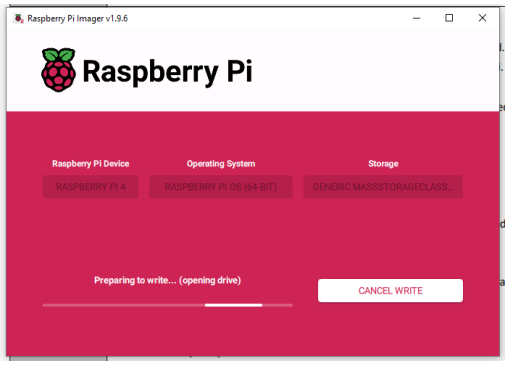




• **Write OS to SD card and confirm erase:**

The Imager displayed a warning about erasing all data.After confirmation, the OS image was written and verified successfully. Click YES.



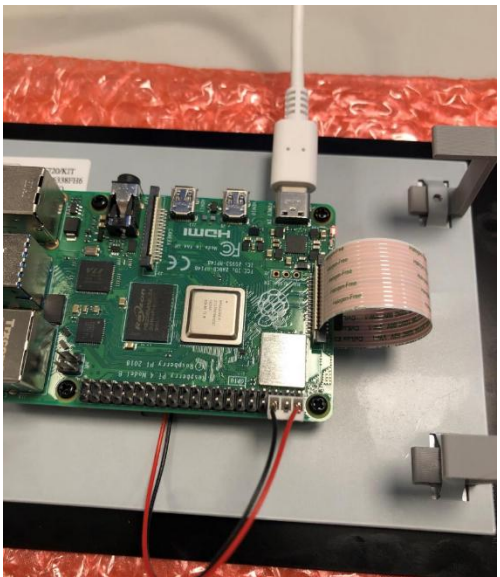


Brief Introduction:

This section describes the physical installation of the Raspberry Pi onto the touchscreen display. It includes connecting all required cables, mounting the board, and verifying that the screen powers on correctly.

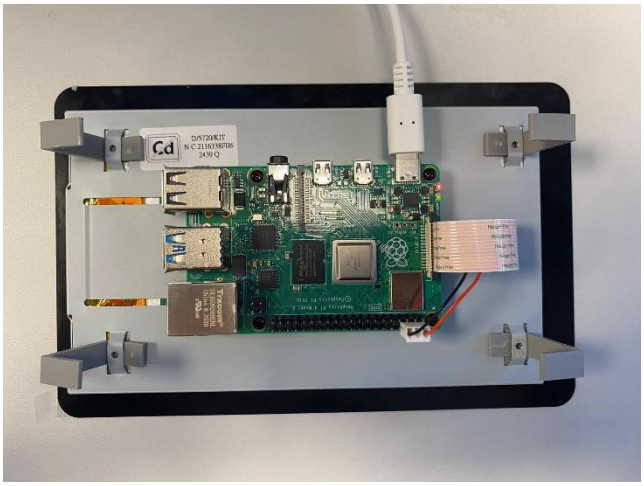
- **Connecting and Mounting the Display + Raspberry Pi**

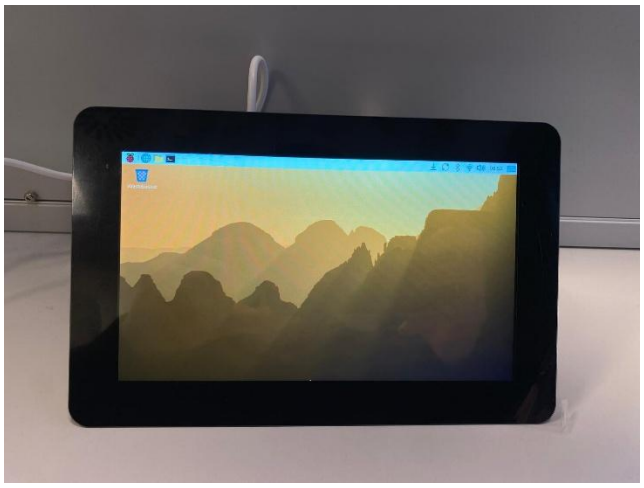
The touchscreen display was placed on a protective surface, and the DSI ribbon cable was attached to its connector.
The Raspberry Pi 4 was then mounted on the back of the display using the provided 3D-printed brackets.
Both required cables (DSI ribbon cable and the display power cable)



1.2.3 Powering the Assembly and First Boot

After connecting the USB-C power cable, the Raspberry Pi powered on successfully and the display showed the Raspberry Pi OS desktop, confirming that the hardware was installed and connected correctly.





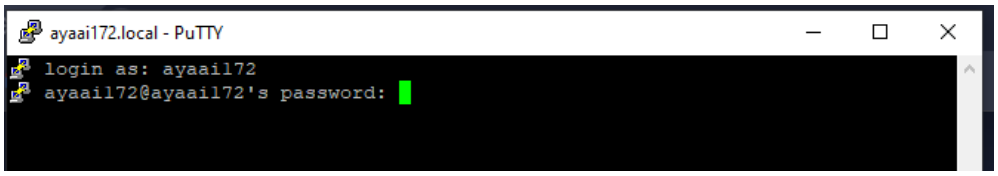
1.2.4 Basic System Configuration

Brief Introduction

In this section, the Raspberry Pi was configured for network access, remote control (SSH + VNC), I^2C communication, correct screen orientation, and a static IP address. Only the steps actually performed are documented.

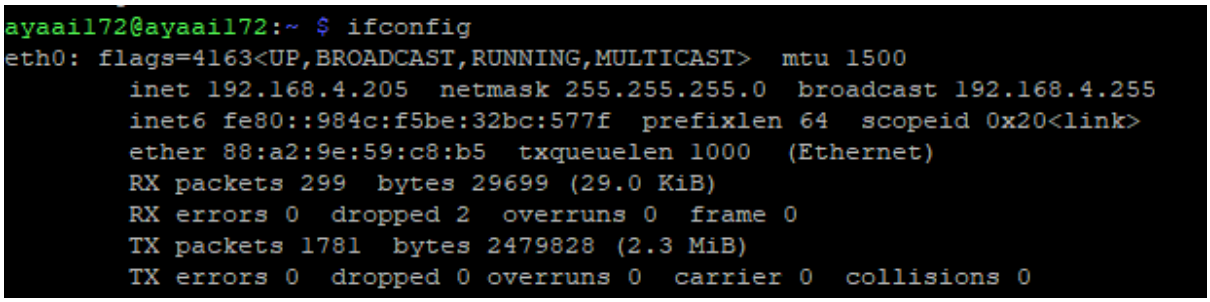
1.2.5 SSH Login to Raspberry Pi

• Connected to the Raspberry Pi using **PuTTY** with the username created in Raspberry Pi Imager.



- Checking IP Address for VNC Connection

• After SSH login, the local IP address was checked using " ifconfig ":

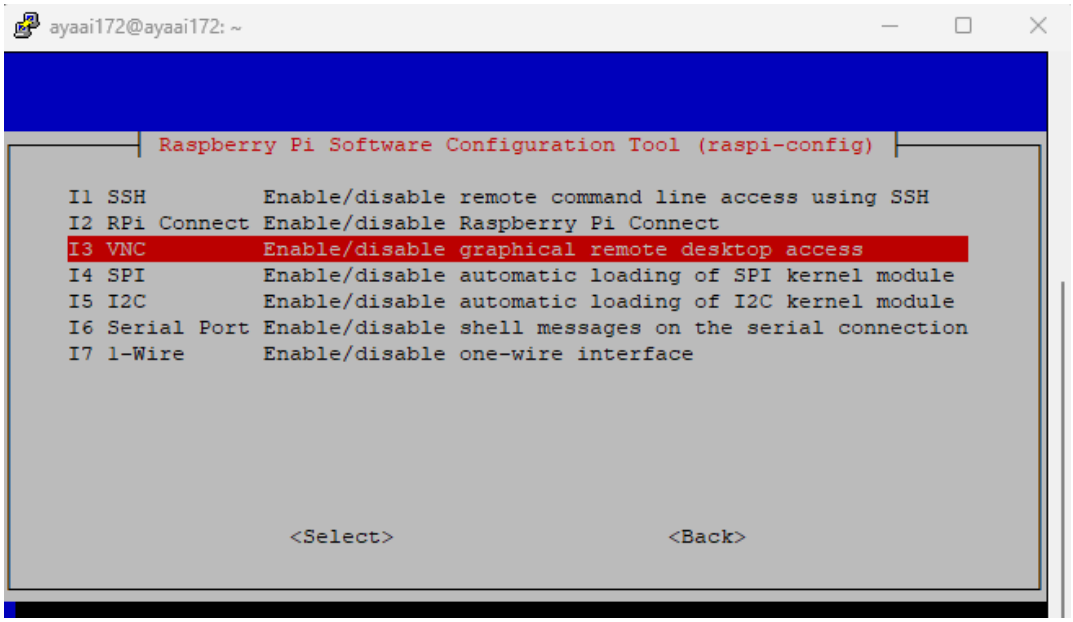


- Enabling VNC

Path:

Interface Options → VNC → Enable

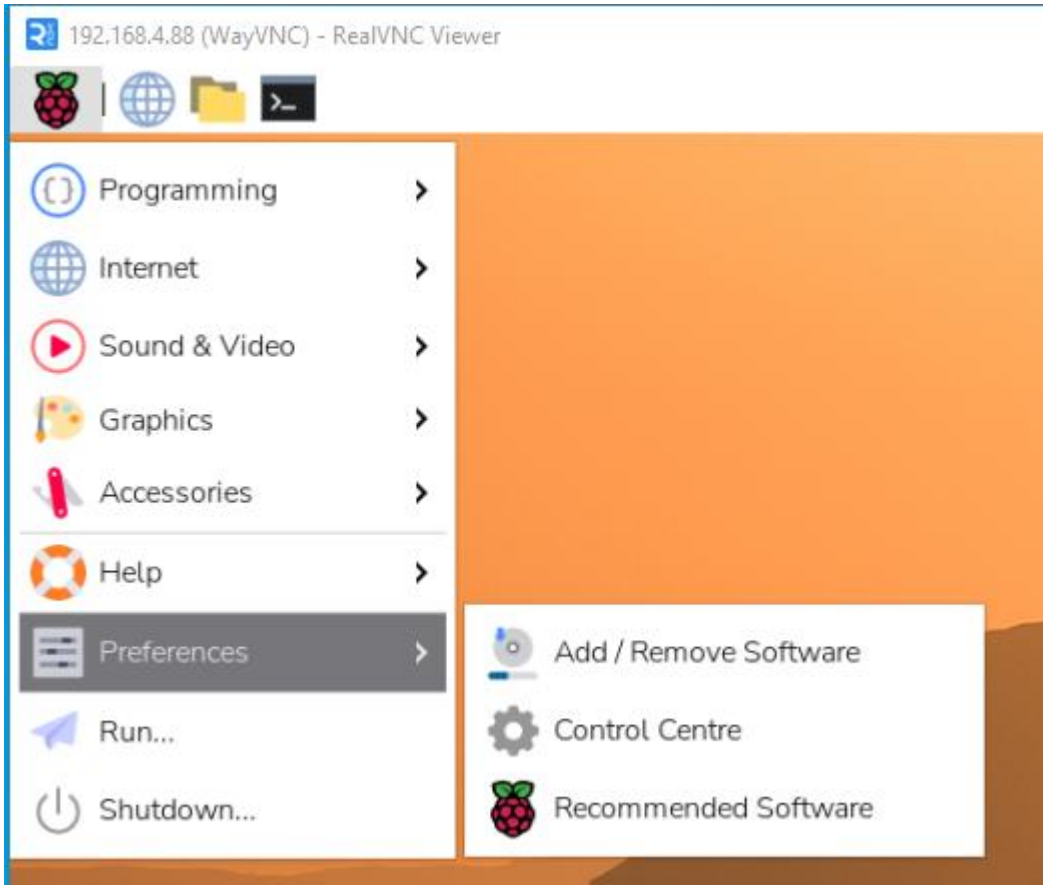
(Screenshots: raspi-config menu + VNC option)

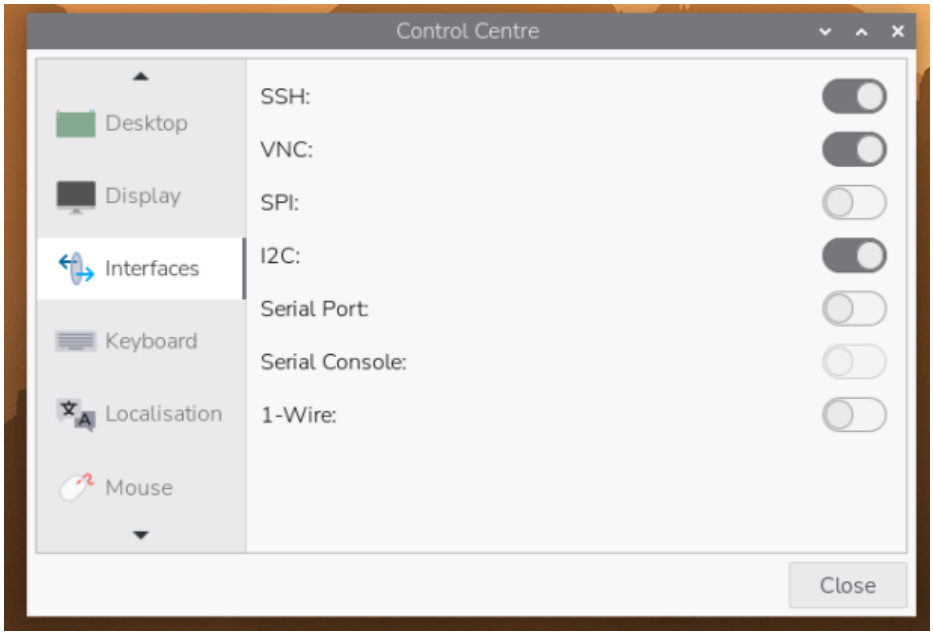


- Enabling I^2C Interface

• I^2C was activated through the graphical Control Centre:

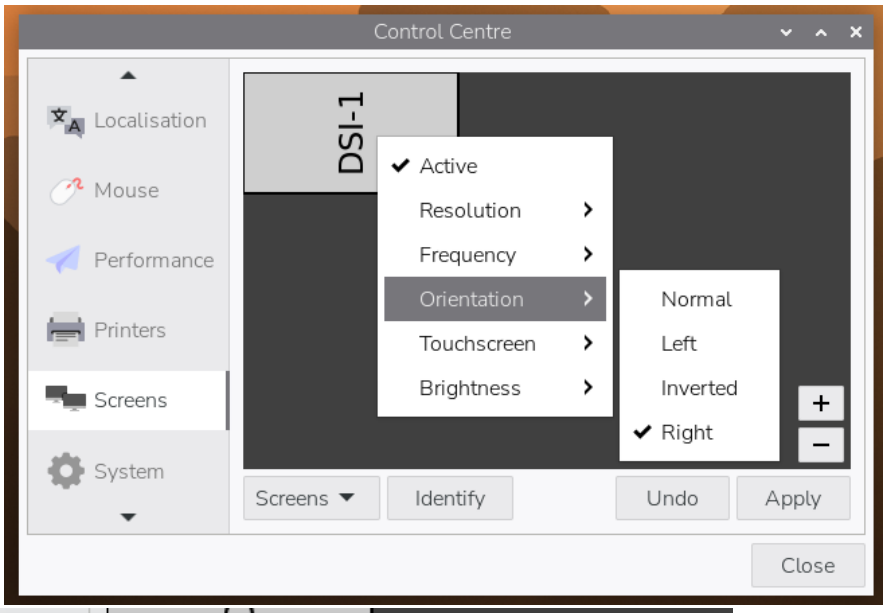
Menu → Preferences → Control Centre → Interfaces → I^2C ON

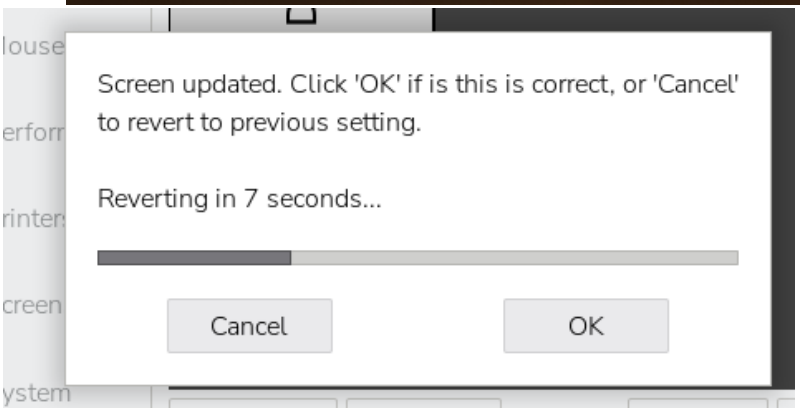




- Screen Orientation (Landscape Mode)

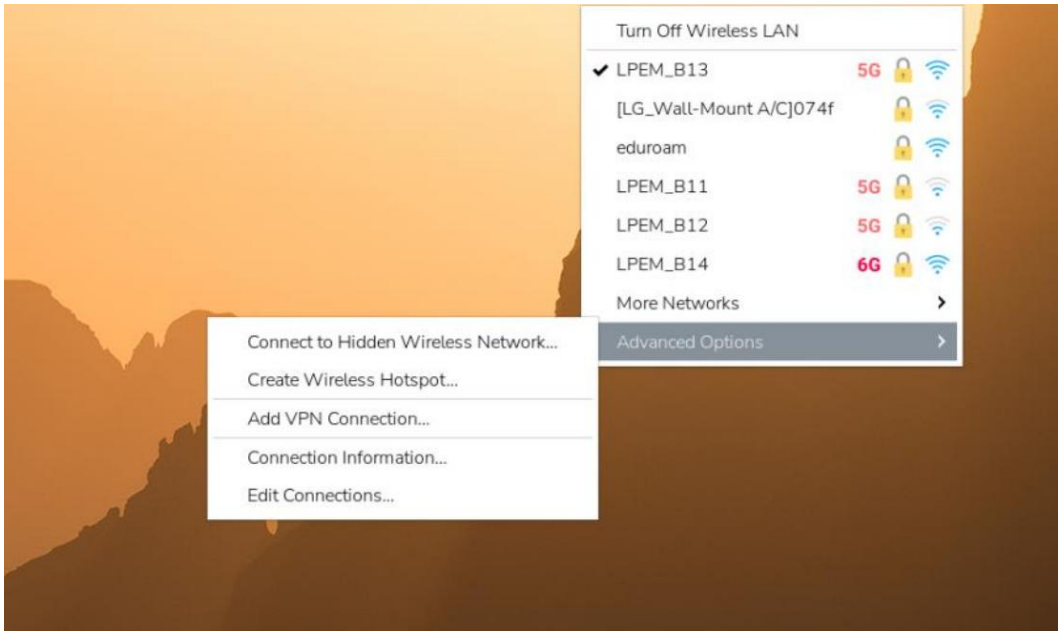
• Using the Display Settings, the screen was rotated to the correct orientation:
Menu → Preferences → Control Centre → Screens → Orientation → Right
• The change was confirmed.

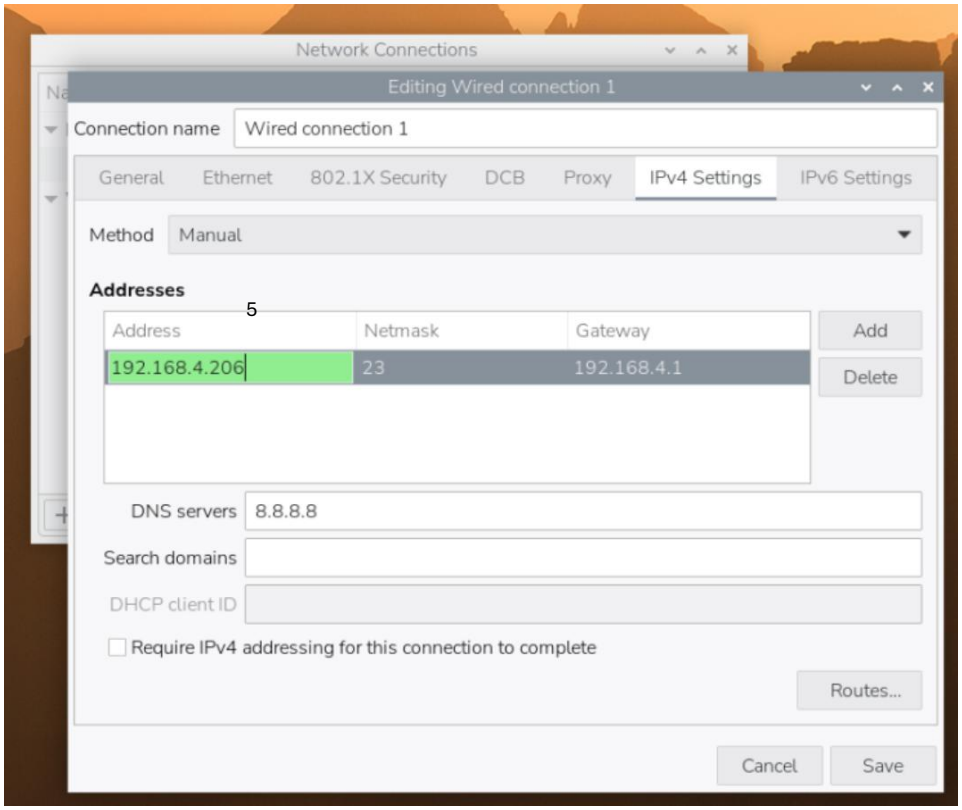




- Setting Static IP Address (WLAN)

• From the network icon → **Advanced Options → Edit Connections...**
• The connection was set to Manual (IPv4).
• IP address, netmask and gateway were entered. Example:


Turn Off Wireless LAN
LPEM_B13 5G
[LG_Wall-Mount A/C]074f
eduroam
LPEM_B11 5G
LPEM_B12 5G
LPEM_B14 6G
More Networks
Advanced Options
Connect to Hidden Wireless Network...
Create Wireless Hotspot...
Add VPN Connection...
Connection Information...
Edit Connections...


Network Connections
Editing Wired connection 1
Connection name
Wired connection 1
General
Ethernet
802.1X Security
DCB
Proxy
IPv4 Settings
IPv6 Settings
Method
Manual
Addresses
Address
Netmask
Gateway
Add
192.168.4.206
23
192.168.4.1
Delete
DNS servers
8.8.8.8
Search domains
DHCP client ID
Require IPv4 addressing for this connection to complete
Routes...
Cancel
Save

1.2.6 Copying Project Files and Folder Setup

Brief Introduction

In this section, the required directories were created on the Raspberry Pi and all project files (configuration file, firmware, scripts, and website) were copied from the GitHub repository to their proper locations.

- Cloning the GitHub Repository

• Git was installed (already the newest version).
• The project repository was cloned from GitHub.

```
ayaai172@ayaai172:~ $ sudo apt install git -y
git is already the newest version (1:2.47.3-0+deb13u1).
git set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
ayaai172@ayaai172:~ $ git clone https://github.com/AyaAi172/RPIF1.git
Cloning into 'RPIF1'...
remote: Enumerating objects: 34, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (30/30), done.
remote: Total 34 (delta 1), reused 34 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (34/34), 38.76 KiB | 2.98 MiB/s, done.
Resolving deltas: 100% (1/1), done.
```

- Creating Required Folders

• Created the folders /opt/station and /var/station using:

```
ayaai172@ayaai172:~ $ sudo mkdir -p /opt/station /var/station
```

• Set correct ownership and permissions:

```
ayaai172@ayaai172:~ $ sudo chown root:root /opt/station /var/station
ayaai172@ayaai172:~ $ sudo chmod 755 /opt/station /var/station
```

- *Copying the Configuration File*

• Copied config.toml from the repository into /var/station:

```
ayaai172@ayaai172:~ $ sudo cp ~/RPIF1/Ressourcen/Ressourcen/"Firmware und Scripts"/config.toml /var/station/
```

• Copied index.php into the website directory:

```
ayaail72@ayaail72:~ $ sudo mkdir -p /opt/station/www
$ayaail72@ayaail72:~ $sudo cp ~/RPIF1/Ressourcen/Ressourcen/Station-Webseite/index.php /opt/station/www/
```

1.2.7 Copying Firmware and Test Scripts

• Copied all firmware, sensor folders, and Python scripts:

```
ayaail72@ayaail72:~ $ sudo cp -r ~/RPIF1/Ressourcen/Ressourcen/"Firmware und Scripts"/* /opt/station/
```

This includes:
– measure.py
– config.toml
– bme280/, bh1750/, mq135/
– test scripts: test-bh1750.py, test-bme280.py, test-mq135.py

1.2.8 Docker Installation and Web Server Setup

Brief Introduction

In this section, Docker was installed on the Raspberry Pi and the web server container (php:8.4-apache) was created. This container will later display the live sensor data through the website stored in `/opt/station/www`.

- Installing Docker

• Add Docker GPG key

The Docker GPG key was downloaded, stored in /etc/apt/keyrings/, and given the correct permissions.

```
ayaail72@ayaail72:~ $ sudo apt install ca-certificates curl
ca-certificates is already the newest version (20250419).
curl is already the newest version (8.14.1-2+deb13u2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

```
ayaail72@ayaail72:~ $ sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc
```

```
ayaail72@ayaail72:~ $ sudo install -m 0755 -d /etc/apt/keyrings
```

• Update package lists

```
ayaail72@ayaail72:~ $ sudo apt update
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://deb.debian.org/debian trixie-updates InRelease
Hit:3 http://deb.debian.org/debian-security trixie-security InRelease
Hit:4 http://archive.raspberrypi.com/debian trixie InRelease
Get:5 https://download.docker.com/linux/debian trixie InRelease [32.5 kB]
Get:6 https://download.docker.com/linux/debian trixie/stable armhf Packages [18.4 kB]
Get:7 https://download.docker.com/linux/debian trixie/stable arm64 Packages [18.4 kB]
Fetched 69.2 kB in 0s (165 kB/s)
All packages are up to date.
```

• **Install Docker components**

```
ayaail72@ayaail72:~ $ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Installing:
  containerd.io  docker-buildx-plugin  docker-ce  docker-ce-cli  docker-compose-plugin
```

• **Verify Docker service**

```
ayaail72@ayaail72:~ $ sudo systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-11-20 10:31:10 CET; 1min 21s ago
 Invocation: 1bf0903bc7504db6b21219b8cb6512e7
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 2646 (dockerd)
      Tasks: 10
     CPU: 745ms
     CGroup: /system.slice/docker.service
             └─2646 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Nov 20 10:31:09 ayaail72 dockerd[2646]: time="2025-11-20T10:31:09.544527030+01:00" level=info msg="Deleting nftables IPv6 rules" error="exit status 1"
Nov 20 10:31:10 ayaail72 dockerd[2646]: time="2025-11-20T10:31:10.272188231+01:00" level=info msg="Loading containers: done."
Nov 20 10:31:10 ayaail72 dockerd[2646]: time="2025-11-20T10:31:10.302474896+01:00" level=warning msg="WARNING: No memory limit support"
Nov 20 10:31:10 ayaail72 dockerd[2646]: time="2025-11-20T10:31:10.302544358+01:00" level=warning msg="WARNING: No swap limit support"
Nov 20 10:31:10 ayaail72 dockerd[2646]: time="2025-11-20T10:31:10.302600561+01:00" level=info msg="Docker daemon" commit=e9ff10b containerd-snapshotter=true storage-driver=overlayfs version=29.0.2
Nov 20 10:31:10 ayaail72 dockerd[2646]: time="2025-11-20T10:31:10.302935759+01:00" level=info msg="Initializing buildkit"
Nov 20 10:31:10 ayaail72 dockerd[2646]: time="2025-11-20T10:31:10.464454926+01:00" level=info msg="Completed buildkit initialization"
Nov 20 10:31:10 ayaail72 dockerd[2646]: time="2025-11-20T10:31:10.487568521+01:00" level=info msg="Daemon has completed initialization"
Nov 20 10:31:10 ayaail72 dockerd[2646]: time="2025-11-20T10:31:10.487745425+01:00" level=info msg="API listen on /run/docker.sock"
Nov 20 10:31:10 ayaail72 systemd[1]: Started docker.service - Docker Application Container Engine.
```

• **Add Docker repository**

A new Docker source file was created in /etc/apt/sources.list.d/docker.sources.

```
ayaail72@ayaail72:~ $ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
>
> Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF


Types: deb
URIs: https://download.docker.com/linux/debian
Suites: trixie
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
```

Run hello-world test

```
ayaail72@ayaail72:~ $ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
198f93fd5094: Pull complete
Digest: sha256:f7931603f70e13dbd844253370742c4fc4202d290c80442b2e68706d8f33ce26
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

1.2.9 Docker Container, Firmware and Autostart (Kiosk Mode)

Brief introduction

In this section the measurement firmware, Docker container and web browser are linked together. After boot, the Raspberry Pi automatically generates measurement data, starts the web server container and shows the website in kiosk mode with the current values.

- Adjusting the firmware configuration

The configuration file for the station was edited:

```
ayaail72@ayaail72:~ $ sudo nano /opt/station/config.toml
```

Changes:

station_serial was set to the IAM code of the station.

All I^2C addresses and destination upload settings were left commented, as they are not needed for this user story.

```
  GNU nano 8.4                                /opt/station/config.toml
station_serial = "AYAAI172"                 # Serial number of the station

interval = 2                            # Time between measurements

toggle_intensity = 100                  # The light level at which the LED should be turned on

[i2c]                                   # I2C-Addresses of the sensors
#bme280 =
#bh1750 =
#mq135 =

[destination]                           # Information about the server
#ip =                                  # IP-Address or Hostname of the Server
#path =                                # Path to the script that accepts the measurements
```

- Preparing the shared data folder

A shared directory for the measurement CSV file was created and made writable:

```
ayaail72@ayaail72:~ $ sudo mkdir -p /tmp/station
```

- Creating the web server container (php:8.4-apache)

The Docker container that serves the website was created with the required volume mappings:

```
ayaail72@ayaail72:~ $ sudo docker run -d --name php -p 8080:80 \
-v /tmp/station:/tmp/station \
-v /opt/station/www:/var/www/html \
php:8.4-apache
```

Verification:

```
ayaail72@ayaail72:~ $ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS              PORTS
                     NAMES
46802ee24dfd   php:8.4-apache   "docker-php-entrypoi…"   About a minute ago   Up About a minute   0.0.0.0:8080->80/tc
p, [::]:8080->80/tcp   php
```

The container php (image php:8.4-apache) is running and exposes port 8080.

- Creating the autostart script (firmware + container + kiosk)

The window manager labwc autostart file was used to start everything automatically after login.

```
ayaail72@ayaail72:~ $ mkdir -p ~/.config/labwc
```

2. Edit the autostart file:

```
ayaail72@ayaail72:~ $ nano ~/.config/labwc/autostart
```

3. Insert the script:

```
  GNU nano 8.4                          /home/ayaail72/.config/labwc/autostart *
#!/bin/bash

# 1. Create the shared folder
mkdir -p /tmp/station

# 2. Start the firmware (Python3) in the background
python3 /opt/station/measure.py --offline > /tmp/station/result.csv &

# 3. Start the Docker container (php)
docker start php &

# 4. Start the browser in kiosk mode and load the website
chromium-browser --kiosk http://localhost:8080 &
```

4. Make the script executable:

```
ayaail72@ayaail72:~ $ chmod +x ~/.config/labwc/autostart
```

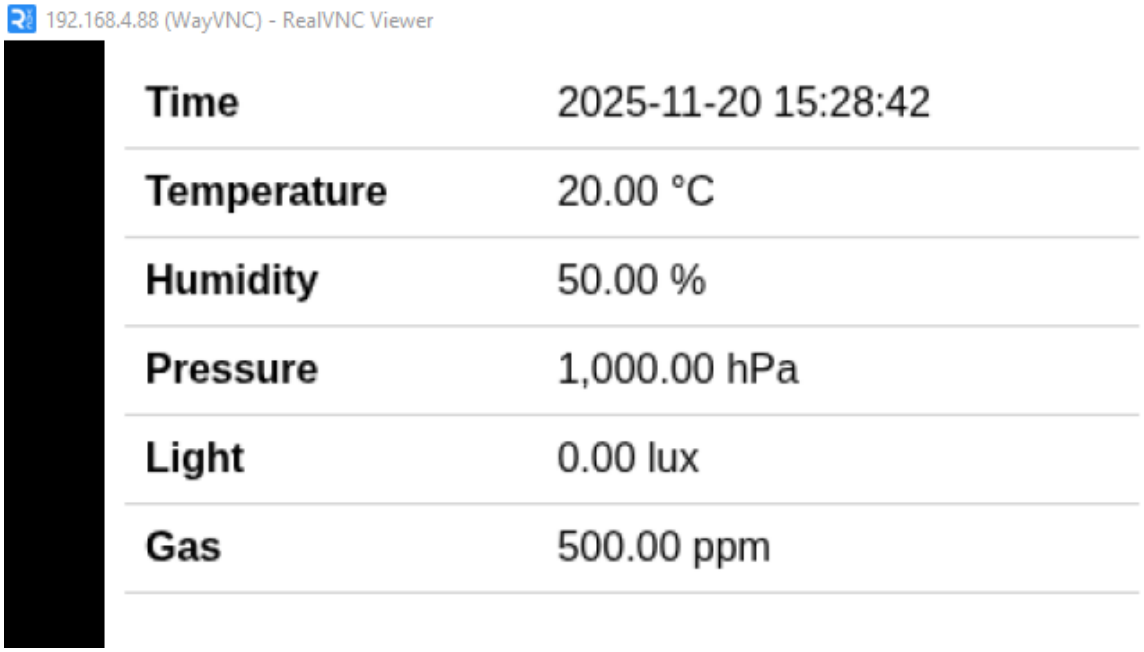
- Reboot and verification of kiosk display

After rebooting the Raspberry Pi:

- The script created /tmp/station/result.csv using measure.py --offline.
- Docker automatically started the php container.
- Chromium opened automatically in kiosk mode and loaded http://localhost:8080.

The website displayed the measurement table with time, temperature, humidity, pressure, light and gas values (simulated offline values such as 20 °C, 50 % etc.), confirming that:

- the firmware starts automatically,
- the website is hosted in the Docker container,
- and the page is shown in kiosk mode.

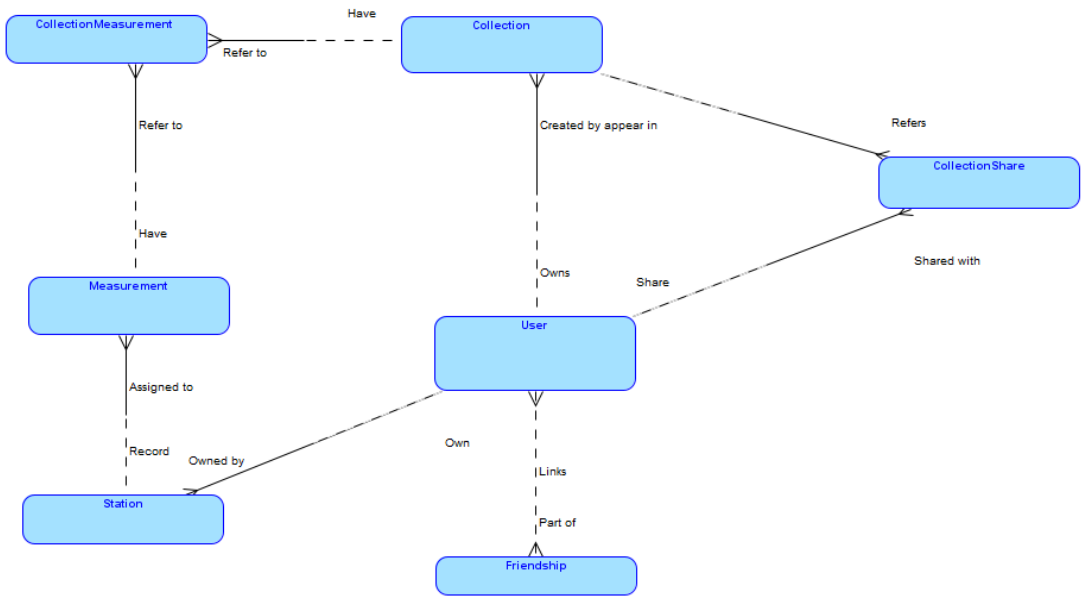


L2 - Database

Introduction

The database stores users, stations, measurements, and collections of measurements for the Portable Indoor Feedback (PIF) project.
It also supports sharing collections and adding friends.

2.2.1 MCD (Conceptual Data Model)



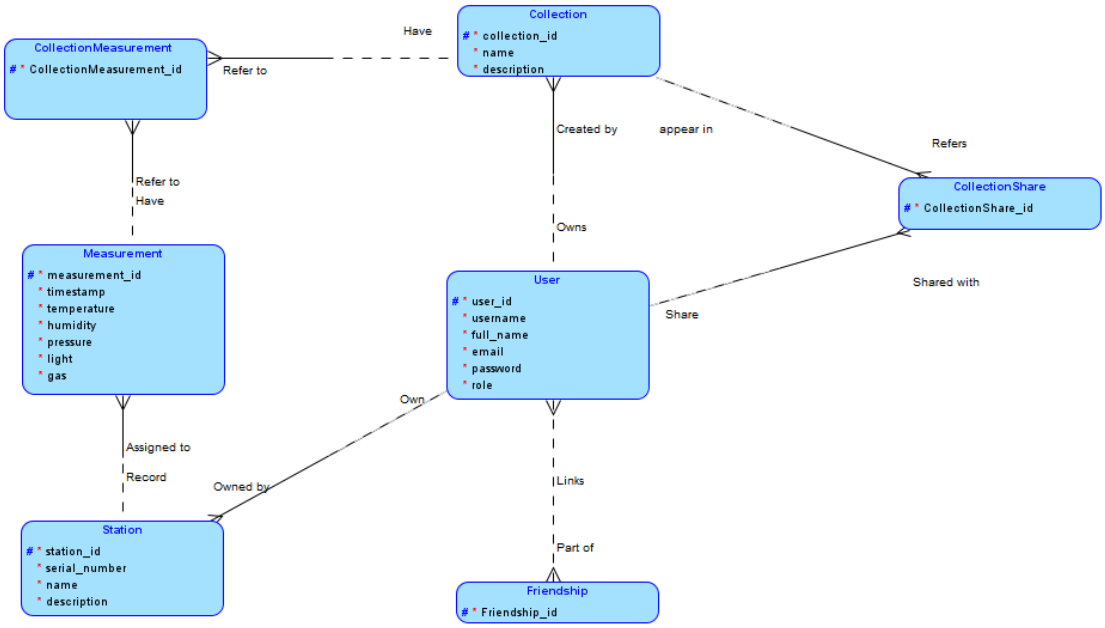
The main entities of the system:

- **User**
- **Station**
- **Measurement**
- **Collection**
- **Friendship** (link between users)
- **CollectionShare** (sharing collections)
- **CollectionMeasurement** (link between collections and measurements)

2.2.2 Relationships:

- **User – Station**:
  One user can have many stations.
  Each station belongs to one user.
- **Station – Measurement**:
  One station records many measurements.
  Each measurement belongs to one station.
- **User – Collection**:
  One user can create many collections.
  Each collection is created by one user.
- **Collection – Measurement**:
  A collection can contain many measurements.
  A measurement can be in many collections.
  (M:N via CollectionMeasurement)
- **User – User (Friendship)**:
  Users can have many friends.
  (M:N via Friendship)
- **User – CollectionShare**:
  A user can have access to many shared collections.
  A collection can be shared with many users.
  (M:N)

3. MLD (Logical Data Model)



The conceptual data model represents the main entities of the Portable Indoor Feedback system and how they are related:

- **User** represents each person using the system. A user can own stations, create collections, share collections, and have friends.
- **Station** represents a physical device that belongs to a user and records measurements.
- **Measurement** stores a single set of sensor values produced by a station.
- **Collection** allows users to group measurements.
- **CollectionMeasurement** links collections and measurements, supporting the many-to-many relationship.
- **CollectionShare** allows users to share their collections with other users.
- **Friendship** represents the friendship relationships between users.

Each relationship in the diagram shows how the entities interact:
users own stations, stations record measurements, users create collections, collections contain measurements, collections can be shared, and users can have friends.

L3 – Website

AUser Story	L3 - Website	to	KW 05
Informal description	As a user or administrator of a station		
	... I want a functional and user-friendly website		
	... so that I can manage and share stations and measured values.		

	Required
	• **Documentation:**
	  ○ **Annotated source code of the website**
	  ○ **Planning this user story**
	Without login:
	• **Login with username and password is possible.**
	• **Registration/creation of a new user account is possible.**
	Logged-in users:
	• **Logout is possible.**
	• **A simple welcome page is displayed.**
	• **Your own stations can be listed. The name and description of the stations can be changed.**
	• **Stations not yet assigned to a user can be registered by serial number.**
	• **Your own user account (username, password, email, first and last name) can be edited.**
	• **Measurement data from your own stations can be viewed and filtered by date/time.**
	• **Users can be added as friends. Friends can be displayed and friendships can be ended.**
	• **Measurement data can be added to collections. Users can only select from their own stations. A collection contains measurement data from a specific station from a start date/time to an end date/time. Your own collections can be renamed and deleted.**
	• **Collections can be shared with friends. Sharing can be undone.**		

	• **Collections shared with the user can be viewed.** **Admins (additional):**

	• All user accounts can be edited and deleted. Admin rights can be assigned to users. New user accounts can be created. • All stations can be edited and deleted. New stations can be created. • Measurement data from all stations can be viewed, filtered by date/time, and deleted. • Measurement data from all stations can be added to collections. All collections can be renamed and deleted. Admins can only share collections they have created themselves. **Should** • The welcome page contains a dashboard that displays current measurement data from your own stations. • The website has an appealing and user-friendly design. • User entries are validated. • Users cannot be added as friends directly, but via a friend request system that requires confirmation of the request. • When a friendship is ended, all collections shared between the two users are automatically unshared. **Could** • Registration of stations via QR code. • Measurement data is displayed using diagrams. • The website automatically adapts to different screen sizes (responsive). • Different themes (e.g., light/dark mode). The selected theme is retained even after logging out. • The website is available in different languages and the selected language remains unchanged after logging out. • Users can be notified of various events (friend requests, sharing of measurement data, errors, etc.) via email and/or notifications. • Users can chat with each other on the website. • Users can send invite links to invite new users to the platform.

Tasks according to the work assignment

- **Plan the structure and navigation of the website according to the work order.**
- **Create a website connected to the database.**
- **Implement user registration and login using username and password.**
- **Implement logout functionality.**
- **Display a simple welcome page for logged-in users.**

- **Implement a user area where users can:**
  - list their own stations
  - edit the name and description of their stations
  - register stations that are not yet assigned using a serial number
- **Implement a user account page where users can edit:**
  - username
  - password
  - email
  - first and last name
- **Implement functionality to view measurement data from the user's own stations.**
- **Allow measurement data to be filtered by date and time.**
- **Plan and implement a friends system where users can:**
  - add friends
  - view their friends
  - end friendships
- **Plan and implement collections where users can:**
  - add measurement data from their own stations
  - select a start and end date/time for a collection
  - rename and delete their own collections
- **Implement functionality to share collections with friends and undo sharing.**
- **Allow users to view collections that were shared with them.**
- **Implement an administrator area where admins can:**
  - edit and delete all user accounts
  - assign and revoke admin rights
  - create new user accounts
  - edit, delete, and create stations
  - view, filter, and delete measurement data from all stations
  - add measurement data to collections
  - rename and delete all collections
- **Document the website with annotated source code and planning of this user story.**

Resources

L4 - WEB-Database Server

User Story	**L4 - Web database server**	**to**	**Calendar week 10**
Informal description	As a technician		
	… I would like a web and database server that is accessible on the network		
	… so that I can access the website and the database to perform various operations.		
Requirements according to the work order	**Must** • A single virtual or physical server that acts as a web and database server is correctly installed and configured. • A test database and website have been created, including test users. • The website can be successfully accessed with a browser. • Documentation: The relevant installation steps for Apache, MariaDB, PHP, and phpMyAdmin, including all necessary commands and explanations. The installation and configuration of the Ubuntu server does not need to be documented. **Should** • MySQL should be secured with *mysql_secure_installation*. • The rights to the website files should be set so that unauthorized persons cannot access them. • A separate database user with minimal rights should be created for the website. • The current version of phpMyAdmin should be downloaded from the website, installed manually, and configured. • All installed packages and the server's operating system should be up to date. **Could**		

	• A self-signed certificate can be issued and the virtual host can be switched to HTTPS. HTTP should be redirected to HTTPS.

Tasks according to the work order

Plan the installation and configuration of the Ubuntu server as a web and database server.

Configure the network so that the server is accessible on the local network.

Install and configure Apache as the web server.

Install and configure MariaDB as the database server.

Install and configure PHP so Apache can execute PHP scripts and connect to MariaDB.

Create a test database.

Create test users for the database and website.

Create a simple test website connected to the database.

Verify that the website can be accessed successfully in a browser.

Install and configure phpMyAdmin.

Secure MariaDB using mysql_secure_installation.

Set correct file permissions for the website files.

Create a separate database user with minimal rights for the website.

Update installed packages and ensure the operating system is up to date.

Optionally create a self-signed certificate and enable HTTPS.

Optionally configure automatic redirection from HTTP to HTTPS.

Document all relevant installation and configuration steps for Apache, MariaDB, PHP, and phpMyAdmin.

Resources

Environment setup

For this user story, an Ubuntu Server virtual machine was prepared to host the web server and database server.
The server was installed in a virtual machine and configured to use **Bridged Network Mode** so that it behaves like a separate device on the local network and can later be accessed by other devices such as a browser or the Raspberry Pi.

After the installation, the system was updated to make sure all installed packages were current.

```
ayaai172@ayaau172:~$ sudo apt update
Hit:1 https://lu.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 https://lu.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 https://lu.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
ayaai172@ayaau172:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ayaai172@ayaau172:~$ S_
```

Apache Web Server Installation

Purpose

Apache is installed to host the website that allows users to access measurement data through a web browser.

Apache is one of the most widely used open-source web servers and will serve the website files to clients over HTTP.

The Apache service starts automatically after installation and listens on **port 80 (HTTP)** by default.

1.2 Installation

```
ayaail72@ayaaul72:~$ sudo apt install apache2 -y
[sudo] password for ayaail72:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libaprlt64 libaprutil1-dbd-s
  liblua5.4-0 ssl-cert
```

1.3 Service Verification

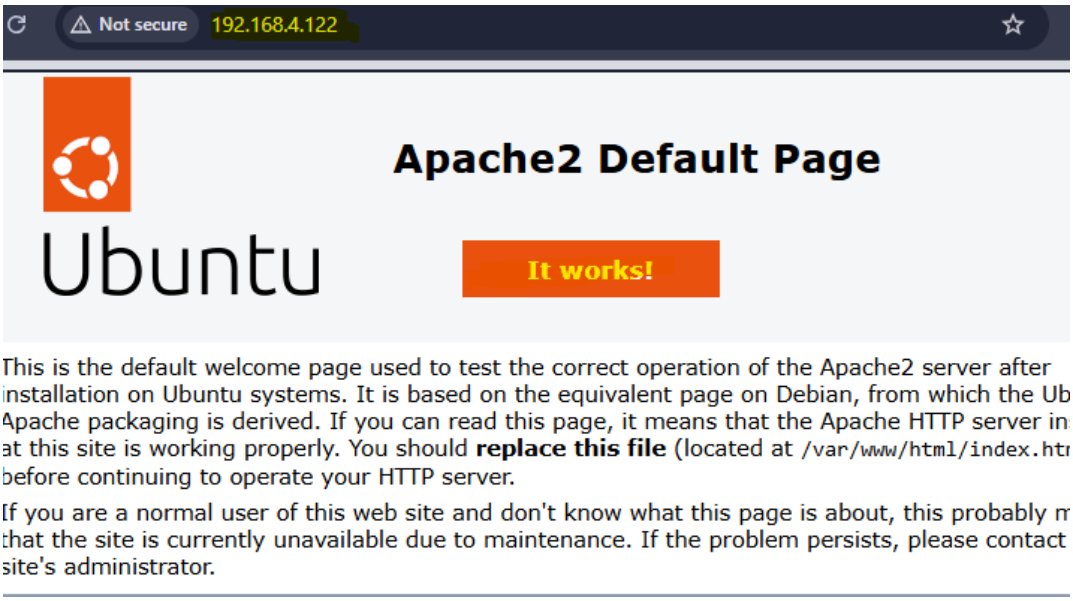
After installation, the Apache service should be running automatically.

```
ayaail72@ayaaul72:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-03-11 07:20:48 UTC; 1min 42s ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 1879 (apache2)
      Tasks: 55 (limit: 16563)
     Memory: 5.4M (peak: 6.2M)
        CPU: 66ms
     CGroup: /system.slice/apache2.service
```

1.4 Test in Browser

Check the IP of the server:

```
ayaail72@ayaaul72:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.4.122  netmask 255.255.254.0  broadcast 192.168.5.255
        inet6 fe80::a00:27ff:fe3e:6e31  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:3e:6e:31  txqueuelen 1000  (Ethernet)
```



Install MariaDB (database server)

Purpose

MariaDB is installed to store the data used by the website.
It is a relational database management system compatible with MySQL and commonly used in the **LAMP stack (Linux, Apache, MariaDB, PHP)**.

1.5 Install MariaDB

```
ayaail72@ayaaul72:~$ sudo apt install apache2 -y
[sudo] password for ayaail72:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libaprlt64 libapru
  liblua5.4-0 ssl-cert
```

1.6 Verify MariaDB Service

After installation, MariaDB should start automatically.

```
ayaail72@ayaaul72:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.11.14 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-03-11 07:29:19 UTC; 2min 28s ago
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
   Main PID: 2887 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 11 (limit: 109316)
     Memory: 78.8M (peak: 82.4M)
        CPU: 566ms
     CGroup: /system.slice/mariadb.service
             └─2887 /usr/sbin/mariadbd
```

This confirms that the MariaDB server is running and ready to accept database connections.

1.7 Access the MariaDB Console

Ubuntu allows the root user to access MariaDB using **unix_socket authentication**.

```
ayaail72@ayaaul72:~$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

This opens the MariaDB command-line interface where database administration tasks can be performed.

PHP Installation

Purpose

PHP is installed to allow the web server to execute dynamic scripts.
It will later be used to connect the website to the MariaDB database and process user requests.

PHP is a widely used server-side scripting language designed for web development.

1.8 Install PHP and Required Modules

```
ayaail72@ayaaul72:~$ sudo apt install php libapache2-mod-php php-mysql -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libapache2-mod-php8.3 php-common php8.3 php8.3-cli php8.3-common php8.3-mysql
Suggested packages:
  php-pear
The following NEW packages will be installed:
  libapache2-mod-php libapache2-mod-php8.3 php php-common php-mysql php8.3 php8
  php8.3-opcache php8.3-readline
```

1.9 Verify PHP Installation

```
ayaail72@ayaaul72:~$ php -v
PHP 8.3.6 (cli) (built: Jan  7 2026 08:40:32) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.3.6, Copyright (c) Zend Technologies
    with Zend OPcache v8.3.6, Copyright (c), by Zend Technologies
```

This confirms PHP is correctly installed on the system.

1.10 Test PHP with Apache

To verify that Apache can correctly execute PHP scripts, a simple PHP test file was created in the web server directory.

```
ayaail72@ayaaul72:~$ sudo nano /var/www/html/Test.php
  GNU nano 7.2                                  /var/www/html/Test.php *

<?php
echo("Hello World! From AyaAil72");
?>
```

Save and exit.

1.11 Test in Browser

Hello World! From AyaAi172

The message displayed in the browser confirms that Apache successfully executed the PHP script.

Creating the Test Database

Purpose

A test database is created to verify that the web server and the database server can work together.
This database will store example data used by a test website.

1.12 Open the MariaDB Console

```
ayaail72@ayaaul72:~$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 32
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

This command opens the MariaDB command-line interface using administrative privileges.

1.13 Create the Test Database

```
MariaDB [(none)]> CREATE DATABASE test_db;
Query OK, 1 row affected (0.001 sec)
```

Here I created a new database named **test_db** that will be used for testing.

1.14 Verify the Database

```
MariaDB [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| test_db            |
+--------------------+
5 rows in set (0.001 sec)
```

This confirms that the database was successfully created.

1.15 Create a Test Database User

```
MariaDB [(none)]> CREATE USER 'AyaAil72'@'localhost' IDENTIFIED BY 'LPEM';
Query OK, 0 rows affected (0.005 sec)
```

This command creates a database user named **user1** that can connect from the local server.

1.16 Grant Permissions to the User

```
MariaDB [(none)]> GRANT ALL PRIVILEGES ON test_db.* TO 'AyaAil72'@'localhost';
Query OK, 0 rows affected (0.003 sec)
```

This gives the user permission to read and modify all tables in the **test_db** database.

1.17 Apply Privileges

```
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)
```

Reloads the privilege tables so the new permissions take effect immediately.

Creating a Test Table

Purpose

A table is created in the test database to store example data that will later be retrieved by the test website.

1.18 Select the Test Database

```
MariaDB [(none)]> USE test_db;
Database changed
MariaDB [test_db]>
```

This tells MariaDB that the following commands will apply to the **test_db database**.

1.19 Create a Test Table

```
MariaDB [test_db]> CREATE TABLE users (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     name VARCHAR(50),
    ->     email VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.063 sec)
```

1.20 Insert Test Data

```
MariaDB [test_db]> INSERT INTO users (name, email) VALUES ('AyaAil72', 'ayaail72@example.com');
Query OK, 1 row affected (0.004 sec)
```

This inserts a sample record so the website can display real data.

1.21 Verify the Data

```
MariaDB [test_db]> SELECT * From users;
+----+----------+----------------------+
| id | name     | email                |
+----+----------+----------------------+
|  3 | AyaAi172 | ayaai172@example.com |
+----+----------+----------------------+
1 row in set (0.000 sec)
```

Exit MariaDB

```
MariaDB [test_db]> EXIT;
Bye
```

Create the Test Website

Purpose

This PHP script connects to the database and displays the stored data in a browser.

1.22 Create the PHP File and add the PHP code

```
ayaai172@ayaaul72:~$ sudo nano /var/www/html/database_test.php
  GNU nano 7.2                                  /var/www/html/database_test.php
<?php

$servername = "localhost";
$username = "AyaAi172";
$password = "LPEM";
$dbname = "test_db";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$sql = "SELECT name, email FROM users";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    while($row = $result->fetch_assoc()) {
        echo "Name: " . $row["name"] . " - Email: " . $row["email"] . "<br>";
    }
} else {
    echo "No results found";
}

$conn->close();

?>
```

1.23 Test the Website

Name: AyaAi172 - Email: ayaai172@example.com

phpMyAdmin Installation

Purpose

phpMyAdmin is a web-based administration tool for MariaDB/MySQL databases.
It provides a graphical interface that allows users to manage databases, tables, and users through a web browser instead of using the command line.

1.24 Download phpMyAdmin

The current version of phpMyAdmin was downloaded from the official website.

```
ayaai172@ayaaul72:~$ cd /tmp
wget https://www.phpmyadmin.net/downloads/phpMyAdmin-latest-all-languages.tar.gz
--2026-03-11 13:42:06--  https://www.phpmyadmin.net/downloads/phpMyAdmin-latest-all-languages.tar.gz
```

1.25 Extract the Archive

```
ayaai172@ayaaul72:/tmp$ tar xvf phpMyAdmin-latest-all-languages.tar.gz
```

This command extracts the downloaded archive and creates a folder containing the phpMyAdmin files.

1.26 Move phpMyAdmin to the Web Server Directory

```
ayaai172@ayaaul72:/tmp$ sudo mv phpMyAdmin-* /var/www/html/phpmyadmin
[sudo] password for ayaai172:
mv: target '/var/www/html/phpmyadmin': No such file or directory
```

when I ran the command this error message appeared, it is telling us that:

No such file or directory. So we have first to Create the destination folder, then move the files.

Create the destination folder:

```
ayaail72@ayaaul72:/tmp$ sudo mkdir /var/www/html/phpmyadmin
```

Then run your move command again:

```
ayaail72@ayaaul72:/tmp$ sudo mv phpMyAdmin-* /var/www/html/phpmyadmin
```

Verify:

```
ayaail72@ayaaul72:/tmp$ ls /var/www/html
database_test.php  index.html  info.php  phpmyadmin  Test.php
```

A directory for phpMyAdmin was created inside Apache's web root and the extracted phpMyAdmin files were moved there so that the application can be accessed through the web server.

1.27 Set File Permissions

```
ayaail72@ayaaul72:~$ sudo chown -R www-data:www-data /var/www/html/phpmyadmin
```

This command ensures that the Apache web server user (www-data) has permission to access the phpMyAdmin files.

1.28 Access phpMyAdmin in the Browser

I landed to the Apache directory listing, which means Apache is showing the folder contents instead of opening phpMyAdmin.

Fix:

Move phpMyAdmin files to the web root

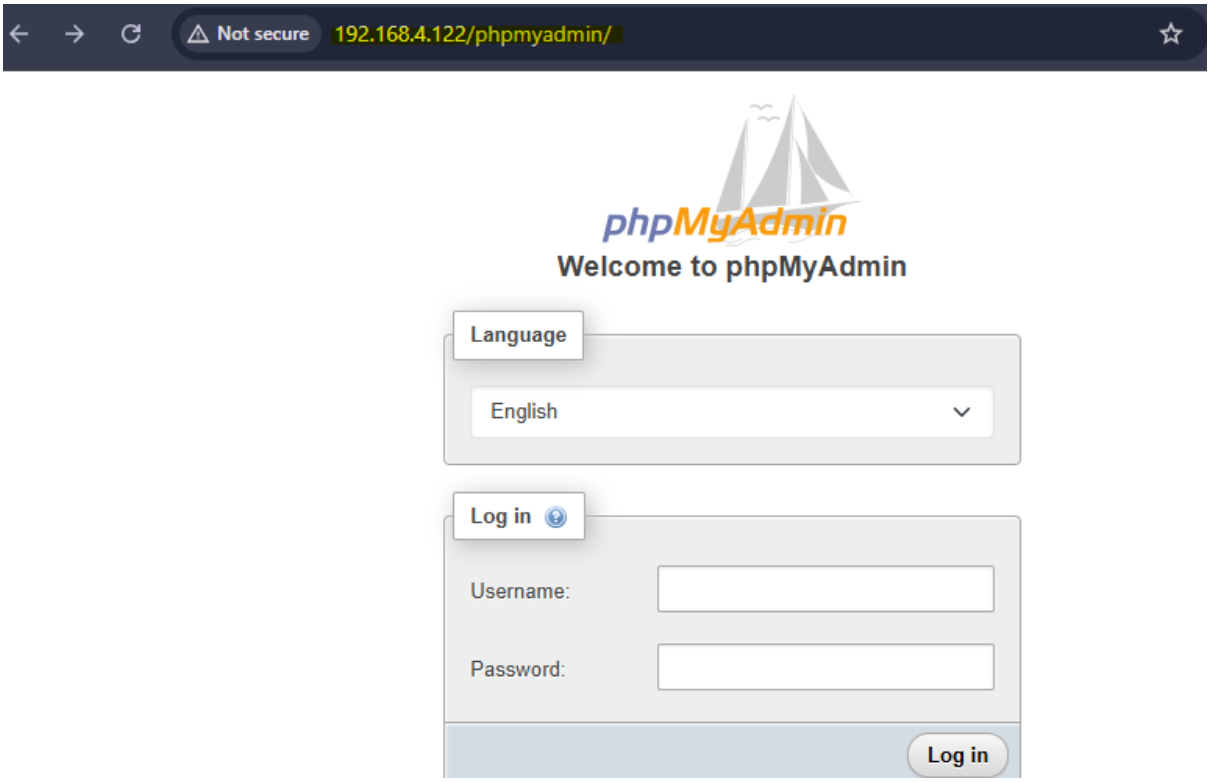
Then adjust the folder structure so that the phpMyAdmin application files are directly inside /var/www/html/phpmyadmin.

```
ayaail72@ayaaul72:~$ cd /var/www/html/phpmyadmin
ayaail72@ayaaul72:/var/www/html/phpmyadmin$ sudo mv phpMyAdmin-5.2.3-all-languages/* .
```

```
ayaail72@ayaaul72:/var/www/html/phpmyadmin$ sudo rm -r phpMyAdmin-5.2.3-all-languages
```

Verify:

```
ayaail72@ayaaul72:/var/www/html/phpmyadmin$ ls /var/www/html/phpmyadmin
babel.config.json  config.sample.inc.php  favicon.ico  LICENSE       RELEASE-DATE-5.2.3      sql        url.php
ChangeLog          CONTRIBUTING.md        index.php    locale        robots.txt              templates  vendor
composer.json      doc                    js           package.json  setup                   themes     yarn.lock
composer.lock      examples               libraries    README        show_config_errors.php  tmp
ayaail72@ayaaul72:/var/www/html/phpmyadmin$
```



1.29 Create the phpMyAdmin Configuration File

Purpose

phpMyAdmin requires a configuration file to define how it connects to the MariaDB database server.

Go to the phpMyAdmin directory

```
ayaail72@ayaaul72:~$ cd /var/www/html/phpmyadmin
```

Copy the sample configuration

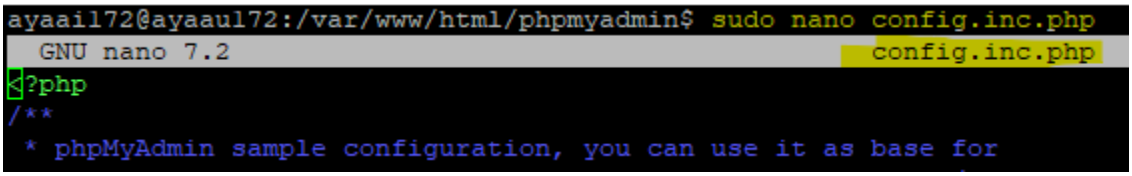
phpMyAdmin provides a template file.

```
ayaail72@ayaaul72:/var/www/html/phpmyadmin$ sudo cp config.sample.inc.php config.inc.php
```

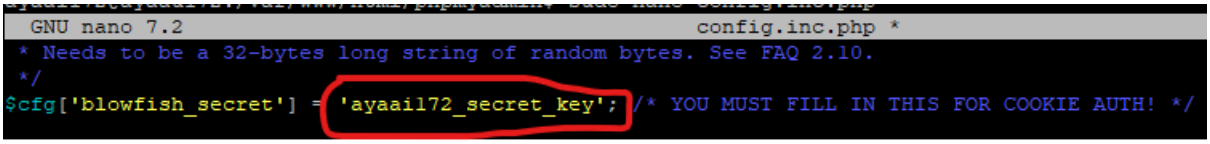
The sample configuration file is copied and renamed to **config.inc.php**, which phpMyAdmin reads when it starts.

Generate a secret key

Edit the configuration file:



Find the line : $cfg['blowfish_secret'] = ''; Replace it with something random

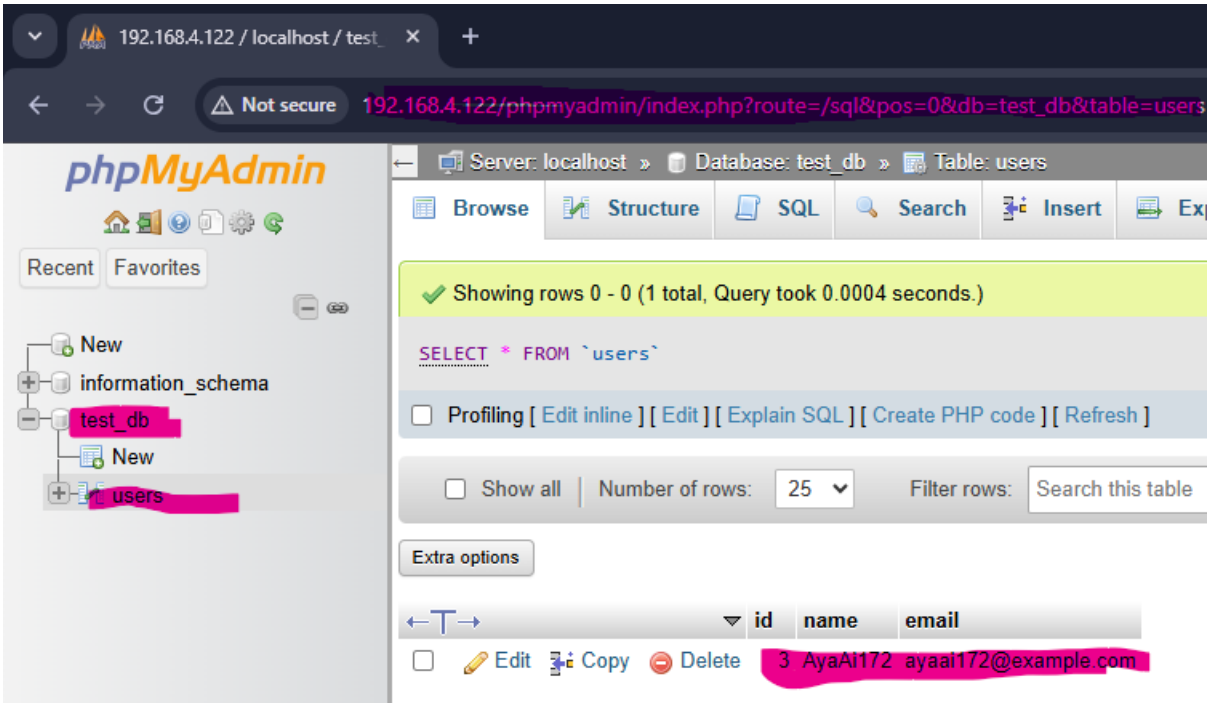


This secret key is required for cookie encryption used by phpMyAdmin authentication.

Save and exit.

1.30 Test phpMyAdmin

Now I will Login with my0 **MariaDB user:**



Securing MariaDB with mysql_secure_installation

Purpose

The **mysql_secure_installation** script is used to improve the security of the MariaDB database server by removing insecure default settings and restricting access.

This script performs several security checks such as removing anonymous users, disabling remote root login, and deleting unnecessary test databases.

1.31 Run the Security Script

```
ayaai172@ayaaul72:~$ sudo mysql_secure_installation
```

This command launches the MariaDB security configuration script, which guides the administrator through several steps to secure the database installation.

First it will ask for you password:

```
Enter current password for root (enter for none):
OK, successfully used password, moving on...
```

Switch to unix_socket authentication?

```
Switch to unix_socket authentication [Y/n] y
Enabled successfully!
```

Allows the Linux root user to authenticate automatically when using **sudo**.

Change the root password?

```
Change the root password? [Y/n] n
 ... skipping.
```

Ubuntu uses **unix_socket** authentication for root, so a password is not required.

Remove anonymous users?

```
Remove anonymous users? [Y/n] y
 ... Success!
```

Anonymous users allow login without authentication and should be removed.

Disallow root login remotely?

```
Disallow root login remotely? [Y/n] y
 ... Success!
```

Prevents the root database account from being accessed from remote systems.

Remove test database?

```
Remove test database and access to it? [Y/n] y
 - Dropping test database...
 ... Success!
 - Removing privileges on test database...
 ... Success!
```

Deletes the default MariaDB test database that is accessible to all users.

Reload privilege tables?

```
Reload privilege tables now? [Y/n] y
 ... Success!
```

Applies the security changes immediately.

After you answer all the questions this message will appear:

```
All done!  If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
```

Setting Website File Permissions

Purpose

File permissions are configured to ensure that only the Apache web server and authorized users can access or modify the website files. This prevents unauthorized access and improves server security.

1.32 Check Current Permissions

First, inspect the permissions of the web directory.

```
ayaail72@ayaaul72:~$ ls -l /var/www/html
total 28
-rw-r--r--  1 root     root       539 Mar 11 08:45 database_test.php
-rw-r--r--  1 root     root     10671 Mar 11 07:20 index.html
-rw-r--r--  1 root     root        45 Mar 11 07:43 info.php
drwxr-xr-x 13 www-data www-data  4096 Mar 11 14:11 phpmyadmin
-rw-r--r--  1 root     root        46 Mar 11 07:46 Test.php
```

This command lists all files in the Apache web directory and displays their ownership and permissions.

1.33 Set Correct Ownership

Apache runs as the user **www-data**, so the web files should belong to this user.

```
ayaail72@ayaaul72:~$ sudo chown -R www-data:www-data /var/www/html
```

1.34 Set Secure File Permissions

```
ayaail72@ayaaul72:~$ sudo chmod -R 755 /var/www/html
```

1.35 Verify Permissions

```
drwxr-xr-x 13 www-data www-data  4096 Mar 11 14:11 phpmyadmin
```

Create a Database User with Minimal Privileges

Purpose

A separate database user with limited privileges is created for the website.
This improves security by ensuring that the web application can only access the data it needs and cannot modify database structure or user permissions.

1.36 Create a Website Database User

Open MariaDB :

```
ayaail72@ayaaul72:~$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
```

Then run this command :

```
MariaDB [(none)]> CREATE USER 'web_user'@'localhost' IDENTIFIED BY '12345';
Query OK, 0 rows affected (0.009 sec)
```

This creates a new database user that can log in from the local server.

1.37 Grant Limited Permissions

```
MariaDB [(none)]> GRANT SELECT, INSERT, UPDATE, DELETE ON test_db.* TO 'web_user'@'localhost';
Query OK, 0 rows affected (0.004 sec)
```

Apply the Changes

```
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)
```

Verify the User

```
MariaDB [(none)]> SELECT User, Host FROM mysql.user;
+-------------+-----------+
| User        | Host      |
+-------------+-----------+
| AyaAil72    | localhost |
| mariadb.sys | localhost |
| mysql       | localhost |
| root        | localhost |
| web_user    | localhost |
+-------------+-----------+
5 rows in set (0.002 sec)
```

PHP test file should now use this user instead of my IAM user.

```
ayaail72@ayaaul72:~$ sudo nano /var/www/html/database_test.php
  GNU nano 7.2                                /var/www/html/database_test.php *
sudo nano /var/www/html/database_test.php<?php

$servername = "localhost";
$username = "web_user";
$password = "12345";
$dbname = "test_db";
```

Enabling HTTPS with a Self-Signed Certificate

Purpose

A self-signed SSL certificate is created to enable secure HTTPS communication between the web server and clients. HTTPS encrypts data transmitted between the browser and the server, improving security.

1.38 Enable Apache SSL Module

```
ayaai172@ayaau172:~$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
```

This command enables the SSL module for Apache, allowing the server to handle encrypted HTTPS connections.

Restart Apache

```
ayaai172@ayaau172:~$ sudo systemctl restart apache2
```

1.39 Create a Self-Signed Certificate

```
ayaai172@ayaau172:~$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/apache-selfsigned.key \
-out /etc/ssl/certs/apache-selfsigned.crt
....+.+.....+..........+.....+.+...+..+.........++++++++++++++++++++++++++++++++++++++++++++++++++++++++
.+....+..........+.+.....+...+..+.........+.+..+.............+.+.....+....+...+..+...+....+..
.....+..+...+.......+.........................+...+.......+......+..+.........+....+..+..........
..+.....+..........+.....+...+....+............+.+.........+......+.....+.+.....+..........+...
...+............+.+.........+...+..+..........+...+........+.........+.+............+.+.....+
```

```
Country Name (2 letter code) [AU]:LU
State or Province Name (full name) [Some-State]:LU
Locality Name (eg, city) []:LU
Organization Name (eg, company) [Internet Widgits Pty Ltd]:LPEM
Organizational Unit Name (eg, section) []:LPEM
Common Name (e.g. server FQDN or YOUR name) []:192.168.4.122
Email Address []:AyaAi172@school.lu
```

1.40 Enable Default SSL Virtual Host

```
ayaai172@ayaau172:~$ sudo a2ensite default-ssl.conf
Site default-ssl already enabled
```

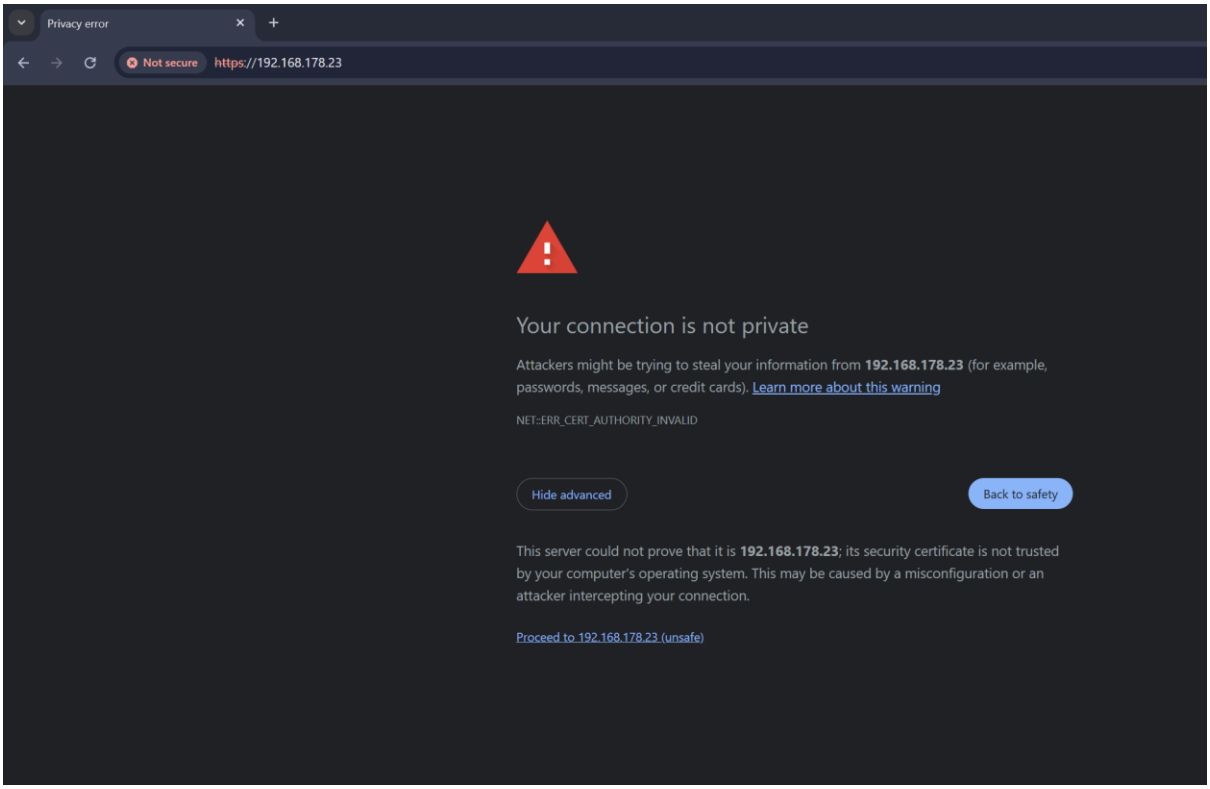
This activates Apache's default HTTPS configuration.

Restart Apache Again

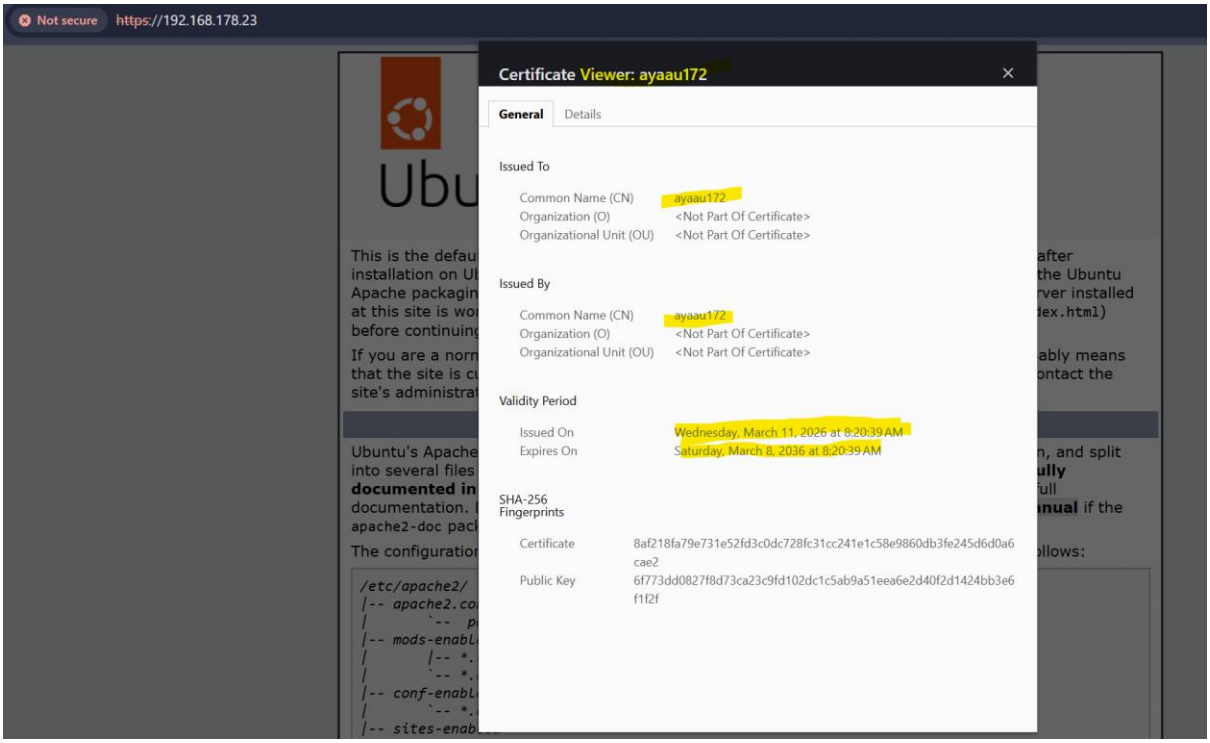
```
ayaai172@ayaau172:~$ sudo systemctl restart apache2
```

1.41 Test HTTPS

This is when I landed to the page:



And this is Self-Signed Certificate:



1.42 Redirect HTTP to HTTPS

Edit the Apache configuration, by adding this line inside your <VirtualHost *:80> block:

```
ayaai172@ayaau172:~$ sudo nano /etc/apache2/sites-available/000-default.conf
  GNU nano 7.2
<VirtualHost *:80>

Redirect "/" "https://192.168.178.23/"
```

Save and Exit.

Restart Apache Again

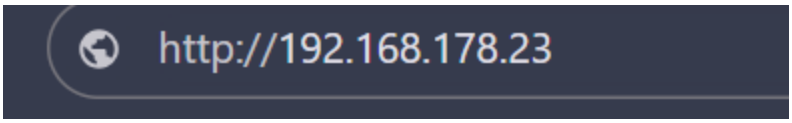
```
ayaai172@ayaau172:~$ sudo systemctl restart apache2
```

Now when visiting:

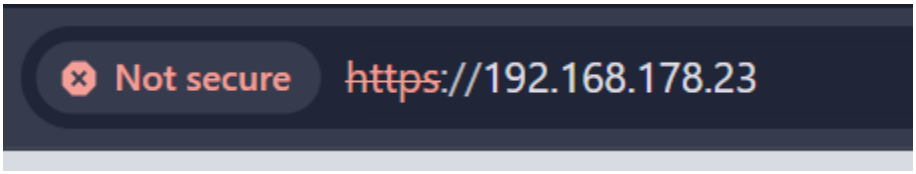
http://192.168.4.122

the browser will automatically redirect to:

https://192.168.4.122



After I clicked enter it redirected me to:



L5 – DataTransfer

er Story	L5 – Data Transfer		Calendar week 12
ormal description	As the operator of Portable Indoor Feedback		
	... I would like a functioning script for data transfer		
	... so that the stations purchased by my customers can transfer their measured data to the central database.		

quirements according to the work order	**Must**
	• A PHP script for data transfer has been created.
	• Serial number, timestamp, and all measured values are transferred correctly and stored in the database.
	• The data transfer script is accessible on the installed web and database server.
	• Documentation:
	  ○ Annotated source code.
	  ○ User documentation explaining how the script works.
	Should
	• The PHP script contains a form that can be used to manually enter measurement data.
	• The form has an appealing design and is user-friendly. Entered data is validated.
	Could
	• The script is extended to include the option of querying data.		

Tasks according to the work order			

- Create a PHP script.
- Write code for transferring values using HTTP POST, which complies with the requirements of the work order.
- Save data correctly in the database.
- Transfer the script to the installed web and database server and test it.

Purpose

The goal of this user story is to send measurement data from the station to the server and save it in the project database.

Define the transferred data

First, I checked which values need to be sent from measure.py to the server. The values are station serial number, timestamp, temperature, humidity, pressure, light, and gas..

I checked the measure.py file to identify the transferred values.

```
# Function for sending the data to the server
def send(host, path, station_serial, time, temperature, humidity, pressure, light, gas)
    data = {
        'station_serial': station_serial,
        'timestamp': time,
        'temperature': temperature,
        'humidity': humidity,
        'pressure': pressure,
        'light': light,
        'gas': gas
    }
```

Transfer the project files to the VM

I cloned my website project from GitHub to the VM so I could use the project files directly on the server.

```
ayaail72@ayaaul72:~$ cd /var/www/html
ayaail72@ayaaul72:/var/www/html$ sudo git clone https://github.com/AyaAil72/RPIF1.git
[sudo] password for ayaail72:
Cloning into 'RPIF1'...
remote: Enumerating objects: 196, done.
remote: Counting objects: 100% (196/196), done.
remote: Compressing objects: 100% (134/134), done.
remote: Total 196 (delta 54), reused 186 (delta 44), pack-reused 0 (from 0)
Receiving objects: 100% (196/196), 3.16 MiB | 19.35 MiB/s, done.
Resolving deltas: 100% (54/54), done.
ayaail72@ayaaul72:/var/www/html$
```

Verify:

```
ayaail72@ayaaul72:/var/www/html/RPIF1$ ls
admin  database  public  user
```

Import the database

Next, I imported the SQL file from the project into MariaDB on the VM. This created the project database and the required tables. After the import, I checked the database tables.

We open MariaDB as root:

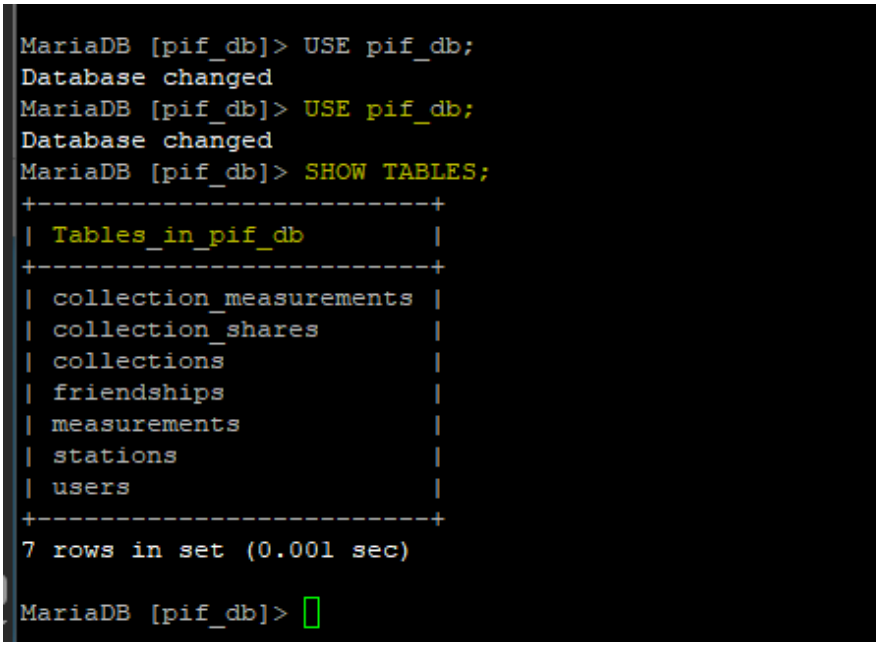
```
ayaail72@ayaaul72:~$ sudo mysql -u root
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 39
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Then run this command to import the database from the correct source:

```
MariaDB [(none)]> SOURCE /var/www/html/RPIF1/database/L2_Database.sql;
Query OK, 1 row affected (0.001 sec)

Database changed
Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected, 1 warning (0.052 sec)

Query OK, 0 rows affected, 1 warning (0.032 sec)

Query OK, 0 rows affected, 1 warning (0.032 sec)
```

```
Query OK, 0 rows affected (0.094 sec)

Query OK, 3 rows affected (0.018 sec)
Records: 3  Duplicates: 0  Warnings: 0

Query OK, 7 rows affected (0.025 sec)
Records: 7  Duplicates: 0  Warnings: 0
```

Verify:



Check the database tables

Next, I checked the **stations** and **measurements** tables in the project database. I verified that the station data already exists in the **stations** table and that the **measurements** table is ready to store the transferred values.

```
MariaDB [(none)]> USE pif_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [pif_db]> DESCRIBE stations;
+---------------+------------------+------+-----+---------+----------------+
| Field         | Type             | Null | Key | Default | Extra          |
+---------------+------------------+------+-----+---------+----------------+
| station_id    | int(10) unsigned | NO   | PRI | NULL    | auto_increment |
| serial_number | varchar(50)      | NO   | UNI | NULL    |                |
| name          | varchar(150)     | NO   |     | NULL    |                |
| description   | text             | YES  |     | NULL    |                |
| user_id       | int(10) unsigned | YES  | MUL | NULL    |                |
+---------------+------------------+------+-----+---------+----------------+
5 rows in set (0.001 sec)

MariaDB [pif_db]> DESCRIBE measurements;
+----------------+---------------------+------+-----+---------+----------------+
| Field          | Type                | Null | Key | Default | Extra          |
+----------------+---------------------+------+-----+---------+----------------+
| measurement_id | bigint(20) unsigned | NO   | PRI | NULL    | auto_increment |
| station_id     | int(10) unsigned    | NO   | MUL | NULL    |                |
| measured_at    | datetime            | NO   |     | NULL    |                |
| temperature    | decimal(5,2)        | YES  |     | NULL    |                |
| humidity       | decimal(5,2)        | YES  |     | NULL    |                |
| pressure       | decimal(7,2)        | YES  |     | NULL    |                |
| light          | int(11)             | YES  |     | NULL    |                |
| gas            | int(11)             | YES  |     | NULL    |                |
+----------------+---------------------+------+-----+---------+----------------+
8 rows in set (0.001 sec)

MariaDB [pif_db]> SELECT * FROM stations;
+------------+---------------+-----------+-------------------+---------+
| station_id | serial_number | name      | description       | user_id |
+------------+---------------+-----------+-------------------+---------+
|          1 | ST-4004-001   | Station 1 | Available station |    NULL |
|          2 | ST-4004-002   | Station 2 | Available station |    NULL |
|          3 | ST-4004-003   | Station 3 | Available station |    NULL |
+------------+---------------+-----------+-------------------+---------+
3 rows in set (0.000 sec)

MariaDB [pif_db]>
```

Create transfer.php

Next, I created the transfer.php script in the web server directory. This script receives the transmitted measurement data with HTTP POST, checks the station serial number, and stores the values in the measurements table.

```
ayaail72@ayaaul72:/var/www/html/RPIF1/public$ sudo nano transfer.php
  GNU nano 7.2                                                transfer.php
<?php
$host = "localhost";
$dbname = "pif_db";
$username = "AyaAil72";
$password = "LPEM";

if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    die("Only POST requests are allowed.");
}
```

Comment the PHP script:

Next, I added comments to the transfer.php file. This makes the source code easier to understand and explains the main parts of the script, such as receiving POST data, checking the station, and storing the values in the database.

```
 GNU nano 7.2                                    transfer.php *
<?php
// Database connection settings
$host = "localhost";
$dbname = "pif_db";
$username = "AyaAi172";
$password = "LPEM";

// Only allow POST requests
if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    die("Only POST requests are allowed.");
}

// Read transferred values from the POST request
$station_serial = $_POST['station_serial'] ?? '';
$timestamp      = $_POST['timestamp'] ?? '';
$temperature    = $_POST['temperature'] ?? null;
$humidity       = $_POST['humidity'] ?? null;
$pressure       = $_POST['pressure'] ?? null;
$light          = $_POST['light'] ?? null;
$gas            = $_POST['gas'] ?? null;

// Check that required values are present
if (empty($station_serial) || empty($timestamp)) {
    die("Missing required data.");
}

try {
    // Create connection to MariaDB
    $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Find the station_id for the received serial number
    $stmt = $pdo->prepare("SELECT station_id FROM stations WHERE serial_number = ?");
    $stmt->execute([$station_serial]);
    $station = $stmt->fetch(PDO::FETCH_ASSOC);

    // Stop if the station does not exist
    if (!$station) {
        die("Station not found.");
    }

    $station_id = $station['station_id'];

    // Insert the received measurement values into the measurements table
    $stmt = $pdo->prepare("
        INSERT INTO measurements
        (station_id, measured_at, temperature, humidity, pressure, light, gas)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ");

    $stmt->execute([
        $station_id,
        $timestamp,
        $temperature,
        $humidity,
        $pressure,
        $light,
        $gas

} catch (PDOException $e) {
    // Show an error message if database access fails
    die("Database error: " . $e->getMessage());
}
?>
```

Test the transfer script

After updating the database access in transfer.php, I tested the script by sending sample measurement data with curl. Then I checked the measurements table in MariaDB to verify that the values were stored correctly.

```
ayaail72@ayaaul72:/var/www/html/RPIF1/public$ curl -k -L -X POST https://localhost/RPIF1/public/transfer.php \
-d "station_serial=ST-4004-001" \
-d "timestamp=2026-03-20 14:45:00" \
-d "temperature=23.5" \
-d "humidity=50" \
-d "pressure=1005.2" \
-d "light=120" \
-d "gas=450"
Data stored successfully.ayaail72@ayaaul72:/var/www/html/RPIF1/public$
```

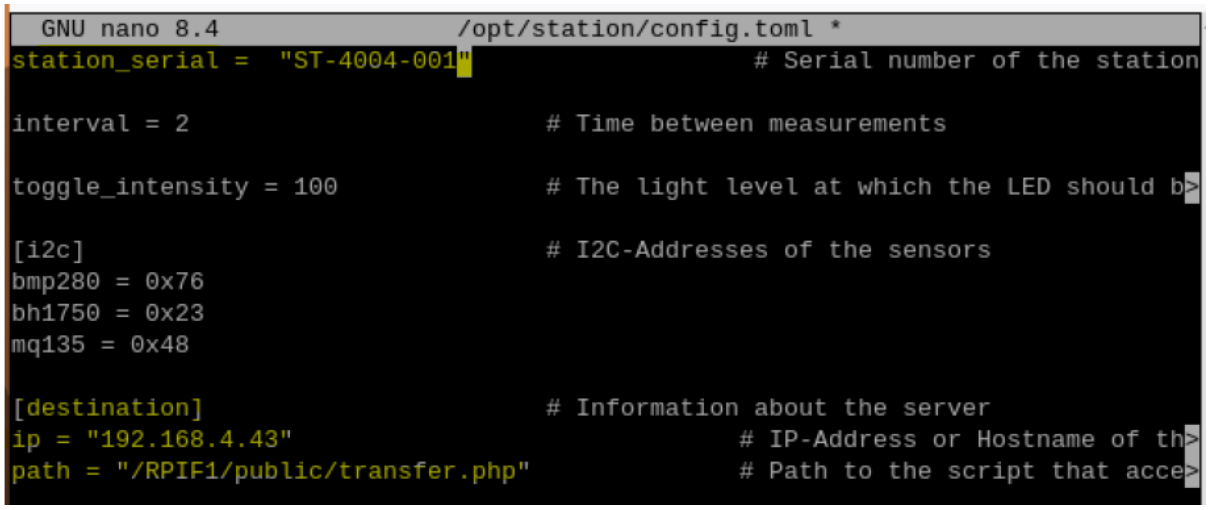
Verify:

```
Database changed
MariaDB [pif_db]> SELECT * FROM measurements;
+----------------+------------+---------------------+-------------+----------+----------+-------+------+
| measurement_id | station_id | measured_at         | temperature | humidity | pressure | light | gas  |
+----------------+------------+---------------------+-------------+----------+----------+-------+------+
|              1 |          1 | 2026-01-18 09:00:00 |       21.50 |    45.20 |  1012.30 |   320 |  410 |
|              2 |          1 | 2026-01-18 10:00:00 |       22.10 |    44.10 |  1012.10 |   500 |  420 |
|              3 |          1 | 2026-01-18 11:00:00 |       22.90 |    43.50 |  1011.80 |   650 |  430 |
|              4 |          1 | 2026-01-18 12:00:00 |       23.40 |    42.90 |  1011.40 |   720 |  440 |
|              5 |          1 | 2026-01-18 13:00:00 |       23.10 |    43.20 |  1011.60 |   680 |  435 |
|              6 |          1 | 2026-01-18 14:00:00 |       22.60 |    44.00 |  1011.90 |   540 |  425 |
|              7 |          2 | 2026-01-18 15:00:00 |       22.00 |    45.10 |  1012.40 |   400 |  415 |
|              8 |          1 | 2026-03-20 14:45:00 |       23.50 |    50.00 |  1005.20 |   120 |  450 |
+----------------+------------+---------------------+-------------+----------+----------+-------+------+
8 rows in set (0.000 sec)
```

Connect measure.py to the VM.

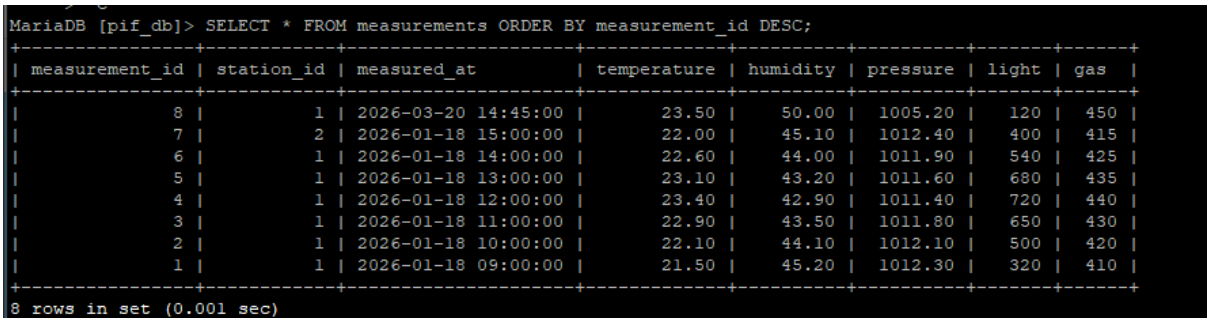
Next, I adjusted the station configuration so that measure.py sends the measurement data to the VM. I entered the server IP address, the path to transfer.php, and the correct station

serial number.



Verify the transferred data

Finally, I checked the measurements table again in the pif_db database. The transferred values were visible in the table, which confirmed that the data transfer was successful.



L6 – BackupServer

Purpose:

The purpose of this user story is to ensure that the project database is backed up regularly to a separate backup server. This allows the data to be restored in case of data loss or system failure on the web and database server.

Setup and verification of the backup server

A separate virtual machine was used as a backup server.
The system was checked to ensure it is properly connected to the network.
The hostname and IP address were verified, and the SSH service was confirmed to be running.
This ensures that the backup server can communicate with the web and database server.

```
ayaai172@ayaai172:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-04-16 09:29:21 UTC; 3s ago
TriggeredBy: ● ssh.socket
```

```
ayaai172@ayaai172:~$ hostname
ayaai172
ayaai172@ayaai172:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.4.98  netmask 255.255.254.0  broadcast 192.168.5.255
        inet6 fe80::a00:27ff:fec4:fe06  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:c4:fe:06  txqueuelen 1000  (Ethernet)
        RX packets 28194  bytes 19295597 (19.2 MB)
        RX errors 0  dropped 2453  overruns 0  frame 0
        TX packets 1576  bytes 134219 (134.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Installation of required software

Install MySQL client and FTP server:

```
ayaai172@ayaai172:~$ sudo apt install mysql-client vsftpd -y
Reading package lists... Done
Building dependency tree... Done
```

Verify installation:

MySQL:

```
ayaai172@ayaai172:~$ mysql --version
mysql  Ver 8.0.45-0ubuntu0.24.04.1 for Linux on x86_64 ((Ubuntu))
```

FTP server:

```
ayaail72@ayaail72:~$ systemctl status vsftpd
● vsftpd.service - vsftpd FTP server
     Loaded: loaded (/usr/lib/systemd/system/vsftpd.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-04-16 12:54:00 UTC; 1min 7s ago
    Process: 1662 ExecStartPre=/bin/mkdir -p /var/run/vsftpd/empty (code=exited, status=0/SUCCESS)
   Main PID: 1664 (vsftpd)
      Tasks: 1 (limit: 16206)
```

On the backup server, the required software was installed.
The MySQL client was installed to allow remote access to the database server and perform backups.
Additionally, an FTP server (vsftpd) was installed to provide access to the backup files.
After installation, both services were verified to ensure they are working correctly.

Configuration of the database server for remote access

Part 1 – Allow MySQL remote access

On the web/database server

Open MySQL config:

```
ayaail72@ayaaul72:~$ sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
[sudo] password for ayaail72:
  GNU nano 7.2                          /etc/mysql/mariadb.conf.d/50-server.cnf
#
# These groups are read by MariaDB server.
# Use it for options that only the server (but not clients) should see

# this is read by the standalone daemon and embedded servers
[server]

# this is only for the mysqld standalone daemon
[mysqld]

#
# * Basic Settings
#

#user                   = mysql
pid-file                = /run/mysqld/mysqld.pid
basedir                 = /usr
#datadir                = /var/lib/mysql
#tmpdir                 = /tmp

# Broken reverse DNS slows down connections considerably and name resolve is
# safe to skip if there are no "host by domain name" access grants
#skip-name-resolve

# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address            = 127.0.0.1
```

Change the bind-address to 0.0.0.0:

```
bind-address            = 0.0.0.0
```

Restart MySQL:

```
ayaail72@ayaaul72:~$ sudo systemctl restart mariadb
```

Part 2 – Create backup user

Log into MariaDB:

```
ayaail72@ayaaul72:~$ sudo mysql -u root
Welcome to the MariaDB monitor.  Commands end with ; or \g.
```

Create user using the backup server IP and give permissions:

```
MariaDB [(none)]> CREATE USER 'backup_user'@'192.168.4.98' IDENTIFIED BY 'Backup_1';
Query OK, 0 rows affected (0.007 sec)
```

```
MariaDB [(none)]> GRANT ALL PRIVILEGES ON pif_db.* TO 'backup_user'@'192.168.4.98';
Query OK, 0 rows affected (0.005 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)
```

The database server was configured to allow connections from external systems.
The bind address in the MariaDB configuration file was changed to allow connections from all network interfaces.
After restarting the database service, a new MySQL user was created specifically for the backup server.
This user was granted access to the project database and restricted to connections from the backup server's IP address.

Create backup directory on backup server

Create backup directory:

```
ayaail72@ayaail72:~$ sudo mkdir -p /backup
[sudo] password for ayaail72:
```

```
ayaail72@ayaail72:~$ sudo chmod 755 /backup
```

Verify:

```
ayaail72@ayaail72:~$ ls -ld /backup
drwxr-xr-x 2 root root 4096 Apr 16 13:14 /backup
```

A directory was created on the backup server to store the database backups.
The directory permissions were set to ensure that the backup files can be stored and accessed securely.

Create the backup script:

Create the script on the backup server:

```
ayaail72@ayaail72:~$ nano /backup/backup.sh
  GNU nano 7.2                                  /backup/backup.sh *
#!/bin/bash

DB_HOST="192.168.4.57"
DB_USER="backup_user"
DB_NAME="pif_db"
BACKUP_DIR="/backup"
DATE=$(date +"%Y-%m-%d_%H-%M-%S")
BACKUP_FILE="$BACKUP_DIR/${DB_NAME}_$DATE.sql"

mysqldump -h "$DB_HOST" -u "$DB_USER" -p "$DB_NAME" > "$BACKUP_FILE"
```

Make it executable:

```
ayaail72@ayaail72:~$ sudo chmod +x /backup/backup.sh
```

Run the script:

```
ayaail72@ayaail72:~$ sudo /backup/backup.sh
Enter password:
-- Warning: column statistics not supported by the server.
```

Check the backup:

```
ayaail72@ayaail72:~$ ls /backup
backup.sh                    pif_db_2026-04-16_13-27-55.sql  pif_db_2026-04-16_13-32-22.sql
pif_db_2026-04-16_13-27-47.sql  pif_db_2026-04-16_13-28-02.sql
```

A shell script was created on the backup server to export the pif_db database using the mysqldump utility.
The script was executed manually to verify its functionality, and SQL backup files were successfully created in the backup directory.
The filenames include timestamps to distinguish between different backup versions.

Automation of the backup process

On the **backup server**:

Open cron editor:

```
ayaail72@ayaail72:~$ crontab -e
no crontab for ayaail72 - using an empty one
```

Add this line:

```
0 8 * * * sudo /backup/backup.sh
```

Verify:

```
ayaail72@ayaail72:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
0 8 * * * sudo /backup/backup.sh
# For more information see the manual pages of crontab(5) and cron(8)
#
```

The backup script was scheduled to run automatically using a cron job.
The cron job was configured to execute the backup script every day at 08:00, ensuring regular backups without manual intervention.

Configuration of FTP access

Edit FTP configuration:

- Change / ensure these settings are enabled:

```
# Allow anonymous FTP? (Disabled by default).
anonymous_enable=NO
#
# Uncomment this to allow local users to log in.
local_enable=YES
#
# Uncomment this to enable any form of FTP write command.
write_enable=YES
```

```
# You may restrict local users to their home directories.  See the FAQ for
# the possible risks in this before using chroot_local_user or
# chroot_list_enable below.
chroot_local_user=YES
```

- Restart FTP:

```
ayaail72@ayaail72:~$ sudo systemctl restart vsftpd
```

Create FTP user and set a password and username:

```
ayaail72@ayaail72:~$ sudo adduser ftpuser
info: Adding user `ftpuser' ...
```

```
    Full Name []: FTPuser
```

Give access to backup folder:

```
ayaail72@ayaail72:~$ sudo /backup/backup.sh
```

Test FTP connection:

```
ayaai172@ayaai172:~$ ftp 192.168.4.98
Connected to 192.168.4.98.
220 (vsFTPd 3.0.5)
Name (192.168.4.98:ayaai172): ftpuser
331 Please specify the password.
Password:
230 Login successful.
```

```
ftp> ls
229 Entering Extended Passive Mode (|||39099|)
150 Here comes the directory listing.
drwxr-xr-x    2 1001     1001         4096 Apr 17 11:08 backup
226 Directory send OK.
```

```
ftp> cd backup
250 Directory successfully changed.
```

```
ftp> ls
229 Entering Extended Passive Mode (|||46432|)
150 Here comes the directory listing.
-rw-r--r--    1 0        0               0 Apr 17 11:07 pif_db_2026-04-17_11-07-44.sql
-rw-r--r--    1 0        0               0 Apr 17 11:08 pif_db_2026-04-17_11-08-08.sql
-rw-r--r--    1 0        0               0 Apr 17 11:08 pif_db_2026-04-17_11-08-12.sql
-rw-r--r--    1 0        0               0 Apr 17 11:08 pif_db_2026-04-17_11-08-29.sql
-rw-r--r--    1 0        0               0 Apr 17 11:08 pif_db_2026-04-17_11-08-34.sql
-rw-r--r--    1 0        0               0 Apr 17 11:08 pif_db_2026-04-17_11-08-46.sql
226 Directory send OK.
```